

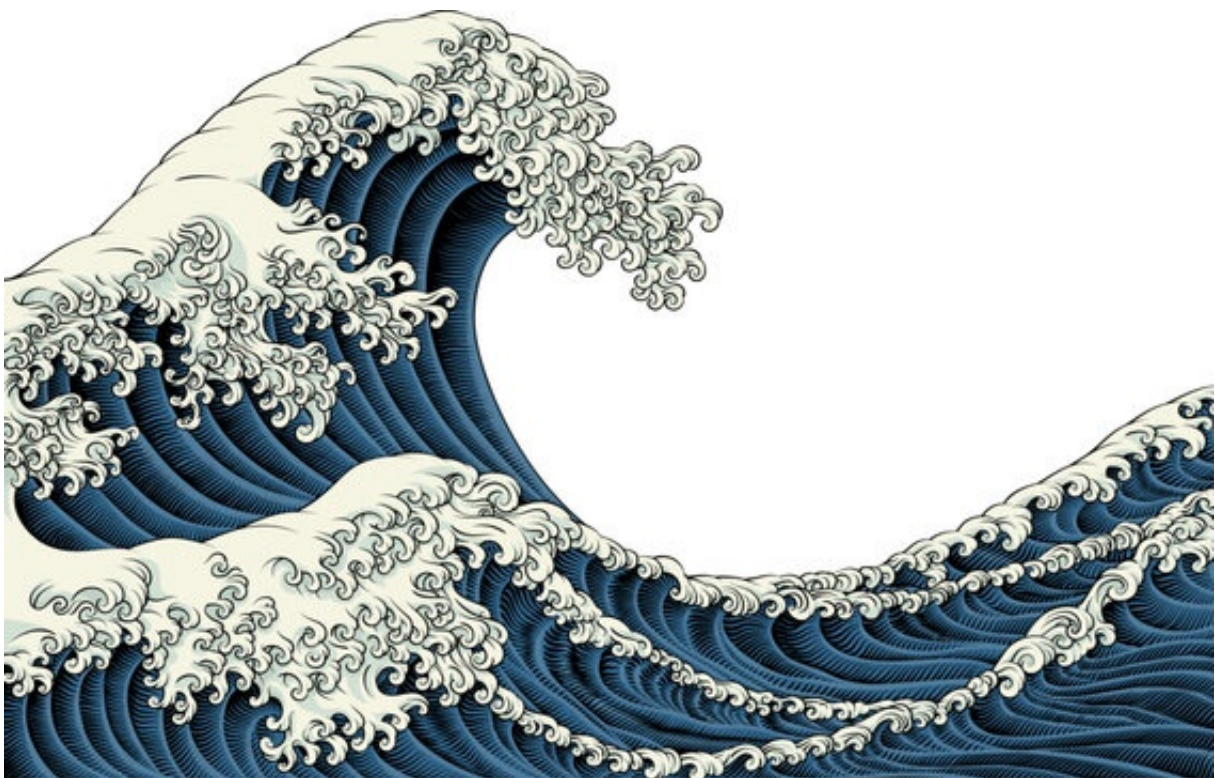
pyCoastal: a Python Tool for Coastal Engineering

Stefano Biondi

University of Florida

User manual and reference, version 0.2.0

September 2026



Contents

Preface	5
I Getting started	6
1 Introduction	6
1.1 The design chain	6
1.2 Principles	7
1.3 Before the math	7
1.4 How this manual is organized	7
2 Installation and first steps	8
2.1 Requirements	8
2.2 Installing	8
2.3 Repository layout	9
2.4 Conventions	9
2.5 Quick start	10
2.6 Reading case files	10
II Numerical framework	11
3 Grids, operators, and time integration	11
3.1 Grids	11
3.2 Finite differences	11
3.3 Time integration	13
3.4 Boundary conditions	14
4 Governing equations and closures	15
4.1 Shallow water equations	15
4.2 Incompressible Navier-Stokes	16
4.3 Poisson solver	16
4.4 Turbulence closures	17
5 Simulation examples	17
5.1 Case files	18
5.2 Water drop: the 2D wave equation	18
5.3 Regular waves: <code>waves2D.py</code>	20
5.4 Irregular waves: a numerical flume	20
5.5 Current and pollutant: advection and diffusion	20
5.6 Viscous fluid: velocity decay	20
5.7 Boundary-condition test: <code>2D_irr_turb.py</code>	21
5.8 Equilibrium shoreline behind detached breakwaters	21
5.9 Diagnostics, verification, and validation	21
5.10 Extending the framework	21
5.11 Limitations of the numerical framework	22
III Engineering tools	23

6	Standalone formulae: <code>pyCoastal.tools</code>	23
6.1	Waves: <code>tools.wave</code>	23
6.2	Sediment transport: <code>tools.sediment_transport</code>	24
6.3	Structures and runoff: <code>tools.structural</code>	24
6.4	One-line shoreline model: <code>tools.shoreline</code>	24
6.5	Morphodynamics helpers: <code>tools.morphodynamics</code>	25
IV	Design applications	26
7	Design conditions from a record	26
7.1	Two routes	26
7.2	Theory	26
7.3	Using the module	27
7.4	Worked example	27
7.5	Limits	28
8	Sediment and soil	28
8.1	The material catalogue	30
8.2	Mobility	30
8.3	Earth pressure	31
9	Rubble-mound breakwaters	31
9.1	The design condition	31
9.2	Armor stability	32
9.3	Overtopping	32
9.4	Designing a section	33
9.5	Worked example	34
10	Vertical seawalls	34
10.1	Goda pressures	36
10.2	Scour, toe, and stability relations	36
10.3	Designing a wall	37
10.4	Worked example	37
11	Harbor agitation and port layout	38
11.1	The solver	38
11.2	Geometry	40
11.3	Running a simulation	41
11.4	Worked examples	41
12	Wave loads on piles	43
12.1	The Morison equation	43
12.2	Scour at a pile	43
12.3	Worked example	44
13	Scour at piers and bridges	44
13.1	Pier scour in combined waves and current	45
13.2	Total bridge scour	47
13.3	What is not modeled	50

14	Open-channel hydraulics and backwater	50
14.1	The channel	52
14.2	Two depths	52
14.3	Gradually varied flow	52
14.4	Afflux at bridge piers	52
14.5	Flow distribution	53
14.6	Worked example	53
15	Navigation channels	54
15.1	The vessel	54
15.2	The depth chain	54
15.3	The width chain	55
15.4	Side slopes, volumes, and turning basins	55
15.5	Worked example	55
16	Berthing energy and fenders	56
16.1	The energy chain	58
16.2	Fender, panel, and spacing	58
16.3	Worked example	59
17	Beach nourishment	60
17.1	The planform: how long a fill lasts	61
17.2	The profile: what a borrow source is worth	62
17.3	Limits	63
18	Storm surge and coastal flooding	66
18.1	The water level budget	66
18.2	Flood mapping	66
18.3	Worked example	67
V	Drawings and deliverables	69
19	Drawings, sections, and sheets	69
19.1	Drafting primitives	69
19.2	Sections for each design	70
19.3	The drawing sheets	71
19.4	Plot helpers	71
VI	PyCoaTools	78
20	The browser app	78
20.1	How it works	78
20.2	Modules and inputs	78
20.3	Theory and sources	82
20.4	Verification and review tools	85
VII	PyCoaPedia and agentic retrieval	86

21 PyCoaPedia: the knowledge base	86
21.1 What is in it	86
21.2 How it is built	86
21.3 Navigating the knowledge	87
21.4 The design modules and their topics	88
21.5 Coverage and its limits	89
22 The repository for AI agents	89
22.1 Entry points	90
22.2 Querying the literature	90
22.3 Keeping it current	90
VIII Quality assurance	92
23 The test suite	92
23.1 What the tests check	92
23.2 Tests by file	92
23.3 The browser engine	93
Appendices	94
A List of symbols	94
B Engineering references	95

Preface

pyCoastal started as the culmination of my academic journey in numerical modeling, fluid dynamics, and hydraulics. Students often believe that numerical models are overwhelmingly complex tools; however, their structure follows recurrent patterns common to many modeling frameworks. When properly configured and validated, these models can reproduce real-world phenomena, provide insight, and let us explore scenarios beyond direct observation. Some researchers prefer to work in the analytical domain, which has its own advantages and limitations, and I admire those who can capture the complexity of natural processes through concise formulations that, even if simplified, reflect the essence of how the world operates.

Since the first edition of this manual the package has grown in a different direction as well. Next to the numerical framework it now carries a complete design chain: a wave record becomes a design condition, the design condition sizes a structure, and the structure comes out as a dimensioned drawing with its quantities and its specification notes. Every relation names its source, states its range of validity, and says so when it is pushed outside that range. A browser version of the design modules, PyCoaTools, puts each result next to what the peer-reviewed literature says about the ground it stands on, through PyCoaPedia, the package's knowledge base of the coastal and ocean engineering literature.

This manual documents all of it. Part I gets the package installed and running. Parts II and III describe the numerical framework and the standalone engineering formulae. Part IV is the core of the book: one chapter per design application, each with its theory, its interface, a worked example with the output the code actually prints, and the limits of the method. Part V covers drawings and deliverables and Part VI the PyCoaTools browser app. Part VII explains PyCoaPedia, how it is built and how to navigate it from a browser, from Python, from the shell and in SQL, and how the repository is laid out for AI agents. Part VIII describes the test suite that keeps all of it honest.

I do not expect pyCoastal to be the tool behind groundbreaking discoveries, although I hope it may contribute to them, or at least serve as a starting point for someone.

Stefano Biondi

Part I

Getting started

1 Introduction

Coastal zones concentrate people, infrastructure, and ecosystems in environments that are energetic and constantly changing. Waves, tides, surges, and currents interact with complex shorelines to drive erosion, flooding, sediment transport, and loads on structures. Anticipating these processes is essential for safe navigation, coastal protection, habitat conservation, and climate adaptation.

pyCoastal is a Python toolbox for coastal, port, and ocean engineering. It has two faces that share one code base:

1. **A numerical framework** of uniform grids, finite-difference operators, explicit time integrators, boundary handlers, Poisson solvers, and physics building blocks (shallow water, incompressible Navier-Stokes, eddy-viscosity closures). It favors clarity over ultimate efficiency so that students and practitioners can see and modify every numerical brick.
2. **A design chain** of scenario-level applications that take an engineering design and report the quantities a project actually turns on: a 100-year wave with its confidence band, an armor stone size and a crest level, a seawall base width, a scour depth over a tidal cycle, a dredge level, a fender size, a flood extent, and the drawing sheet that carries all of it.

1.1 The design chain

A real coastal scheme runs as a chain, and pyCoastal is organized the same way (Table 1). Each step hands its result to the next as an object, not as a number you retype, so a drawing cannot drift out of step with the calculation behind it.

Table 1: The design chain and the modules that implement each step.

Step	Module	What it gives you
Design condition	<code>applications.extremes</code>	100-year wave or water level, with a confidence band
Bed and soil	<code>applications.sediment</code>	grain mobility, fall velocity, earth pressure
Nearshore	<code>applications.port</code>	phase-resolved diffraction into a harbor, berth agitation
Structure	<code>applications.structures</code> , <code>applications.seawall</code>	armor size, crest level, stability checks
Loads	<code>applications.piles</code> , <code>applications.berthing</code>	Morison loads through the wave cycle, berthing energy and fenders
Scour	<code>applications.scour</code> , <code>applications.river</code>	contraction, pier and abutment scour over the tidal cycle; the flow split that feeds it
Navigation	<code>applications.channel</code>	dredge level, channel width, dredge volume
Coastline	<code>applications.nourishment</code> , <code>applications.surge</code>	fill life and profile, water level budget and a connected flood map

Step	Module	What it gives you
Deliverable	<code>drafting, applications.sections</code>	a dimensioned drawing sheet and a DXF

1.2 Principles

Three rules run through the whole package, and they explain most of the design decisions you will meet in the later chapters.

Every relation names its source. Module docstrings list their sources (Van der Meer 1988, EurOtop 2018, Goda 2010, PIANC 2014, HEC-18, and so on), and each function says which equation it implements.

Nothing silently extrapolates. Where a formula has a documented range of validity, the functions compute outside it if asked, because engineering judgement outside a range is the engineer’s call, but they say so through a `warnings` list, a flag such as `within_range`, `impulsive`, or `screening_only`, or a note in the returned dictionary.

Assumptions are reported, not buried. Width components of a channel that you did not specify are taken at their most benign class and reported as assumed. The +1 safety term in Froehlich’s abutment equation is reported as a separate `safety_margin`. The steady-current floor added to the Sumer and Fredsoe scour relation is flagged as `current_governs`. A reviewer should always be able to see which assumption to argue with.

1.3 Before the math

A few terms recur throughout the numerical parts of this manual. A *field* is any variable defined over the domain, such as water depth, velocity, or pressure. A *grid* divides the domain into cells where these variables are stored; in `pyCoastal` the values live at cell centers. A *stencil* is the small pattern of neighboring cells used to approximate derivatives through finite differences, and these approximations form the spatial *operators*. Once the spatial terms are discretized, a *time integrator* updates the solution step by step, using a right-hand side that gathers fluxes, advection, diffusion, pressure gradients, and sources. *Boundary conditions* specify what happens at the domain edges.

On the design side, a *design condition* is the wave and water level a structure is sized against, a *design object* (for example `SeawallDesign` or `ChannelDesign`) is the fully dimensioned result of a design function, and a *sheet* is a drawing at a true, stated scale with a border, a title block, and specification notes.

1.4 How this manual is organized

- **Part I** covers installation, the repository layout, conventions, and a quick start.
- **Part II** describes the numerical framework and the governing equations, with the simulation examples that exercise them.
- **Part III** documents the standalone engineering formulae in `pyCoastal.tools`.
- **Part IV** has one chapter per design application.
- **Part V** covers the drafting layer: sections, sheets, and DXF export.
- **Part VI** describes `PyCoaTools`, the browser version of the design modules.
- **Part VII** describes `PyCoaPedia`, the knowledge base of the coastal and ocean engineering literature, how to navigate it, and how the repository serves AI agents.

- **Part VIII** describes the test suite and the verification of the browser engine against the Python.
- **Appendices** give a list of symbols and the engineering references.

2 Installation and first steps

2.1 Requirements

pyCoastal needs Python 3.10 or newer. The runtime dependencies are kept to two, so the design modules install anywhere:

- `numpy` ≥ 1.21 for arrays, gradients, and linear algebra;
- `PyYAML` ≥ 5.4 for YAML case files.

Everything else is optional and imported lazily, only by the functions that need it (Table 2).

Table 2: Optional dependency groups declared in `pyproject.toml`.

Extra	Installs	Needed for
<code>sparse</code>	<code>scipy</code> ≥ 1.7	sparse direct solver in <code>physics.poisson</code>
<code>vtk</code>	<code>vtk</code> ≥ 9.0	<code>io.write_vtk</code>
<code>plots</code>	<code>matplotlib</code> ≥ 3.4	<code>drafting</code> , <code>applications.sections</code> , <code>plotting</code> , and the <code>examples</code>
<code>dev</code>	<code>pytest</code> , <code>scipy</code> , <code>matplotlib</code>	running the test suite
<code>docs</code>	<code>sphinx</code> , <code>sphinx-rtd-theme</code>	the Sphinx API pages in <code>docs/</code>

2.2 Installing

From PyPI:

```
pip install pyCoastal
pip install "pyCoastal[plots]"      # with drawings and plotting
```

From a clone, in editable mode, which is the right choice for development and for running the examples:

```
git clone https://github.com/stebiondi/pyCoastal.git
cd pyCoastal
python -m venv .venv
source .venv/bin/activate      # on Windows: .venv\Scripts\activate
pip install -e "[dev]"
```

Check the installation and run the tests:

```
python -c "import pyCoastal; print(pyCoastal.__version__)"
python -m pytest tests/
```

Continuous integration runs the same suite on Ubuntu and Windows under Python 3.10, 3.11, and 3.13 on every push to `main` and on every pull request (`.github/workflows/tests.yml`). A separate workflow builds and publishes the distribution to PyPI (`.github/workflows/publish.yml`).

2.3 Repository layout

pyCoastal/	
├── pyCoastal/	the package
│ ├── numerics/	grids, operators, time integrators, BCs, solver
│ ├── physics/	shallow water, Navier-Stokes, Poisson, turbulence
│ ├── tools/	standalone formulae: waves, sediment, structures, one-line shoreline, morphodynamics
│ ├── applications/	design modules (extremes, structures, seawall, port, piles, scour, river, channel, berthing, nourishment, surge, sediment, sections)
│ ├── drafting.py	drawing primitives, sheets, title blocks, DXF
│ ├── plotting.py	color maps and helpers for the examples
│ ├── config.py, io.py	case-file reading and VTK output
├── examples/	worked examples and their YAML configs
├── tests/	the pytest suite
├── pyCoastal/pedia/	PyCoaPedia reader and its SQLite database
├── pedia/	PyCoaPedia as Markdown, its schema, and the builder
├── webapp/	PyCoaTools and the PyCoaPedia explorer (browser)
├── media/	figures, animations, drawing sheets, DXF files
├── docs/	this manual (docs/manual), the API reference (docs/reference) and the example index
├── AGENTS.md, llms.txt	the repository map for AI agents
├── paper/	JOSS-style paper
└── pyCoastal manual.pdf	this manual

2.4 Conventions

The same conventions hold in every module. Where a module departs from them its docstring says so.

Coordinates. x and y span the horizontal plane and z is elevation, positive upward. In every 2D array x is axis 0 and y is axis 1, matching `UniformGrid`, which builds its coordinates with `indexing="ij"` and flattens as $i*ny + j$. In cross-shore work x is the chainage and in alongshore work (the one-line model) x is alongshore and y is the cross-shore shoreline offset, positive seaward.

Units. SI throughout: meters, seconds, kilograms, newtons. Forces on walls and walls' weights are kN per meter run; overtopping discharges are liters per second per meter (l/s/m), the unit of the tolerability tables; ship speeds are in knots where the formula is written in knots and every such function says so. Grain sizes are meters, not millimeters. Angles are degrees where a geotechnical or navigational input is conventionally stated in degrees, and radians in wave-direction and slope-angle arguments of the low-level tools; each function states which.

Levels. Levels are meters relative to chart datum (m CD) and increase upward. Depths and draughts are positive numbers. Pier scour drawings use the initial bed as datum, because every dimension there is a depth below it.

Wave parameters. Design wave heights are spectral significant heights H_{m0} at the toe of the structure. EurOtop relations use the spectral period $T_{m-1,0}$; if only the peak period is known, `DesignConditions.from_peak_period` converts with $T_{m-1,0} = T_p/1.1$. Goda's pressure method is written around the significant period, so the seawall module passes T_p to it.

Slopes. Slopes are given as $\cot \alpha$, the horizontal run per unit rise, because that is how they appear on drawings.

2.5 Quick start

A seawall from a design condition to a drawing sheet, in eight lines:

```
from pyCoastal.applications.structures import DesignConditions
from pyCoastal.applications.seawall import design_seawall
from pyCoastal.applications.sections import seawall_sheet

conditions = DesignConditions.from_peak_period(Hm0=2.8, Tp=9.5, depth=8.5)
wall = design_seawall(conditions, still_water_level=2.9, seabed_level=-5.6,
                      tolerable_use="trained_staff")
print(wall.summary())
sheet = seawall_sheet(wall, project="Bayfront promenade protection", size="A3")
sheet.save("seawall.png")      # 600 dpi, at a true scale stated on the sheet
sheet.to_dxf("seawall.dxf")    # the geometry on named layers, for CAD
```

The printed summary is the design report, in the order an engineer checks it (crest, founding level, base, wave force, safety factors, bearing). The full worked example is in Section 10.

Every example in `examples/` runs from the repository root:

```
python examples/seawall_section.py
python examples/breakwater_design.py
python examples/design_wave.py
```

and writes its figures to `media/`. `docs/examples.md` lists them all.

2.6 Reading case files

The numerical examples read their parameters from YAML files in `examples/configs/`. Two helpers parse case files into dictionaries:

- `pyCoastal.config.load_config(path)` reads `.yaml/.yml` (PyYAML), `.json`, and `.ini/.cfg` files;
- `pyCoastal.io.read_data(path)` does the same for any path-like object, and `pyCoastal.io.write_vtk(grid, filename)` writes a VTK grid object to a legacy `.vtk` file (needs the `vtk` extra).

`pyCoastal.numerics.domain.Domain(cfg)` builds a `Mesh1D` or `Mesh2D` from a domain: block with dimension, `x0`, `x1`, `nx` (and `y0`, `y1`, `ny` in 2D).

Part II

Numerical framework

3 Grids, operators, and time integration

Computational coastal engineering connects governing equations with the numerical choices that approximate them. `pyCoastal.numerics` exposes those choices one brick at a time: a grid, the finite-difference operators that act on it, the integrators that march it in time, and the boundary conditions that close it. Every choice is visible in the code and can be changed one at a time.

3.1 Grids

3.1.1 UniformGrid

Most numerical examples use a uniform Cartesian mesh, the simplest setting in which to illustrate discretization ideas. Two storage layouts are common: cell-centered (unknowns at cell centers) and node-centered (unknowns at grid intersections). `pyCoastal` adopts the cell-centered layout, which keeps stencils symmetric and boundary handling simple on rectangular domains.

```
from pyCoastal.numerics.grid import UniformGrid

grid = UniformGrid(shape=(200, 100), spacing=(2.0, 2.0), origin=(0.0, 0.0))
grid.Xc      # cell-center coordinates, a list [X, Y] built with indexing="ij"
grid.Xf      # face coordinates
grid.spacing  # (dx, dy)
grid.n_cells, grid.cell_volume
grid.boundary_indices["west"] # flat indices of the west boundary cells
```

`UniformGrid(shape, spacing, origin=None)` works in one and two dimensions (three-dimensional coordinates are built, but boundary indexing is 1D/2D only). Cell centers sit at $x_i = x_0 + (i + \frac{1}{2})\Delta x$. In 2D the flat index of cell (i, j) is $in_y + j$, so that `x` is axis 0 and `y` is axis 1 in every array. The grid precomputes, for each side ("west", "east", "south", "north"), the flat indices of the boundary cells in `boundary_indices` and a copy in `sponge_indices`, which the boundary conditions of Section 3.4 use. `neumann_indices(side)` returns the pair of boundary and first-interior indices needed to impose a gradient.

3.1.2 Mesh1D, Mesh2D, and Domain

`pyCoastal.numerics.domain` provides simple structured meshes built from a case file. `Mesh1D(x0, x1, nx)` stores cell centers `x` and spacing `dx`; `Mesh2D(x0, x1, nx, y0, y1, ny)` stores 2D center arrays `x`, `y` and spacings `dx`, `dy`. `Domain(cfg)` reads a `domain:` block with `dimension:` 1 or 2 and instantiates the right mesh; `Domain.info()` prints a summary.

Indexing and memory layout. Arrays follow NumPy C order, so the last index is contiguous. Spacings Δx , Δy are constant.

3.2 Finite differences

Finite differences approximate derivatives with algebraic expressions built from nearby grid values. On a uniform grid, a first derivative at a cell center can be estimated by a centered difference, which samples one point on either side to keep symmetry and second-order accuracy. Higher derivatives and one-sided formulas follow the same idea. The approach turns differential

equations into systems of algebraic equations, which is why it remains a common choice for structured-grid problems in fluid mechanics and wave modeling.

3.2.1 Centered operators

Centered operators combine values symmetrically around the point of interest, which keeps the approximation free of directional bias and gives higher accuracy for smooth solutions. For first derivatives and the Laplacian (Equations 1, 2):

$$\left. \frac{\partial f}{\partial x} \right|_{i,j} \approx \frac{f_{i+1,j} - f_{i-1,j}}{2\Delta x}, \quad \left. \frac{\partial f}{\partial y} \right|_{i,j} \approx \frac{f_{i,j+1} - f_{i,j-1}}{2\Delta y}, \quad (1)$$

$$\nabla^2 f|_{i,j} \approx \frac{f_{i+1,j} - 2f_{i,j} + f_{i-1,j}}{\Delta x^2} + \frac{f_{i,j+1} - 2f_{i,j} + f_{i,j-1}}{\Delta y^2}. \quad (2)$$

3.2.2 Upwind differences for advection

For an advection term with velocity u the solution at a point is determined by what arrives from upstream, so the stencil is taken from the upstream side (Equation 3):

$$\left. \frac{\partial f}{\partial x} \right|_i \approx \begin{cases} \frac{f_i - f_{i-1}}{\Delta x}, & u > 0, \\ \frac{f_{i+1} - f_i}{\Delta x}, & u < 0. \end{cases} \quad (3)$$

The one-sided stencil introduces numerical diffusion but is stable for hyperbolic problems and prevents non-physical oscillations.

3.2.3 The operator library

`pyCoastal.numerics.operators` implements these stencils on a `UniformGrid` (Table 3). The operators use periodic (roll) indexing, so boundary values must be imposed afterwards by the boundary conditions.

Table 3: Operators in `pyCoastal.numerics.operators`.

Function	Operation
<code>laplacian(field, grid)</code>	5-point Laplacian, Equation 2
<code>gradient(field, grid)</code>	centered gradient, returns (fx, fy)
<code>grad_x, grad_y</code>	centered first derivatives
<code>upwind_x(field, u, grid), upwind_y(field, v, grid)</code>	first-order upwind derivatives
<code>divergence(ux, uy, grid)</code>	centered $\nabla \cdot \mathbf{u}$
<code>curl_z(u, v, grid)</code>	$\partial v / \partial x - \partial u / \partial y$
<code>biharmonic(field, grid)</code>	$\nabla^4 f$ as the Laplacian of the Laplacian
<code>mixed_xy(field, grid)</code>	$\partial^2 f / \partial x \partial y$
<code>advect(u, v, field, grid, scheme="upwind")</code>	$u \partial_x f + v \partial_y f$
<code>smooth3(field, grid)</code>	5-point smoothing filter, weights summing to one

`pyCoastal.numerics.scheme` adds axis-generic versions, `central_difference(phi, spacing, axis)` and `upwind(phi, vel, spacing, axis)`.

3.3 Time integration

Once space is discretized, the semi-discrete variable $u(t)$ evolves under a right-hand side that gathers fluxes and sources, $du/dt = \text{RHS}(u, t)$. Explicit schemes compute the new state from information at the current level only, which makes them simple to apply and to check. The choice of integrator controls stability, accuracy, and how fast information moves through the grid.

Forward Euler (first order) treats the rate as constant over the step (Equation 4):

$$u^{n+1} = u^n + \Delta t \text{RHS}(u^n). \quad (4)$$

Heun / RK2 (second order) averages the slopes at the start and the end of the step (Equation 5):

$$k_1 = \text{RHS}(u^n, t), \quad k_2 = \text{RHS}(u^n + \Delta t k_1, t + \Delta t), \quad u^{n+1} = u^n + \frac{\Delta t}{2}(k_1 + k_2). \quad (5)$$

Classical RK4 (fourth order) is Equation 6:

$$u^{n+1} = u^n + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4). \quad (6)$$

SSP RK3 (third order, strong stability preserving) blends each stage with the earlier state, which keeps monotonicity for advection-dominated problems under a CFL limit (Equation 7):

$$\begin{aligned} u^{(1)} &= u^n + \Delta t \text{RHS}(u^n), \\ u^{(2)} &= \frac{3}{4}u^n + \frac{1}{4}[u^{(1)} + \Delta t \text{RHS}(u^{(1)})], \\ u^{n+1} &= \frac{1}{3}u^n + \frac{2}{3}[u^{(2)} + \Delta t \text{RHS}(u^{(2)})]. \end{aligned} \quad (7)$$

SSP RK3 is the recommended all-rounder for wave and transport demonstrations.

Adams-Bashforth 2 (second order, multistep) reuses the previous right-hand side, and needs one starting step from another method (Equation 8):

$$u^{n+1} = u^n + \frac{\Delta t}{2}(3 \text{RHS}(u^n) - \text{RHS}(u^{n-1})). \quad (8)$$

These live in `pyCoastal.numerics.time_intg` as array functions, `euler_step(u, t, dt, rhs)`, `rk2_step`, `rk4_step`, `rk3_ssp_step`, and `ab2_step(u, u_prev, t, dt, rhs)`, where `rhs(u, t)` returns an array of the same shape as `u`.

For multi-field states, `pyCoastal.numerics.scheme` provides integrator classes that act on a dictionary of arrays: `EulerIntegrator(dt)` and `SSPRK2Integrator(dt)` (Shu-Osher second-order TVD), both with `step(state, rhs, t, **kwargs)` returning `(new_state, new_time)`, where `rhs(state, t, **kwargs)` returns a dictionary of tendencies.

3.3.1 The Solver driver

`pyCoastal.numerics.solver.Solver(grid, physics, bc, integrator=None)` ties the pieces together. `physics` provides `initialize_state(grid)`, `rhs(state, t, ...)` and a `dt`; `bc` provides `apply(state, t)`; the integrator defaults to forward Euler. `Solver.run(t0, t_end, callback=None)` applies the boundary conditions, takes one step, and calls `callback(state, t)` after every step for output or plotting.

3.3.2 Stability

The CFL condition states that information carried by advection cannot move farther than one cell per step. For an advective speed c (Equation 9):

$$\text{CFL} = \max \left(\frac{c \Delta t}{\Delta x}, \frac{c \Delta t}{\Delta y} \right) < 1. \quad (9)$$

Diffusion imposes its own, often more severe, limit (Equation 10):

$$\Delta t \leq \frac{1}{2\nu} \left(\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2} \right)^{-1}. \quad (10)$$

The time step must satisfy both. The design applications apply the same rule internally: the port solver steps at a Courant number of 0.35 by default, and the one-line shoreline model at 0.9 of its diffusive limit (`tools.shoreline.suggest_dt`).

3.4 Boundary conditions

Boundary conditions close the discrete system. They fix a value, prescribe a gradient, let waves or flow enter or leave, or enforce a wall, and they control how signals reflect and how energy enters or exits the domain.

3.4.1 Ghost-cell formulas

For a cell-centered scheme the classical formulas at $x = 0$ are:

- **Dirichlet** (value), $q|_{\Gamma} = q_D(x, t)$: $q_0 = 2q_b - q_1$ with $q_b = q_D$.
- **Neumann** (gradient), $\partial q / \partial n|_{\Gamma} = g_N$: $q_0 = q_1 - \Delta x g_N$.
- **Solid wall**: $u_n = 0$, mirroring the normal component antisymmetrically; free slip mirrors the tangential velocity symmetrically, no slip antisymmetrically.
- **Sponge layer** (wave absorption): a damping zone relaxes the solution toward a reference state (Equation 11),

$$\frac{\partial q}{\partial t} = -\sigma(x) [q - q_{\text{ref}}], \quad \sigma(s) = \sigma_{\text{max}} \sin^2 \left(\frac{\pi s}{2} \right), \quad s = \frac{x - x_0}{L_{\text{sp}}} \in [0, 1]. \quad (11)$$

A sponge of $L_{\text{sp}} \approx 1$ to 3 wavelengths with $\sigma_{\text{max}} \sim \kappa c / L_{\text{sp}}$, $\kappa \approx 3$ to 6, absorbs well; monitor reflections with gauges inside the domain.

3.4.2 The boundary classes

`pyCoastal.numerics.boundary` implements boundary conditions that act directly on the boundary cells, using the flat indices precomputed by the grid. Every class takes a side name and a list of field keys and implements `apply(fields, grid, t)`:

- `DirichletBC(location, var_names, value)`, where `value` is a number or a function of time;
- `NeumannBC(location, var_names, gradient=0.0)`, which copies the interior value plus `gradient * dx` onto the boundary;
- `WallBC(location, var_names)`, which sets the named fields (normally the normal velocity or flux) to zero;
- `SpongeBC(location, var_names, damping)`, which multiplies the boundary strip by a constant factor or by `damping(t, k)` evaluated per flat index `k`, so a spatial ramp can be built from the index;
- `BoundaryManager(bcs=())` holds a list of conditions; `add(bc)` registers one and `apply_all(fields, grid, t)` applies them all, in order, at time `t`.

```
import numpy as np
from pyCoastal.numerics.grid import UniformGrid
from pyCoastal.numerics.boundary import (
    DirichletBC, NeumannBC, WallBC, SpongeBC, BoundaryManager,
)

grid = UniformGrid(shape=(200, 200), spacing=(2.0, 2.0))
fields = {"eta": np.zeros(grid.shape), "u": np.zeros(grid.shape),
          "v": np.zeros(grid.shape)}

bcs = BoundaryManager()
bcs.add(DirichletBC("south", ["eta"], lambda t: 0.5 * np.sin(2 * np.pi * t / 3.0)))
bcs.add(DirichletBC("west", ["u"], value=0.2))           # inflow
bcs.add(NeumannBC("east", ["u", "eta", "v"]))           # zero-gradient outflow
bcs.add(WallBC("north", ["v"]))                         # no normal flow
bcs.add(SpongeBC("north", ["eta"], damping=0.9))        # damp the strip
bcs.apply_all(fields, grid, t=0.0)
```

The complete version of this demonstration is `examples/numerics/2D_irr_turb.py`.

4 Governing equations and closures

`pyCoastal.physics` holds the governing equations as building blocks: flux functions and source terms that a solver assembles into a right-hand side, a pressure-projection step, Poisson solvers, and eddy-viscosity closures.

4.1 Shallow water equations

The shallow water equations describe a fluid layer whose horizontal scales are much larger than its depth. They track the total depth and the depth-averaged velocity, with the pressure hydrostatic. They capture wave propagation, flooding and draining, and large-scale currents (Equations 12, 13, 14):

$$\frac{\partial h}{\partial t} + \frac{\partial(hu)}{\partial x} + \frac{\partial(hv)}{\partial y} = 0, \quad (12)$$

$$\frac{\partial(hu)}{\partial t} + \frac{\partial}{\partial x} \left(hu^2 + \frac{1}{2}gh^2 \right) + \frac{\partial(huv)}{\partial y} = S_x, \quad (13)$$

$$\frac{\partial(hv)}{\partial t} + \frac{\partial(huv)}{\partial x} + \frac{\partial}{\partial y} (hv^2 + \frac{1}{2}gh^2) = S_y. \quad (14)$$

Here h is the total depth, u and v the depth-averaged velocities, and S_x , S_y collect bed slope, bottom friction, and Coriolis. When adding sources, preserve the lake-at-rest balance.

`ShallowWater2D(g=9.81)` provides `fluxes(h, hu, hv)`, which returns the three flux pairs (F_h, G_h) , (F_{hu}, G_{hu}) , (F_{hv}, G_{hv}) , and `source_bed_slope(h, zb, grid=None)`, which returns $S_x = -gh \partial z_b / \partial x$ and $S_y = -gh \partial z_b / \partial y$. Pass the grid so the derivatives use the real spacing; without it the spacing is taken as one.

4.2 Incompressible Navier-Stokes

For incompressible viscous flow a popular explicit strategy is Chorin's projection (fractional step) method: predict the velocity with advection and diffusion, solve a Poisson equation for the pressure that removes the divergence, then correct (Equations 15, 16, 17):

$$\tilde{\mathbf{u}} = \mathbf{u}^n + \Delta t (N(\mathbf{u}^n) + \nu \nabla^2 \mathbf{u}^n), \quad (15)$$

$$\nabla^2 p^{n+1} = \frac{1}{\Delta t} \nabla \cdot \tilde{\mathbf{u}}, \quad (16)$$

$$\mathbf{u}^{n+1} = \tilde{\mathbf{u}} - \Delta t \nabla p^{n+1}, \quad (17)$$

with $\mathbf{u} = (u, v)$, p the kinematic pressure, and ν the viscosity. `pyCoastal.physics.navier_stokes` provides `initialize_state(grid)`, which returns zero `u`, `v`, `p` fields, and `rhs(state, t, grid, bc_mgr, v=0.001)`, which returns the convection and diffusion tendencies for `u` and `v` (the predictor of Equation 15), with the velocity boundary conditions enforced on the result.

4.3 Poisson solver

A Poisson solver computes a field whose Laplacian matches a given source. It appears in the pressure projection, in potential flow, and in diffusion steady states (Equation 18):

$$\nabla^2 \phi = b \text{ in } \Omega, \quad \phi|_{\Gamma_D} = \phi_D, \quad \frac{\partial \phi}{\partial n} \Big|_{\Gamma_N} = g_N, \quad (18)$$

with the compatibility condition $\int_{\Omega} b \, d\Omega + \int_{\Gamma_N} g_N \, d\Gamma = 0$ for a pure Neumann problem. The five-point Laplacian of Equation 2 gives a sparse linear system. The Jacobi iteration, with $h = \Delta x = \Delta y$, is Equation 19,

$$\phi_{i,j}^{(k+1)} = \frac{1}{4} \left(\phi_{i+1,j}^{(k)} + \phi_{i-1,j}^{(k)} + \phi_{i,j+1}^{(k)} + \phi_{i,j-1}^{(k)} - h^2 b_{i,j} \right), \quad (19)$$

stopping when $\|b - A\phi^{(k)}\|_2 / \|b\|_2 < \varepsilon$.

`pyCoastal.physics.poisson` offers:

- `build_laplacian(nx, ny, dx, dy)`, the sparse CSR operator with zero Dirichlet boundaries (needs SciPy);
- `solve_direct(rhs, dx, dy, bc_mask=None, bc_values=None)`, a sparse direct solve;

- `solve_jacobi(rhs, dx, dy, bc_mask=None, bc_values=None, tol=1e-6, maxiter=5000)`;
- `solve_poisson(rhs, dx, dy, ..., method="auto")`, which picks the direct solver when SciPy is available and Jacobi otherwise;
- `apply_dirichlet_bc(phi, bc_mask, bc_values)`, which imposes Dirichlet values in place.

4.4 Turbulence closures

When the grid cannot resolve all scales, eddy-viscosity closures emulate the transport by unresolved motions. The closures here are educational scaffolds: production RANS or LES needs careful calibration and near-wall treatment.

Smagorinsky (LES). The eddy viscosity is proportional to the local strain rate and the filter width (Equation 20):

$$\nu_t = (C_s \Delta)^2 |S|, \quad |S| = \sqrt{2S_{ij}S_{ij}}, \quad S_{ij} = \frac{1}{2} \left(\frac{\partial u_i}{\partial x_j} + \frac{\partial u_j}{\partial x_i} \right), \quad (20)$$

with $C_s \approx 0.1$ to 0.2 . `SmagorinskyModel(Cs=0.17, filter_width=1.0)` computes `eddy_viscosity(u, v, grid)`.

k - ε (RANS). Two transport equations for the turbulent kinetic energy k and its dissipation ε , with $\nu_t = C_\mu k^2 / \varepsilon$ (Equations 21, 22):

$$\partial_t k + U_j \partial_{x_j} k = P - \varepsilon + \partial_{x_j} \left[\left(\nu + \frac{\nu_t}{\sigma_k} \right) \partial_{x_j} k \right], \quad (21)$$

$$\partial_t \varepsilon + U_j \partial_{x_j} \varepsilon = C_{\varepsilon 1} \frac{\varepsilon}{k} P - C_{\varepsilon 2} \frac{\varepsilon^2}{k} + \partial_{x_j} \left[\left(\nu + \frac{\nu_t}{\sigma_\varepsilon} \right) \partial_{x_j} \varepsilon \right], \quad (22)$$

with $P = 2\nu_t S_{ij} S_{ij}$. `KEpsilonModel(Cmu=0.09, sigma_k=1.0, sigma_e=1.3, C1=1.44, C2=1.92)` returns the eddy viscosity from `k` and `eps`; the transport equations belong to the solver.

k - ω (RANS). Pairs k with the specific dissipation $\omega = \varepsilon / k$, which behaves better near walls, with $\nu_t = k / \omega$ (Equations 23, 24):

$$\partial_t k + U_j \partial_{x_j} k = P - \beta^* k \omega + \partial_{x_j} \left[(\nu + \sigma_k^* \nu_t) \partial_{x_j} k \right], \quad (23)$$

$$\partial_t \omega + U_j \partial_{x_j} \omega = \alpha \frac{\omega}{k} P - \beta \omega^2 + \partial_{x_j} \left[(\nu + \sigma_\omega \nu_t) \partial_{x_j} \omega \right]. \quad (24)$$

`KOmegaModel(sigma_k=2.0, sigma_omega=2.0, beta_star=0.09, beta=0.075, gamma=0.52)` returns `k` / `omega` from `eddy_viscosity(k, omega)`.

5 Simulation examples

Each numerical example connects one governing equation, one set of numerics, and one diagnostic to a single learning objective. The scripts read their parameters from YAML files in `examples/configs/`, so an experiment can be repeated or swept by editing a few lines of text. They animate their fields with `matplotlib`; run them from the repository root.

```

python examples/numerics/water_drop.py
python examples/waves2D.py
python examples/wave2D_irregular.py
python examples/current.py
python examples/pollutant.py
python examples/numerics/viscous_fluid.py
python examples/numerics/2D_irr_turb.py
python examples/equilibrium_shoreline.py

```

5.1 Case files

The YAML keys used by the examples are collected in Table 4. Grid keys give cell counts and either spacings or domain lengths; physics keys the parameters of the equation; solver keys the time step (directly or through a CFL number) and the run length; output keys the gauge locations as grid indices.

Table 4: YAML keys used by the numerical examples.

Config	grid	physics	solver / forcing	output
water_drop. yaml	Nx, Ny, Lx, Ly	wave_speed, sigma	CFL, t_final	
waves2D. yaml	nx, ny, dx, dy	gravity, depth	amplitude, period; cfl, t_ final	obs_points
waves2D_ irregular. yaml	nx, ny, dx, dy	gravity, depth	type (pm or jonswap), gamma, Hs, Tp; dt, duration	gauge
current. yaml	nx, ny, dx, dy	speed	cfl, t_final	gauge
pollutant. yaml	Lx, Ly, Nx, Ny	diffusivity, U0, R	t_final, c0_sigma	
viscous_ fluid.yaml	Nx, Ny, Lx, Ly	viscosity, background_ speed, vortex_sigma	dt, t_final	
equilibrium_ shoreline. yaml	Lshore, Gb, Lb1, Lb2, Yi		n_points, sag_side_factor	

5.2 Water drop: the 2D wave equation

Objective. Verify the accuracy of the Laplacian and the stability of the time stepping on a simple hyperbolic system (Equation 25),

$$\frac{\partial^2 \eta}{\partial t^2} = c^2 \nabla^2 \eta. \quad (25)$$

A Gaussian bump of width `sigma` is released at the center of the domain with zero Dirichlet boundaries, and the circular front should travel at c . Refining the grid and the time step should converge the solution at the expected order.

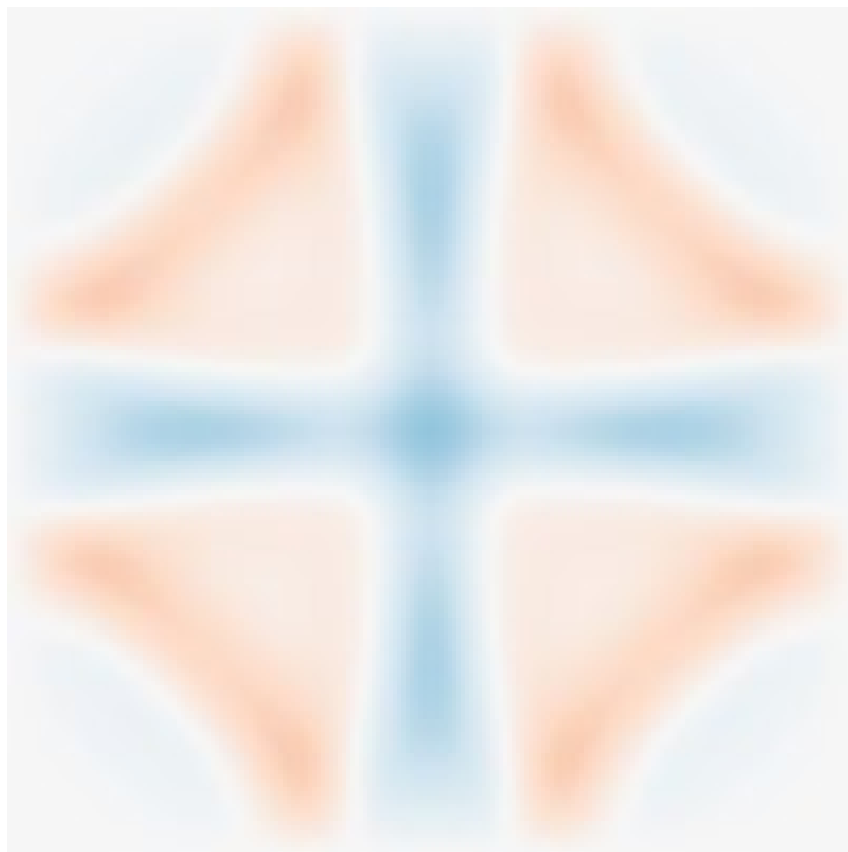


Figure 1: Circular wave propagation from a central disturbance (`examples/numerics/water_drop.py`).

5.3 Regular waves: waves2D.py

A depth-averaged wave field in uniform depth, forced by a sinusoid of amplitude `amplitude` and period `period` at the $y = 0$ boundary and periodic in x . The script solves linear advection along y column by column with an upwind stencil and forward Euler, animates η , and records the time series at the observation points. The wave speed comes from the linear dispersion relation for the configured depth and period.

5.4 Irregular waves: a numerical flume

Objective. Generate a target spectrum and manage reflections. The script synthesizes a surface elevation record from a Pierson-Moskowitz or JONSWAP spectrum (Section 6.1), imposes it as a Dirichlet condition at the inflow boundary, and records a gauge inside the domain, so the input and interior spectra (H_s , T_p) can be compared. A sponge at the outflow absorbs the outgoing waves.

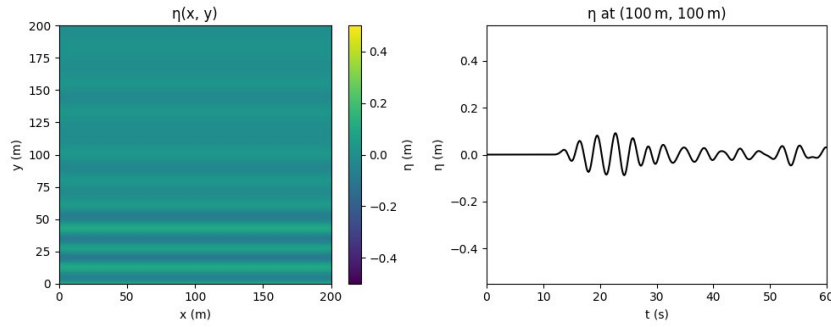


Figure 2: Irregular wave field at the end of the run (`examples/wave2D_irregular.py`).

5.5 Current and pollutant: advection and diffusion

Objective. Observe upwind smearing and diffusive spreading. `current.py` advects a passive field with a uniform current, $u_t + c u_x = 0$, with a Dirichlet inlet on the west and a Neumann outlet on the east, and records a gauge. `pollutant.py` solves the full advection-diffusion equation for a Gaussian patch in a vortex flow in a shallow pond (Equation 26),

$$\frac{\partial C}{\partial t} + u \frac{\partial C}{\partial x} + v \frac{\partial C}{\partial y} = D \nabla^2 C, \quad (26)$$

with upwind advection, centered diffusion (`operators.gradient` and `operators.laplacian`), and zero-flux boundaries. For pure diffusion the variance of the patch grows as $2Dt$, which the run can be checked against.

5.6 Viscous fluid: velocity decay

Objective. Check the diffusive time scale and operator accuracy. `viscous_fluid.py` evolves two counter-rotating vortices on a periodic domain with central differences and forward Euler. The diffusion part can be checked against the decaying mode (Equation 27)

$$u(x, y, t) = U_0 \sin(k_x x) \sin(k_y y) \exp \left[-\nu(k_x^2 + k_y^2)t \right]. \quad (27)$$

5.7 Boundary-condition test: 2D_irr_turb.py

A pure boundary-condition exercise on a 400 m by 400 m grid: a sinusoidal Dirichlet wave maker on the south side, a spatially ramped sponge on the north, a Dirichlet inflow velocity and a zero-gradient surface on the west, zero-gradient outflow on the east, and walls for the normal velocity. No physics is advanced; the script applies the `BoundaryManager` every step and records a gauge, which isolates what the boundary classes do.

5.8 Equilibrium shoreline behind detached breakwaters

`equilibrium_shoreline.py` builds a schematic equilibrium planform behind a pair of detached breakwaters from three circular arcs (a left lateral arc, a central salient, and a right lateral arc), following the Hsu and Evans style of parabolic bay construction. The geometry comes from `equilibrium_shoreline.yaml` (shoreline length, gap between breakwaters, their lengths, and offshore distance) and the construction uses `pyCoastal.tools.morphodynamics` (Section 6.5), which also reports the retreat and the accretion percentage.

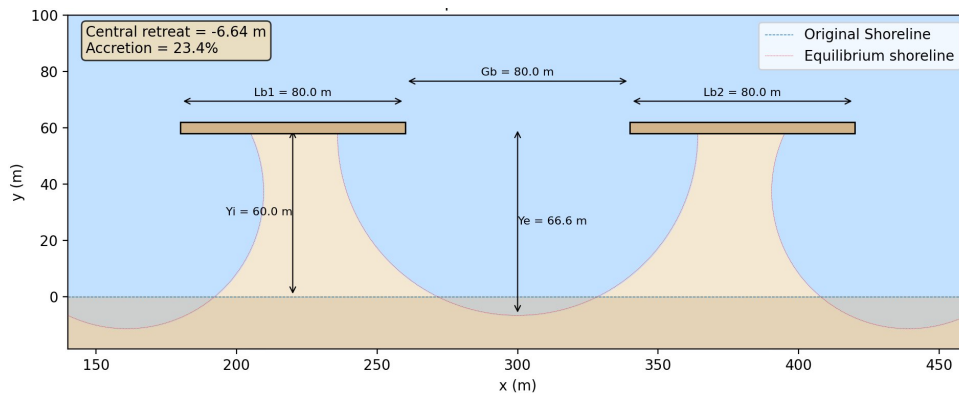


Figure 3: Equilibrium shoreline behind two detached breakwaters (`examples/equilibrium_shoreline.py`).

5.9 Diagnostics, verification, and validation

Numerical experiments should be accompanied by simple, repeatable checks:

- **CFL monitor.** Compute and report the Courant number each step, and adapt Δt if needed.
- **Divergence.** After a projection step, $\|\nabla \cdot \mathbf{u}\|_2$ should sit near the Poisson solver tolerance.
- **Energy and variance.** Monitor $\int \eta^2$ or the kinetic energy to spot drift or spurious reflections.
- **Convergence.** Refine Δx , Δy , and Δt and verify the expected order.
- **Spectral targets.** Compare H_s and T_p at interior gauges with the input, and estimate reflection by separating incident and reflected waves.

Save arrays as NumPy files and figures with matplotlib; export animations as `.gif` or `.mp4`. Store the run metadata (git hash, YAML, random seeds) beside the outputs so a figure can be regenerated exactly.

5.10 Extending the framework

To add physics or numerics:

1. implement a right-hand side `rhs(u, t)` and pass it to an integrator;
2. provide boundary handlers consistent with the stencils;
3. add YAML keys with sensible defaults;
4. add a small example and a docstring that states the equations, the variables, the numerics, and the stability limits.

5.11 Limitations of the numerical framework

- Uniform Cartesian grids only; no curvilinear or unstructured meshes.
- Explicit time stepping, which is restrictive for stiff sources such as the ε or ω transport.
- The turbulence closures are educational scaffolds, not production RANS or LES.
- No wetting and drying or moving shorelines in the 2D solvers. (The design applications handle shoreline change with the one-line model and flooding with connectivity-aware mapping; see Section 17 and Section 18.)

Part III

Engineering tools

6 Standalone formulae: `pyCoastal.tools`

`pyCoastal.tools` collects standalone coastal engineering formulae. Every function takes its parameters explicitly, with no hidden hard-coded values, so defaults can be set or overridden from a case file. These are the first checks an engineer runs; the design applications of Part IV build full design chains on top of the same relations, with validity checks and reporting. Where a tool and an application overlap, prefer the application for design.

6.1 Waves: `tools.wave`

6.1.1 Linear dispersion and derived numbers

Under linear (small-amplitude) theory, frequency, wavenumber, and depth are linked by the dispersion relation, which sets the phase and group speeds and underpins most coastal scalings:

$$\omega^2 = gk \tanh(kh), \quad c = \frac{\omega}{k}, \quad c_g = \frac{c}{2} \left(1 + \frac{2kh}{\sinh 2kh} \right). \quad (28)$$

`dispersion(T, h, g=9.81, tol=1e-12)` solves it for the wavelength L by Newton iteration and raises on a non-positive period or depth; `wave_number(T, h)` returns $k = 2\pi/L$. The deep-water wavelength is $L_0 = gT^2/(2\pi)$.

Two nondimensional numbers classify the waves:

- **Iribarren (surf similarity)**, $\xi_0 = \tan \beta / \sqrt{H_0/L_0}$: `surf_similarity(alpha, H, T)` returns it and `breaker_type(alpha, H, T)` names the breaker, spilling for $\xi < 0.4$, plunging for $0.4 \leq \xi \leq 2$, and collapsing or surging above 2.
- **Ursell**, $U = HL^2/h^3$: `ursell_number(H, T, h)` returns the value and an interpretation, linear theory for $U < 32$ and a nonlinear theory (Stokes, Boussinesq) above.

`wave_setup(Hb, gamma=0.8)` gives the shoreline setup $\bar{\eta} = \frac{5}{16}\gamma H_b$.

6.1.2 Spectral synthesis

A wave spectrum describes how the energy of a sea state is distributed over frequency. To drive a numerical flume with irregular seas, a surface elevation record is synthesized by random-phase superposition consistent with a target spectrum (Equation 29):

$$\eta(t) = \sum_{i=1}^N \sqrt{2S(f_i)\Delta f} \cos(2\pi f_i t + \varphi_i), \quad \varphi_i \sim U[0, 2\pi]. \quad (29)$$

The spectral moments $m_n = \int_0^\infty f^n S(f) df$ give $H_s \approx 4\sqrt{m_0}$ and $T_z \approx \sqrt{m_0/m_2}$. `generate_irregular_wave(Hs, Tp, duration, dt, spectrum="pm", gamma=3.3)` returns `(t, eta)` for a Pierson-Moskowitz (fully developed) or JONSWAP (fetch-limited, peak enhancement `gamma`) spectrum scaled to the requested H_s .

6.2 Sediment transport: `tools.sediment_transport`

The Shields parameter gauges bed mobility under a bed shear stress $\tau_b = \rho u_*^2$:

$$\theta = \frac{\tau_b}{(\rho_s - \rho) g d}, \quad (30)$$

with a critical value $\theta_c \approx 0.03$ to 0.06 for sand (the design modules compute it from the grain size, Section 8). The transport formulae in Table 5 convert mobility into rates.

Table 5: Transport relations in `pyCoastal.tools.sediment_transport`.

Function	Relation
<code>shields_parameter(tau_b, rho_s, rho, d)</code>	Equation 30
<code>van_rijn_bedload(Ue, h, d50, rho_s, rho)</code>	$q_b = 0.015 \rho_s U_e h (d_{50}/h)^{1.2} M_e^{1.5}$, $M_e = (U_e - U_{cr}) / \sqrt{(s-1) g d_{50}}$ (Van Rijn 1984)
<code>van_rijn_suspended(Ue, h, d50, rho_s, rho)</code>	$q_s = 0.008 \rho_s U_e d_{50} M_e^{2.4} D_*^{-0.6}$
<code>bijker_bedload(tau_wave, tau_current, rho_s, rho, d50)</code>	$\propto \sqrt{\tau_{\text{total}}}$, waves and current combined (Bijker 1971)
<code>cerc_transport(Eb, angle, K=0.39)</code>	$Q = K E_b / (\rho g) \sin \varphi_b \cos \varphi_b$ (CERC)
<code>bagnold_sediment(H, c, rho_s)</code>	$(\rho_s / \rho) H^2 c$, energetics estimate
<code>izbash_current(rho_s, rho, d)</code>	$u_c = 1.7 \sqrt{\Delta g d}$, stone stability in current
<code>einstein_bedload(tau_star, d, s)</code>	$q_b = 8 \sqrt{g(s-1) d^3} \tau_*^{1.5}$

The critical velocity in the Van Rijn functions is $U_{cr} = 0.19 d_{50}^{0.1} \log(12h/3d_{50})$ and $s = \rho_s / \rho$.

6.3 Structures and runup: `tools.structural`

- `hudson_dn50(Hs, Delta, theta, Kd=3.0)`: $H_s / (\Delta D_{n50}) = (K_D \cot \theta)^{1/3} / 1.27$ (SPM 1984).
- `vandermeer_dn50(Hs, Delta, P, N, alpha, xi_m, damage=2.0, safety=1.0)`: Van der Meer (1988), plunging and surging branches (Equation 38 and Equation 39 in Section 9), with the wave count saturating at 7500. The surf similarity must be supplied, because it depends on the period.
- `hunt_runup(beta, H, L)`: $R \approx H \tan \beta / \sqrt{H/L}$ (Hunt 1959).
- `stockdon_runup(H, L, beta)`: the Stockdon et al. (2006) 2% runup.
- `goda_wave_force(H, T, h, beta)`: a screening estimate of the Goda horizontal force on a vertical face, in N/m, taking $H_{\text{design}} = 1.8H$. The full Goda distribution with uplift and stability checks is `applications.seawall.goda_pressures` (Section 10).
- `iribarren_stability(H, alpha, rho_s, rho_w, mu, N)`: the Iribarren stone weight, $W \geq N \rho_s g H^3 / [\Delta^3 (\mu \cos \alpha - \sin \alpha)^3]$.

6.4 One-line shoreline model: `tools.shoreline`

The one-line model treats the shoreline position $y(x, t)$ as the only morphological variable and conserves sediment alongshore:

$$\frac{\partial y}{\partial t} = -\frac{1}{(1-p)D} \frac{\partial Q_{ls}}{\partial x}, \quad Q_{ls} = K_{\text{cerc}} H_b^{5/2} \sin(2\alpha_b), \quad \alpha_b = \alpha_0 - \frac{\partial y}{\partial x}, \quad (31)$$

with $H_b = K_d K_t H_{\text{free}}$, where K_d is a diffraction shadow factor and K_t a transmission factor. The explicit scheme is stable when Equation 32 holds,

$$\Delta t \leq 0.45 \frac{\Delta x^2}{\max G}, \quad G = \frac{2K_{\text{cerc}} H_b^{5/2}}{(1-p)D}. \quad (32)$$

`OneLineParams(D=8, p=0.4, Kcerc=0.4, Hfree=1.0, alpha0=10 deg, Kt=1.0, beta0=-30 deg, morfac=1.0)` holds the parameters; any of `Hfree`, `alpha0`, `Kt` can be arrays over x . The building blocks are `make_grid(Lx, dx)`, `breakwaters_geometry(x, tips, y_tip, crest_freeboard)` for any number of detached breakwaters, `kd_diffraction_field(x, geom, beta0)` for a simple shadow factor $K_d = \frac{1}{2}(1 + \cos \theta)$ from each tip, `compute_flux_Qls, rhs_y_t, suggest_dt`, and `apply_bcs(y, bc="fixed_ends" | "zero_slope")`. `run_one_line_model()` runs a default two-breakwater case. The nourishment application (Section 17) drives this solver with a fill planform.

6.5 Morphodynamics helpers: `tools.morphodynamics`

- `bruuns_rule(S, beta, L=None, h=None, B=None)`: shoreline retreat under sea-level rise S , $R = SL/(h + B)$ when the profile length, closure depth, and berm height are given, and $R = S/\tan \beta$ otherwise.
- `exner_change(qs_dx, porosity=0.64)`: bed change rate from the transport divergence, $\partial \eta / \partial t = -q_{s,x}/(1 - n)$.
- `linear_slope(grid, north_level, south_level)`: a plane beach on a grid.
- `circle_through_points, two_point_arc`, and `build_tombolo_arcs(Lshore, Gb, Lb1, Lb2, yb, ...)`: the three-arc equilibrium planform behind a pair of detached breakwaters used by `examples/equilibrium_shoreline.py`, with retreat $Y_e = 1.204Y_i - 0.07G_b$ below the breakwater line; `accretion_metrics(x_eq, y_eq, geom)` returns the retreat and the accretion percentage.

Part IV

Design applications

7 Design conditions from a record

Module: `pyCoastal.applications.extremes`. *Example:* `examples/design_wave.py`.

Browser: Return values.

Every structure in this package is sized against a design condition, and that condition has to come from somewhere. This module is where it comes from: a measured or hindcast record in, a return value with an honest confidence band out.

7.1 Two routes

Peaks over threshold (POT). Keep the independent storm peaks above a threshold, fit a generalized Pareto distribution (GPD) to the excesses, and combine it with the rate at which peaks arrive. It uses the record efficiently, but the answer depends on the threshold, so the threshold is chosen with the diagnostics below rather than by eye.

Block maxima. Keep the largest value in each year and fit a generalized extreme value (GEV) distribution. It wastes data but is hard to get wrong, and it is the right check on a POT answer.

Both are fitted by **L-moments** (Hosking and Wallis 1997), which are close to unbiased for the sample sizes a coastal record offers, are not troubled by the flat likelihood surfaces that defeat maximum likelihood on short records, and need no optimizer. Shape parameters use the extreme-value convention: a positive shape is a heavy tail with no upper bound, a negative shape a finite upper limit. Hosking writes $k = -\text{shape}$; the conversion is done internally and never exposed.

7.2 Theory

Declustering. `decluster(values, threshold, separation)` walks the record and keeps only the largest value in each run above the threshold. Two runs separated by fewer than `separation` samples below the threshold are one storm. For a 3-hourly record, 24 samples is a three-day separation, the usual choice for storm waves.

GPD fit. With survivor function $1 - F(x) = (1 + \xi x/\sigma)^{-1/\xi}$ (exponential as $\xi \rightarrow 0$), the L-moment estimators from the mean λ_1 and L-CV $\tau = \lambda_2/\lambda_1$ of the excesses are given by Equation 33,

$$\xi = 2 - \frac{1}{\tau}, \quad \sigma = \lambda_1(1 - \xi). \quad (33)$$

A shape at or above one means the fitted mean does not exist; the fit is returned and flagged rather than clipped.

POT return value. With λ peaks per year, the T -year value is Equation 34,

$$x_T = u + \frac{\sigma}{\xi} [(\lambda T)^\xi - 1], \quad (34)$$

with the logarithmic form in the exponential limit. A return period shorter than $1/\lambda$ is outside the model.

GEV fit. With $F(x) = \exp\{-[1 + \xi(x - \mu)/\sigma]^{-1/\xi}\}$, the parameters come from Hosking's L-moment estimators, whose polynomial approximation for k from the L-skewness is accurate to better than 10^{-4} for $-0.5 < k < 0.5$.

Plotting positions. The empirical return periods use the Gringorten position $(i-a)/(n+1-2a)$ with $a = 0.44$, close to unbiased for the Gumbel and GEV families.

Threshold diagnostics. Above a threshold where the GPD holds, the mean excess is linear in the threshold (`mean_residual_life`), and both the fitted shape and the *modified* scale $\sigma^* = \sigma - \xi u$ are flat (`threshold_stability`). The raw scale is not threshold-invariant, and reading it as if it were is a common way to pick a threshold badly.

Confidence. `ExtremeFit.confidence` is a percentile bootstrap over the fitted peaks. It captures sampling error in the parameters and nothing else: not measurement error, not the choice of threshold, and not whether the climate that produced the record is the one the structure will see. Those are usually the larger uncertainties.

7.3 Using the module

```
from pyCoastal.applications.extremes import (
    fit_pot, fit_block_maxima, mean_residual_life, threshold_stability,
)

mrl = mean_residual_life(record, thresholds)          # pick the threshold properly
stab = threshold_stability(record, thresholds, separation=24)
fit = fit_pot(record, threshold=2.5, separation=24, samples_per_year=2920)

fit.return_value(100)                               # 100-year Hm0
fit.confidence(100, level=0.90)                     # and how much it could have been
fit.extrapolation_note(100)                         # how far past the record that reaches
print(fit.summary())

check = fit_block_maxima(record, block=2920)        # annual maxima cross-check

fit_pot and fit_block_maxima return an ExtremeFit with kind ("pot" or "gev"),
parameters, the peaks or maxima in data, their positions, the rate and record length years,
the bootstrap samples, and a warnings list. The fit warns when there are fewer than one or
two peaks per year or fewer than about thirty in total.
```

7.4 Worked example

`examples/design_wave.py` builds a 40-year synthetic 3-hourly hindcast of H_{m0} with known parameters, chooses the threshold with the diagnostics, fits POT and annual maxima, and hands the 100-year wave to the seawall designer. Because the record is synthetic, the right answer is known, so the example also does what a real study cannot: it repeats the whole analysis on 60 fresh 40-year records and measures the true sampling spread of the 100-year estimate, then checks the bootstrap band against it.

Output of `examples/design_wave.py`

```
Record: 40 years, 116800 samples, max 5.21 m, 240 storms (6.0/yr)
Threshold 2.50 m chosen from the diagnostics (the record was built with 2.50 m)

Peaks over threshold
Model           peaks over threshold
Record          40.0 years, 240 peaks
Threshold       2.50
Rate            6.00 peaks per year
```

```
Scale, shape      0.560, -0.071

Return period  value    90% band
   10 yr      4.49    4.19 to 4.83
   50 yr      5.12    4.61 to 5.78
  100 yr      5.38    4.74 to 6.19
the record was built with scale 0.550, shape -0.080, rate 6.0/yr
```

```
Annual maxima, as a cross-check
Model          annual maxima
Record         40.0 years, 40 maxima
Location       3.489
Scale, shape   0.484, -0.058
```

```
Return period  value    90% band
   10 yr      4.51    4.22 to 4.74
   50 yr      5.18    4.63 to 5.58
  100 yr      5.45    4.77 to 5.95
```

```
100-year Hm0 = 5.38 m (4.74 to 6.19 m, 90% bootstrap)
true value for this climate: 5.25 m
2.5 times the 40 year record. Treat the confidence band as a lower bound on the real
↪ uncertainty.
```

```
Repeating the study on 60 fresh 40-year records
100-year estimate: mean 5.23 m, sd 0.44 m
5th to 95th percentile: 4.61 to 5.94 m (width 1.32 m)
bootstrap band from the single record: width 1.45 m
this record sits at the 70th percentile of that spread
The bootstrap band matches the real sampling spread.
```

```
Design wave at the toe 4.68 m (depth limited)
Seawall crest +13.13 m CD, base 20.83 m, concrete 45.0 m3/m
```

```
Wrote media/design_wave.png
```

The bootstrap band from the single record (width 1.45 m) matches the real sampling spread across records (1.32 m), and the true 100-year value of 5.25 m sits inside it. The 100-year wave is 2.5 times the record length, so the band should be treated as a lower bound on the real uncertainty, which is exactly what `extrapolation_note` says.

7.5 Limits

- Stationarity is assumed: no trend or climate change in the record.
- The band is sampling error only.
- Very short records, or thresholds that leave too few peaks, are flagged but still fitted.

8 Sediment and soil

Module: `pyCoastal.applications.sediment`.

Every structure in this package sits on, in, or behind something. This module makes that something a material with properties that propagate: the grain size that decides whether the bed moves at all, the friction angle that decides how hard the backfill pushes on a wall, and the unit weight that decides how much of that push is soil and how much is water. The seawall, breakwater, channel, pile, scour, and nourishment modules all take a `Sediment` or a catalogue key.

Design wave from a 40-year record: 100-year $H_{m0} = 5.38$ m (4.74 to 6.19 m), true value 5.25 m

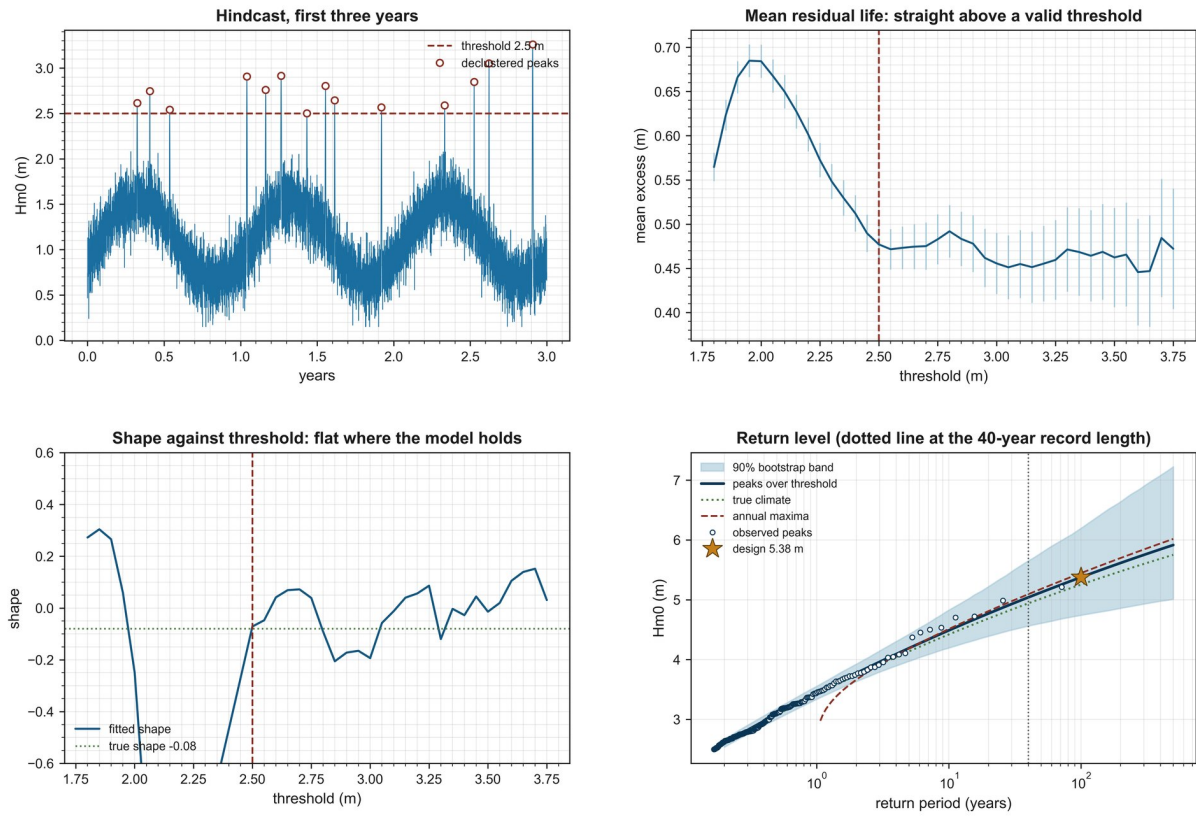


Figure 4: Extreme value analysis of a 40-year record: threshold diagnostics, return level plot with bootstrap band, and the sampling spread of the 100-year estimate.

Two families of relation live here. **Mobility** (grain size, fall velocity, the Shields threshold) gates scour: a bed that never reaches its threshold does not scour, whatever the wave height. **Earth pressure** (Rankine and Coulomb coefficients, and the pressure diagram with the water table and surcharge in the right places) loads the back of a wall.

8.1 The material catalogue

`Sediment(name, d50, specific_gravity=2.65, porosity=0.4, friction_angle=32, cohesion=0, phi_sorting=0.6, d90=None, description="")` stores a material, with derived `relative_density` ($s - 1$), `dry_unit_weight`, `saturated_unit_weight`, `submerged_unit_weight`, `grading` (d_{90}/d_{50} , with d_{90} defaulting to $2.5 d_{50}$), and `cohesive`. `sediment(key)` looks up the catalogue (Table 6) and names the options on a bad key.

Table 6: The built-in material catalogue.

Key	d_{50} (mm)	ϕ' (deg)	Description
<code>soft_clay</code>	cohesive	22	normally consolidated, undrained strength governs
<code>stiff_clay</code>	cohesive	26	overconsolidated
<code>silt</code>	0.03	28	mobile at almost any wave
<code>very_fine_sand</code>	0.09	29	suspends readily, high siltation
<code>fine_sand</code>	0.19	31	the usual beach and nearshore sand
<code>medium_sand</code>	0.38	33	typical dredged fill
<code>coarse_sand</code>	0.75	35	good drained backfill
<code>fine_gravel</code>	6	38	free draining
<code>coarse_gravel</code>	30	40	shingle beach
<code>rock_fill</code>	150	42	engineered granular backfill

8.2 Mobility

The dimensionless grain size, the critical Shields parameter (Soulsby and Whitehouse 1997), and the settling velocity (Soulsby 1997) are given by Equation 35:

$$\begin{aligned}
D_* &= d_{50} \left[\frac{g(s-1)}{\nu^2} \right]^{1/3}, \\
\theta_{cr} &= \frac{0.30}{1 + 1.2D_*} + 0.055 [1 - e^{-0.020D_*}], \\
w_s &= \frac{\nu}{d_{50}} \left[\sqrt{10.36^2 + 1.049D_*^3} - 10.36 \right].
\end{aligned} \tag{35}$$

The fall velocity relation is valid from silt to gravel, which is why it is preferred to Stokes at one end and a drag law at the other.

Under waves, `wave_orbital_velocity(Hs, T, depth)` gives the near-bed orbital amplitude from linear theory and `wave_shields(material, Hs, T, depth)` the Shields parameter, with the Swart friction factor for a rough turbulent bed and a Nikuradse roughness $k_s = 2.5 d_{50}$ (Equation 36):

$$f_w = \exp \left[5.213 \left(\frac{A}{k_s} \right)^{-0.194} - 5.977 \right], \quad f_w \leq 0.3. \tag{36}$$

`bed_mobility(material, Hs, T, depth)` is the gate the scour functions use. It returns whether the bed is live, the regime (live bed, clear water, or cohesive), and by how much the threshold is exceeded. Scour relations are almost all live-bed results, and applying one to a bed that never reaches its threshold predicts a hole that will not form.

8.3 Earth pressure

`earth_pressure_coefficient(friction_angle, kind="active", wall_friction=0, backslope=0)` returns the Rankine coefficient for "active", "at_rest" ($K_0 = 1 - \sin \phi'$), or "passive"; a non-zero wall friction switches the active case to Coulomb, which is the honest choice for a rough concrete face and gives a smaller force. Which coefficient applies is a question about movement, not about soil: a wall that cannot move the millimeter or two that mobilizes the active state carries at-rest pressure, roughly half as large again. Choosing "active" because it is smaller is the most common way a retaining structure is under-designed.

`lateral_earth_pressure(material, height, water_table=0, surcharge=0, kind="active")` builds the pressure diagram with effective stress and pore water apart; `lateral_earth_force(...)` integrates it into soil, water, and total forces with the lever arm of the total. Below the water table the soil pushes with its submerged unit weight and the water separately with its full hydrostatic gradient, $K = 1$. The sum is larger than dry soil alone would give, which is why drainage behind a seawall is a structural matter and not a detail. The soil force can be argued down with a better backfill; the water force can only be drained away.

```
from pyCoastal.applications.sediment import (
    sediment, critical_shields, fall_velocity, wave_shields, lateral_earth_force,
)

sand = sediment("medium_sand")
critical_shields(sand), fall_velocity(sand)          # about 0.035 and 0.05 m/s
wave_shields(sand, Hs=2.0, T=9.0, depth=8.0)["mobile"]
lateral_earth_force(sand, height=10.0, water_table=0.0, surcharge=10.0,
                    kind="at_rest")["total"]         # kN/m
```

9 Rubble-mound breakwaters

Module: `pyCoastal.applications.structures`. *Example:* `examples/breakwater_design.py`.
Browser: Breakwater.

Armor sizing and wave overtopping, with every relation traced to its source: Van der Meer (1988) and Hudson (SPM 1984) for stability, the Rock Manual (2007) for layer geometry, and EurOtop (2018) for overtopping and the tolerable discharge limits. The module is aimed at design work rather than illustration: every relation is named, its validity range stated, and the functions compute outside a range when asked but say so.

9.1 The design condition

`DesignConditions(Hm0, Tm10, depth=15.0, storm_duration=6*3600)` carries the wave and water level at the toe. `DesignConditions.from_peak_period(Hm0, Tp, ...)` converts with $T_{m-1,0} = T_p/1.1$ for a single-peaked spectrum. Derived properties are the wave count $N = t_{\text{storm}}/T_m$, the deep and local wavelengths, and the surf similarity (Equation 37)

$$\xi_{m-1,0} = \frac{\tan \alpha}{\sqrt{H_{m0}/L_{m-1,0}}}, \quad L_{m-1,0} = \frac{gT_{m-1,0}^2}{2\pi}. \quad (37)$$

9.2 Armor stability

Van der Meer (1988). Two regimes, selected by the surf similarity against a critical value (Equations 38, 39, 40):

$$\frac{H_s}{\Delta D_{n50}} = 6.2 P^{0.18} \left(\frac{S}{\sqrt{N}} \right)^{0.2} \xi^{-0.5} \quad (\xi < \xi_{cr}, \text{ plunging}), \quad (38)$$

$$\frac{H_s}{\Delta D_{n50}} = 1.0 P^{-0.13} \left(\frac{S}{\sqrt{N}} \right)^{0.2} \sqrt{\cot \alpha} \xi^P \quad (\xi \geq \xi_{cr}, \text{ surging}), \quad (39)$$

$$\xi_{cr} = \left(6.2 P^{0.31} \sqrt{\tan \alpha} \right)^{1/(P+0.5)}. \quad (40)$$

$\Delta = \rho_s/\rho_w - 1$ (1.585 for 2650 kg/m³ rock in seawater), P is the notional permeability (0.1 impermeable core with a filter, 0.4 permeable core, 0.5 homogeneous, 0.6 very permeable), and S the damage level (DAMAGE_LEVELS: 2 is start of damage and the usual design value, 8 and 12 to 17 failure). `rock_armour_vandermeer(conditions, cot_alpha, Delta=1.585, permeability=0.4, damage=2.0, safety_factor=1.0)` returns Dn50, M50, the governing regime, ξ , and ξ_{cr} . It is valid for non-depth-limited waves at the toe; the shallow-water modification is not applied.

Hudson (SPM 1984). $H_s/(\Delta D_{n50}) = (K_D \cot \alpha)^{1/3}/1.27$, where 1.27 converts the SPM's $H_{1/10}$ basis to H_s . `rock_armour_hudson(conditions, cot_alpha, Delta, Kd=4.0)` is a first estimate and cross-check: it has no dependence on period, storm duration, permeability, or damage.

Layer geometry (Rock Manual). `armour_layer(Dn50, n_layers=2, layer_coefficient=1.0, porosity=0.37)` returns the layer thickness $t = nk_t D_{n50}$ and the number of stones per square meter.

9.3 Overtopping

For a sloping structure, EurOtop (2018) takes the lesser of the breaking and the maximum expressions (Equations 41, 42):

$$\frac{q}{\sqrt{g H_{m0}^3}} = \frac{0.023}{\sqrt{\tan \alpha}} \gamma_b \xi \exp \left[- \left(2.7 \frac{R_c}{\xi H_{m0} \gamma_b \gamma_f \gamma_\beta \gamma_v} \right)^{1.3} \right], \quad (41)$$

$$\frac{q}{\sqrt{g H_{m0}^3}} = 0.09 \exp \left[- \left(1.5 \frac{R_c}{H_{m0} \gamma_f \gamma_\beta} \right)^{1.3} \right]. \quad (42)$$

For a plain vertical wall in non-impulsive conditions (EurOtop eq. 7.1) (Equation 43):

$$\frac{q}{\sqrt{g H_{m0}^3}} = 0.047 \exp \left[- \left(2.35 \frac{R_c}{H_{m0} \gamma_\beta} \right)^{1.3} \right], \quad (43)$$

valid when the impulsiveness parameter $h_* = 1.35 (h/H_{m0}) (2\pi h/gT_{m-1,0}^2)$ exceeds about 0.23; `overtopping_vertical` reports `h_star` and `impulsive`.

The factors are roughness γ_f (ROUGHNESS_FACTORS: 1.0 for smooth concrete, 0.55 and 0.40 for impermeable and permeable two-layer rock, 0.38 to 0.49 for concrete units), obliquity γ_β (obliquity_factor(beta_degrees)), berm γ_b , and wave wall γ_v . EurOtop reports roughly a factor-of-three scatter about the mean discharge; `overtopping_with_uncertainty(result, factor=3.0)` returns the band, and `required_crest_freeboard(conditions, q_allowable, cot_alpha, ...)` inverts the relation by bisection. `assess_overtopping(q)` lists which uses a discharge is tolerable for (Table 7).

Table 7: Tolerable mean discharges q in l/s/m (TOLERABLE_DISCHARGE, EurOtop 2018).

Key	q	Use
<code>pedestrians_unaware</code>	0.03	unaware pedestrians, narrow walkway, sea in view
<code>pedestrians_aware</code>	0.1	aware pedestrians, able to see and avoid the hazard
<code>harbour_quay_equipment</code>	0.4	equipment set back 5 to 10 m from the crest
<code>trained_staff</code>	1	trained staff, well shod and protected, wide walkway
<code>buildings_structural</code>	1	structural damage to buildings behind the defense
<code>vehicles_low_speed</code>	10	vehicles at low speed on a road behind the crest
<code>vehicles_moderate_speed</code>	50	driving at moderate speed, pulsating overtopping
<code>embankment_seaward</code>	50	damage to a maintained grass or armored embankment

9.4 Designing a section

`design_rubble_mound(conditions, cot_alpha=2.0, armour="rock_two_layer_permeable", damage=2.0, permeability=0.4, Delta=1.585, tolerable_use="trained_staff", safety_factor=1.0, scatter_factor=3.0)` sizes the stone with Van der Meer and sets the crest so that the **upper** confidence bound on overtopping, not the mean, meets the limit. Designing to the mean would be exceeded about half the time. It returns a `BreakwaterDesign` with `Dn50`, `M50`, `regime`, `crest_freeboard`, `q_mean`, `q_upper`, `layer`, `governing_limit`, `section`, and `summary()`.

The rest of the section is built from the design:

- **Toe scour.** `reflection_coefficient(cot_alpha, xi, permeable=True)` (Seelig and Ahrens 1981, $K_r = a\xi^2/(b + \xi^2)$ with $a = 0.6$, $b = 6.6$ for rubble) and `toe_scour(bed, Hs, T, depth, reflection)`, which scales Xie's standing-wave result by K_r : $S/H_s = 0.4 K_r / \sinh(kh)^{1.35}$. This scaling is an engineering assumption with the right limits, flagged `screening_only`. `breakwater_toe_scour(design, depth, bed)` applies it to a design, gated by `bed_mobility`.
- **Foundation.** `mound_foundation(design, depth, bed="medium_sand", bedding_material="coarse_sand", settlement_allowance=0.0)` returns the bedding blanket on a geotextile, carried past each toe by the larger of the scour apron and twice the scour depth (at least 3 m), and a toe berm of filter stone, which sits low where the orbital velocities are smaller than at the waterline.
- **Crown wall.** `crown_wall(design, still_water_level, deck_width=7.5, parapet_width=2.0, ...)` proportions a stepped concrete crown block and its concrete volume. It is proportioned, not designed: its sliding and overturning under impact and uplift are a separate calculation.
- **Roundhead.** A head is attacked from more directions and its armor gets less support, so practice uses a lower stability coefficient. With a ratio $r = K_{D,\text{head}}/K_{D,\text{trunk}}$ from `roundhead_kd_ratio(armour, cot_alpha)` (Table 8), $D_{n50,\text{head}} = D_{n50,\text{trunk}}/r^{1/3}$ and

$M_{50,head} = M_{50,trunk}/r$. `roundhead(design, kd_ratio=None, raise_crest=0.0)` returns the head design.

Table 8: Head to trunk stability coefficient ratios (ROUNDHEAD_KD_RATIO).

Armor family	slope 1:1.5	1:2	1:3
rock	0.95	0.80	0.65
cubes	0.85	0.75	0.60
tetrapod	0.72	0.64	0.50
accropode	0.80	0.75	0.65
dolos	0.65	0.58	0.45

```
from pyCoastal.applications.structures import (
    DesignConditions, design_rubble_mound, roundhead, mound_foundation,
)

conditions = DesignConditions.from_peak_period(
    Hm0=4.0, Tp=11.0, depth=12.0, storm_duration=6 * 3600,
)
design = design_rubble_mound(conditions, cot_alpha=2.0, tolerable_use="trained_staff")
print(design.summary())
head = roundhead(design) # heavier armor at the head
base = mound_foundation(design, depth=12.0, bed="medium_sand")
```

9.5 Worked example

Output of examples/breakwater_design.py

```
Design condition  Hm0 = 4.00 m, Tm-1,0 = 10.00 s, depth = 12.0 m
Storm            6.0 h, N = 2160 waves
Section          trunk
Slope            1 : 2
Armour           rock_two_layer_permeable, Dn50 = 1.59 m, M50 = 10.7 t (plunging)
Layer            3.18 m thick, 0.50 stones/m2
Crest freeboard  Rc = 5.69 m (Rc/Hm0 = 1.42)
Overtopping      mean 0.333 l/s/m, upper bound 1 l/s/m
Governing limit  1 l/s/m, Trained staff, well shod and protected, wide walkway
```

```
Tolerability of the mean discharge:
NO    0.03 l/s/m  pedestrians_unaware
NO    0.10 l/s/m  pedestrians_aware
ok    0.40 l/s/m  harbour_quay_equipment
ok    1.00 l/s/m  trained_staff
ok    1.00 l/s/m  buildings_structural
ok    10.00 l/s/m vehicles_low_speed
ok    50.00 l/s/m vehicles_moderate_speed
ok    50.00 l/s/m embankment_seaward
```

Wrote media/breakwater_design.png

Wrote media/breakwater_section.png, media/breakwater_sheet.png and media/breakwater_section.dxf

10 Vertical seawalls

Module: `pyCoastal.applications.seawall`. *Example:* `examples/seawall_section.py`.
Browser: Seawall.

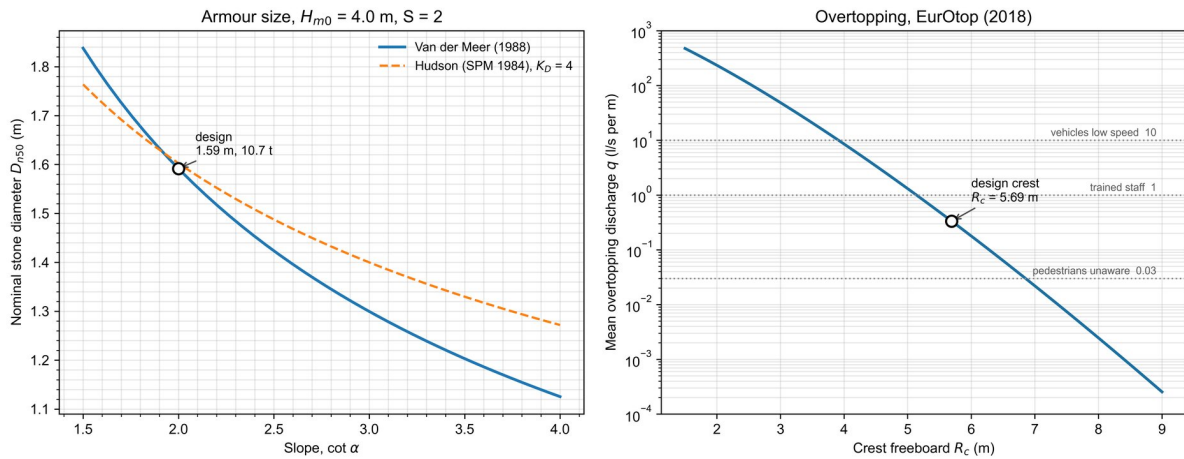


Figure 5: Breakwater design curves: armor size against slope, and overtopping against crest freeboard with the EurOtop scatter band.

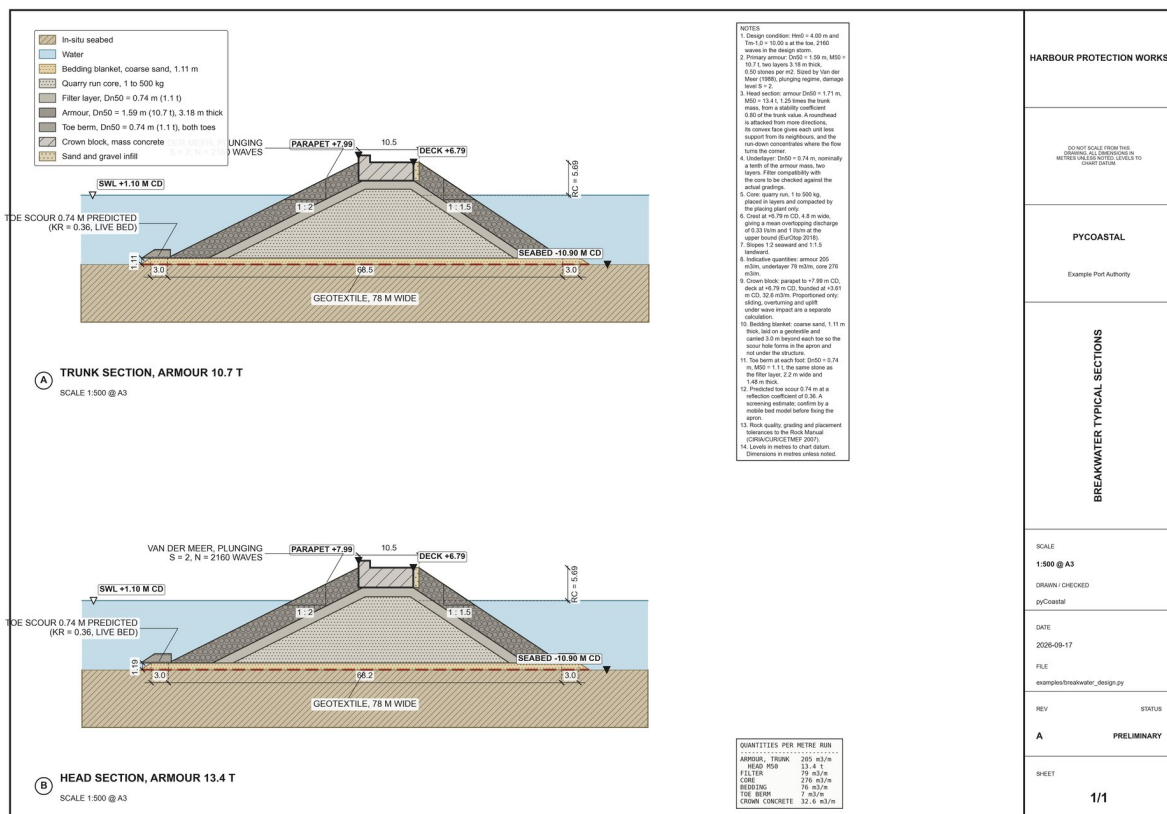


Figure 6: The same design issued as a drawing sheet: trunk and head sections with the armor drawn as individual stones at the computed D_{n50} , the bedding blanket, toe berms, crown wall, and generated specification notes.

The full chain for an L-shaped gravity seawall: crest level from `EurOtop`, founding level from scour, Goda wave pressures on the wetted face, earth and water pressure from the backfill, then the base width grown until sliding, overturning, and the base pressure all pass under every load case. `applications.sections` turns the result into a drawing and a bill of quantities (Section 19).

10.1 Goda pressures

Goda's method (1974, 2010) gives a trapezoidal pressure distribution on the face of a vertical wall. With the design wave $H_{\max} = 1.8H_{m0}$, limited to $\gamma_b d$ in shallow water (`breaker_index=0.78`), the pressure at the still water level, the elevation of the distribution above it, and the pressure at the base of the upright are given by Equation 44:

$$p_1 = \frac{1}{2}(1 + \cos \beta)(\alpha_1 + \alpha_2 \cos^2 \beta)\rho g H_{\max}, \quad \eta^* = 0.75(1 + \cos \beta)H_{\max}, \quad p_3 = \alpha_3 p_1, \quad (44)$$

with the pressure at the crest p_4 interpolated where the crest is below η^* , and a triangular uplift $p_u = \frac{1}{2}(1 + \cos \beta)\alpha_1\alpha_3\rho g H_{\max}$ at the seaward edge. The coefficients in Equation 44 are given by Equation 45:

$$\begin{aligned} \alpha_1 &= 0.6 + \frac{1}{2} \left(\frac{4\pi h/L}{\sinh 4\pi h/L} \right)^2, \\ \alpha_2 &= \min \left[\frac{h_b - d}{3h_b} \left(\frac{H_{\max}}{d} \right)^2, \frac{2d}{H_{\max}} \right], \\ \alpha_3 &= 1 - \frac{h'}{h} \left[1 - \frac{1}{\cosh 2\pi h/L} \right], \end{aligned} \quad (45)$$

where h_b is the depth five wave heights seaward (from the seabed slope), d the depth over the berm, and h' the depth to the underside of the upright. `goda_pressures(Hm0, T, depth, wall_toe_depth, berm_depth=None, crest_freeboard=5.0, beta_degrees=0.0, slope=1/30, Hmax_factor=1.8, breaker_index=0.78)` returns the pressures, `eta_star`, the force F , its lever arm, and the uplift. This is the standard non-impulsive distribution; Takahashi's impulsive coefficient is not applied, so a wall on a high mound needs a separate check.

10.2 Scour, toe, and stability relations

- `scour_depth_vertical_wall(Hm0, T, depth, coefficient=0.4)`: Xie (1981), $S/H = 0.4/\sinh(kh)^{1.35}$, the equilibrium depth a quarter wavelength from the wall, used as a scour allowance for the founding level.
- `toe_stone_size(Hm0, toe_depth, water_depth, Delta=1.585, damage=0.5)`: Van der Meer's toe formula, $H_s/(\Delta D_{n50}) = (2 + 6.2(h_t/h)^{2.7})N_{od}^{0.15}$, calibrated for $0.4 < h_t/h < 0.9$ and flagged outside it.
- `sliding_safety(F, weight, uplift, friction=0.6)`: $FoS = \mu(W - U)/F$, zero when uplift exceeds weight (flotation).
- `overturning_safety(F, arm, restoring_moment, uplift, base_width)`: about the rear heel, with the triangular uplift centroid at two thirds of the base width.
- `bearing_pressures(normal, base_width, net_moment)`: $p_{\max}$, $p_{\min}$, the eccentricity, and `middle_third`; outside the middle third the no-tension triangular distribution is used.
- `hydrostatic_force(depth)`: $\frac{1}{2}\gamma h^2$ at $h/3$, for the water standing in front of the wall when the trough passes.

10.3 Designing a wall

`design_seawall(conditions, still_water_level, seabed_level, tolerable_use="pedestrians_aware", ...)` follows the chain a design office follows:

1. **Crest level** from `EurOtop`, set so the upper bound of the overtopping scatter meets the tolerable limit.
2. **Founding level** from the scour allowance, bracketed by `minimum_embedment` and `maximum_embedment` (deeper is a piling question, not a gravity-wall one).
3. **Goda pressures** on the wetted face, from the still water level down to the seabed.
4. **Earth pressure** from the backfill (`backfill="medium_sand"`, `water_table`, `surcharge`, `earth_pressure_driving="at_rest"`), with the pore water apart.
5. **Base width** grown in `step` increments until sliding, overturning, and, with `require_middle_third`, the bearing check pass under both load cases: the wave crest pushing landward, and the trough with the saturated backfill pushing seaward against only the water left in front (the drawdown case, `drawdown_level`).
6. **Toe stone** from the Van der Meer toe formula, iterated because the berm thickness changes the depth over the berm.

Full Goda uplift is applied under the base even when it is embedded, which is deliberately on the safe side. The result is a `SeawallDesign` with every level and dimension, the pressures and forces, both load cases, `governing_case`, the overtopping, `warnings`, `summary()`, and `quantities()` (concrete, backfill, toe rock, excavation per meter run).

10.4 Worked example

Output of `examples/seawall_section.py`

```
Design condition    Hm0 = 2.80 m, Tm-1,0 = 8.64 s
Water level         SWL +2.90 m CD, depth at wall 8.50 m

Crest level         +8.59 m CD (Rc = 5.69 m, Rc/Hm0 = 2.03)
Promenade level     +7.59 m CD
Founding level      -7.11 m CD (embedment 1.51 m, scour 1.51 m)
Wall height         15.70 m
Base                15.05 m wide x 1.20 m thick, stem 1.00 m
Sizing              73 iterations on base width

Goda Hmax           5.04 m
Pressures           p1 = 45.8 kPa, p3 = 37.5 kPa, pu = 36.8 kPa
Wave force          517 kN/m at 8.08 m above the base
Weight, uplift      2788 kN/m, 277 kN/m

Backfill            Medium sand, phi' = 33 deg, retained 14.70 m
Water table         0.00 m below the surface, saturated to the top
Earth pressure      1643 kN/m at 5.00 m (K = 0.455, 557 soil + 1087 water)

Load case 1, wave crest pushing landward
  sliding           inf
  overturning       inf
Load case 2, trough with the backfill pushing seaward
  front water       253 kN/m at +1.50 m CD
  net seaward       1390 kN/m
  sliding           1.20
  overturning       2.60
Governing case      drawdown
Bearing             p_max = 245 kPa, e = -1.18 m
```

Toe protection	Dn50 = 0.27 m, M50 = 0.05 t, berm 2.0 m wide x 0.55 m thick
Overtopping	mean 0.333 l/s/m, upper bound 1 l/s/m
Governing limit	1 l/s/m, trained staff, well shod and protected, wide walkway

Concrete	32.56 m3/m (78.2 t/m)
Backfill	189.71 m3/m
Toe rock	1.82 t/m
Excavation	22.74 m3/m

Warnings

- Toe depth ratio $ht/h = 0.94$ is outside the 0.4 to 0.9 calibration range of the Van der Meer
↪ toe formula.
- Pore water is 66% of the pressure on the back of the wall. Drainage is a structural matter
↪ here, not a detail: a working drain would cut the total from 1643 to about 854 kN/m.
- The drawdown case governs, not the wave. The wall is sized by the saturated backfill pushing
↪ it seaward at the trough, so the backfill grading and the drainage detail matter more than
↪ the design wave.

Overtopping tolerability at the design discharge

no	0.03 l/s/m	pedestrians_unaware
no	0.1 l/s/m	pedestrians_aware
no	0.4 l/s/m	harbour_quay_equipment
ok	1 l/s/m	trained_staff
ok	1 l/s/m	buildings_structural
ok	10 l/s/m	vehicles_low_speed
ok	50 l/s/m	vehicles_moderate_speed
ok	50 l/s/m	embankment_seaward

Wrote media/seawall_section.png and media/seawall_section.dxf
Wrote media/seawall_sheet.png

Two warnings in this output are the point of the example. Pore water is 66% of the pressure on the back of the wall, and the drawdown case governs, not the wave: the wall is sized by the saturated backfill pushing it seaward at the trough, so the backfill grading and the drainage detail matter more than the design wave. The toe depth ratio is also just outside the calibration range of the toe formula, and the output says so.

11 Harbor agitation and port layout

Module: pyCoastal.applications.port. *Examples:* examples/port_diffraction.py, examples/port_layout_comparison.py.

Phase-resolved wave propagation into a harbor, reported as the wave energy that reaches the berths. Breakwaters are rasterized onto the grid as reflecting walls with a settable absorption, waves enter from a soft source line, and the open boundaries are damped by sponge layers so that outgoing and reflected energy leaves the domain instead of ringing around it.

11.1 The solver

The model integrates the second-order wave equation in flux form (Equation 46),

$$\frac{\partial^2 \eta}{\partial t^2} = \nabla \cdot (c^2 \nabla \eta), \quad (46)$$

with zero flux on every face touching a breakwater cell, by an explicit leapfrog step at a Courant number of 0.35. The celerity comes from the linear dispersion relation (Equation 28) at the local depth and the run period, so the wavelength is correct in intermediate water rather than the

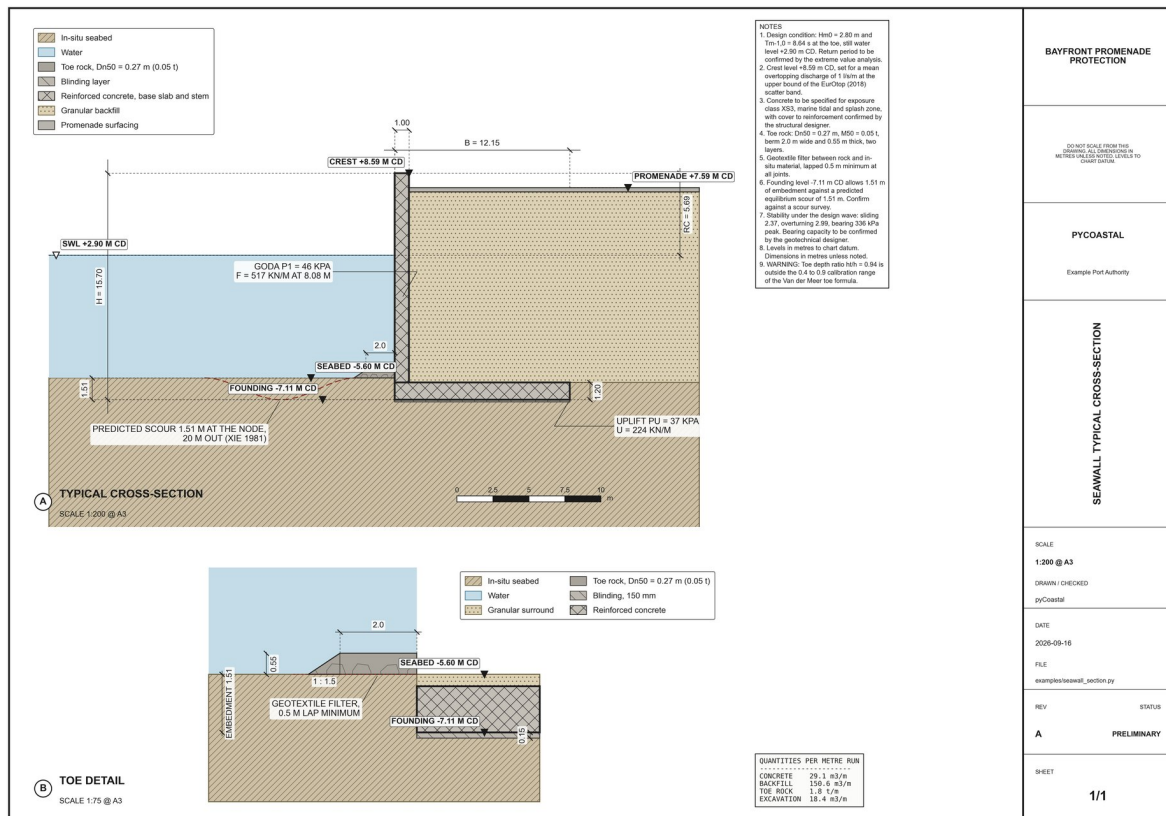


Figure 7: The seawall drawing sheet: typical section at a true stated scale, an enlarged toe detail, the design parameter block, and specification notes generated from the design.

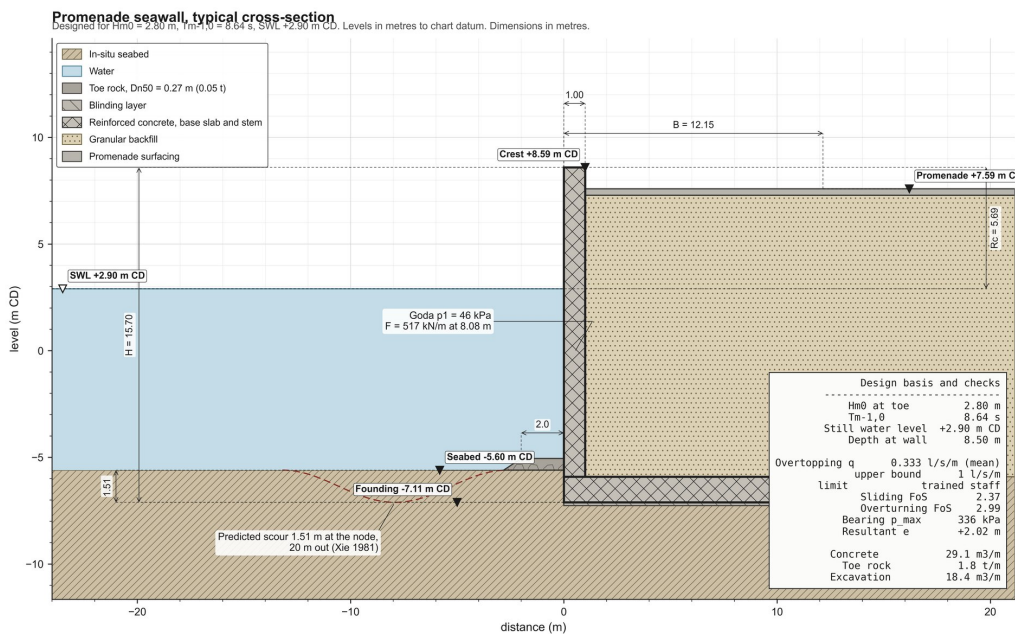


Figure 8: Standalone seawall section with plot axes, for a report.

shallow-water $\sqrt{gh}$. The model is therefore accurate for the single frequency it is run at. It is not a spectral model and does not represent breaking, wave-current interaction, refraction over a varying bed, or nonlinear transfers.

Harbor performance is the disturbance coefficient (Equation 47)

$$K_d = \frac{H_{\text{local}}}{H_{\text{reference}}}, \quad (47)$$

with the reference height measured at a probe in open water outside the harbor, which makes the result independent of how the source is calibrated. Against a semi-infinite breakwater the solver gives a deep quiet shadow, K_d near 0.5 on the geometric shadow boundary, and Fresnel fringes in the illuminated field.

11.2 Geometry

`Breakwater(points, width=20.0, name="breakwater", absorption=0.0)` is a polyline of a given thickness; cells within `width/2` of the centerline become land. `absorption` is a damping collar in the water next to the structure, a model knob rather than the reflection coefficient itself. `measure_reflection(absorption, period, depth)` fires a normally incident wave at a full-width wall and infers $K_r = (H_{\text{max}} - H_{\text{min}})/(H_{\text{max}} + H_{\text{min}})$ from the standing-wave envelope (Table 9). The mapping is not monotonic: past about 0.75 the collar becomes an impedance step and reflects again. Use 0.3 to 0.4 for a rubble mound (K_r near 0.4) and 0 for a vertical caisson.

Table 9: Reflection coefficient against the absorption setting.

absorption	0.00	0.25	0.50	0.75	1.00
K_r	0.95	0.48	0.22	0.19	0.25

`PortLayout(Lx=1200, Ly=900, dx=4.0, depth=10.0, breakwaters=[])` holds the domain, grid spacing, constant depth, and structures. Five layouts are built in (`LAYOUTS`):

- `harbour_layout ("two_arm")`: two shore-normal arms with a gap entrance, and optionally a back wall (drop "east" from the sponge sides so the quay reflects);
- `offset_entrance_layout ("offset_entrance")`: overlapping arms making a dog-leg entrance, so waves must diffract twice;
- `hooked_breakwater_layout ("hooked")`: a main breakwater with a shore-parallel hook, so the entrance faces along the coast;
- `detached_breakwater_layout ("detached_screen")`: two arms with a detached screen off the gap;
- `marina_layout ("marina")`: an outer harbor protecting an inner basin through an offset second opening.

Wave direction. `IncidentWave(height=1.5, period=8.0, direction=0.0)` supports an oblique direction, but a phased source line only lights a parallelogram of the domain (`illuminated_mask`), and `simulate_port` warns when a structure falls outside it. The accurate way to study direction is to turn the harbor into the waves with `rotate_layout(layout, degrees)` and drive it shore-normal, since only the relative angle matters.

Absorption matters. Left fully reflecting, a detached screen forms a pocket with the arms that rings, and basin agitation goes up. For a 150 m gap with 280 m arms at $T = 9$ s, mean basin

K_d went from 0.35 to 0.51 with a reflecting screen, and from 0.20 to 0.04 once the structures were armored.

11.3 Running a simulation

```
import numpy as np
from pyCoastal.applications.port import IncidentWave, harbour_layout, simulate_port

layout = harbour_layout(gap=130, arm_length=300, back_wall=True, absorption=0.35)
wave = IncidentWave(height=1.5, period=9.0, direction=np.deg2rad(0))

result = simulate_port(layout, wave, sponge_sides=("west", "north", "south"))

result.disturbance_coefficient      # Kd over the whole basin
result.probe((1120, 450))          # Kd at one point
result.berth_report({"quay": (1120, 450)})
result.operable_fraction({"quay": (1120, 450)}, limit=0.5)

simulate_port(layout, wave, duration=None, cfl=0.35, sponge_thickness=
None, sponge_strength=2.0, sponge_sides=..., source_x=None, ramp_periods=3.0,
analysis_periods=6.0, n_snapshots=120, store_from=None, reference_point=None) re-
turns a PortResult with the coordinate axes, stored surface snapshots, the land mask, the
analysis-window wave_height, and the reference_height. The default duration lets the wave
cross the domain twice plus the ramp and analysis windows; the sponge defaults to 1.5 wave-
lengths.
```

11.4 Worked examples

examples/port_diffraction.py drives a 9 s swell at 25 degrees into a two-arm harbor (by rotating the layout), writes an animation of the instantaneous surface, and reports K_d at four berths:

Output of examples/port_diffraction.py

```
Wavelength      : 81.7 m   (20 cells per wave)
Celerity        : 9.08 m/s
Entrance gap    : 130 m = 1.59 wavelengths
Approach angle  : 25 deg off shore-normal
Incident height : 1.50 m
```

berth	Kd	Hs (m)
entrance	0.12	0.18
outer basin	0.33	0.49
north berth	0.55	0.82
south berth	0.13	0.20

```
Berths within a 0.5 m operational limit:
entrance    yes
outer basin  yes
north berth  no
south berth  yes
```

```
Wrote media/port_disturbance.png
Wrote media/port_diffraction.gif
```

examples/port_layout_comparison.py runs all five layouts under the same design wave and ranks them by basin agitation:

Output of examples/port_layout_comparison.py

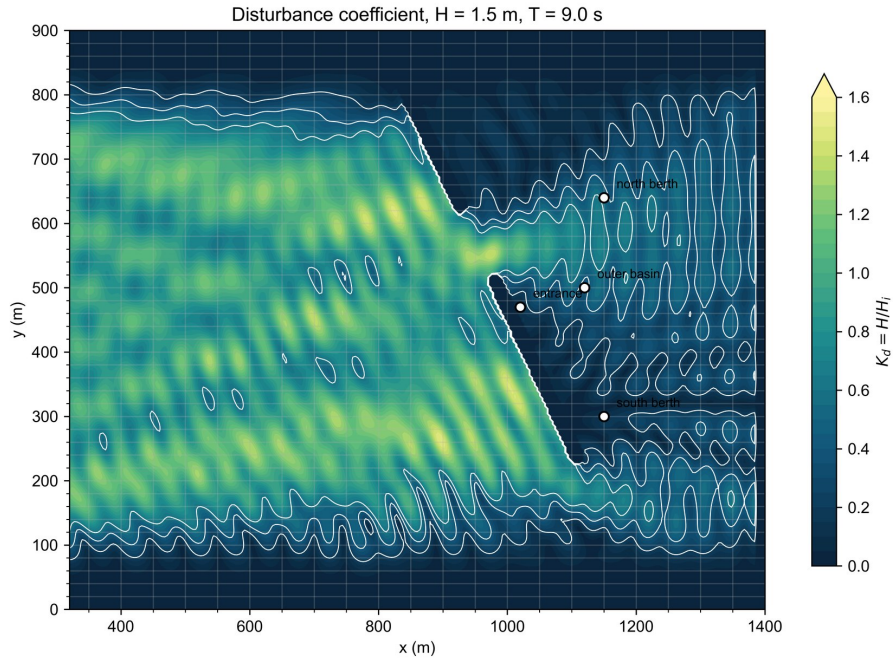


Figure 9: Disturbance coefficient in the harbor. The north berth sits in the diffracted field from the entrance and exceeds the 0.5 m limit.

```

ran two_arm
ran offset_entrance
ran hooked
ran detached_screen
ran marina

```

layout	mean Kd	95th pct Kd
Outer harbour and inner basin	0.007	0.022
Overlapping arms, dog-leg	0.008	0.029
Detached screen off the gap	0.039	0.067
Two arms, straight gap	0.146	0.370
Hooked main breakwater	0.180	0.768

Quietest basin: Outer harbour and inner basin (0.007 mean Kd)

Noisiest basin: Hooked main breakwater (0.180 mean Kd)

Wrote media/port_layouts.png

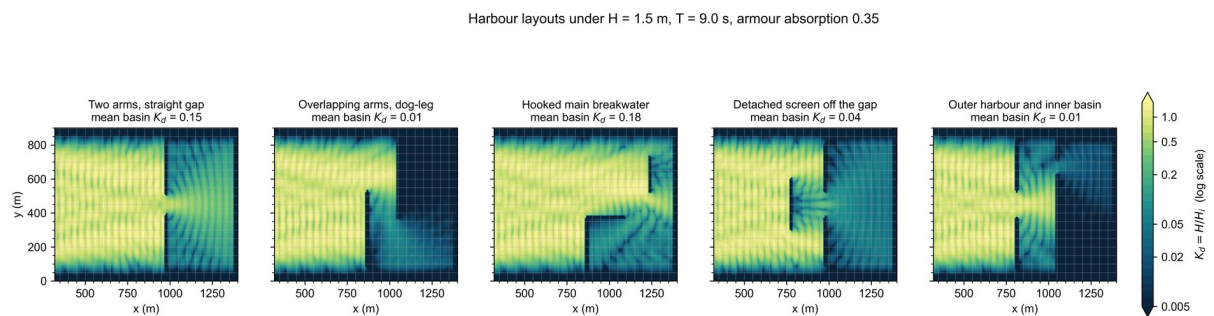


Figure 10: The five built-in layouts under the same design wave.

12 Wave loads on piles

Module: `pyCoastal.applications.piles`. *Example:* `examples/pile_wave_loads.py`.
Browser: Monopile.

The load on a pile is not one number. It is a distribution up the pile that changes through the wave cycle, and the two things a designer needs from it, the base shear and the overturning moment at the mudline, peak at different phases. This module computes the distribution, integrates it, and sweeps the phase, rather than evaluating a formula at the crest.

12.1 The Morison equation

The Morison equation (Morison et al. 1950) splits the load per unit length on a slender member into a drag term in phase with the velocity and an inertia term in phase with the acceleration:

$$f(z) = \frac{1}{2}\rho C_d D u|u| + \rho C_m \frac{\pi D^2}{4} \frac{\partial u}{\partial t}. \quad (48)$$

Slender means D/L below about 0.2; above that the wave diffracts around the member and the functions warn rather than return a number.

Kinematics. `wave_kinematics(H, T, depth, z, phase=0.0, stretching="wheeler")` gives linear-theory velocity and acceleration. Linear theory says nothing about the water between the still water level and the crest, which is exactly where the load is largest, and extrapolating the cosh profile there overstates the velocity badly. Wheeler (1970) stretching maps the still-water profile onto the instantaneous water column and is the default; `"none"` and `"extrapolate"` are available for comparison.

Regime and coefficients. The Keulegan-Carpenter number $KC = u_{\max}T/D$ sets which term dominates: below about 3 the load is almost all inertia, above about 20 drag governs. `drag_inertia_coefficients(KC, rough=True)` returns post-critical values from DNV-RP-C205: $C_d = 1.05$ for a rough cylinder (any pile with marine growth) and 0.65 for a clean one, with C_m falling from the potential-flow value of 2.0 as KC rises. These are concept-design starting values.

Integration and sweep. `morison_load_profile` evaluates Equation 48 up the pile, `integrate_load(z, load, mudline)` integrates the force and the mudline moment, and `morison_pile_load(diameter, H, T, depth, phase=0.0, ...)` returns a `PileLoad` at one phase. `phase_sweep(diameter, H, T, depth, phases=181)` sweeps the cycle and reports where each peaks. `design_monopile(diameter, H, T, depth, rough=True, stretching="wheeler", phases=181, bed=None)` returns the worst-phase load, the scour, and the ratio `crest_underestimate` by which a crest-phase evaluation would understate the moment.

12.2 Scour at a pile

Sumer, Fredsoe and Christiansen (1992), for waves:

$$\frac{S}{D} = 1.3 \left[1 - e^{-0.03(KC-6)} \right] \quad (KC > 6), \quad (49)$$

with no scour below $KC = 6$, where the horseshoe vortex does not form. Under a steady current $S/D = 1.3$ with a standard deviation of 0.7, which is the `current_only` branch and the limit of the wave relation. `scour_depth_pile(diameter, KC, current_only=False, live_bed=True, bed=None, ...)` returns the equilibrium depth; pass a bed to gate it by mobility.

```

from pyCoastal.applications.piles import design_monopile

result = design_monopile(diameter=8.0, H=12.0, T=13.0, depth=30.0)
result["load"].force / 1e3      # kN, at the worst phase
result["load"].moment / 1e6     # MNm about the mudline
result["load"].inertia_fraction # share carried by the inertia term
result["crest_underestimate"]   # what a crest-phase check would miss

```

12.3 Worked example

Output of examples/pile_wave_loads.py

```

Pile          8.00 m diameter in 30.0 m of water
Wave          H = 12.00 m, T = 13.0 s
KC, regime    6.3, mixed
Cd, Cm        1.05, 1.87
D / L         0.041

```

```

Worst phase    56 degrees (crest at 0), surface at +3.36 m
Base shear     4,297 kN
Mudline moment 77.2 MNm
Effective arm   17.96 m above the mudline
Inertia share   90% of the load

```

Warnings

- The worst moment is 62% larger than the moment at the crest phase. Designing on the crest alone would have understated it.

```

At the crest phase    1,437 kN, 29.6 MNm
At the worst phase    4,297 kN, 77.2 MNm
Designing on the crest would miss 62% of the moment

```

```

Max base shear at    70 degrees
Max mudline moment at 56 degrees

```

```

Scour          0.10 m (0.01 D) at KC = 6.3
               under a steady current: 10.40 m (1.3 D, standard deviation 5.60 m)
               waves alone barely scour a pile this large, because KC is small; current is what governs.

```

How the load splits with pile diameter

D (m)	KC	regime	inertia share	moment (MNm)
1.0	50.7	drag dominated	5%	3.8
2.0	25.3	drag dominated	15%	8.1
4.0	12.7	mixed	50%	20.7
8.0	6.3	mixed	90%	77.2
12.0	4.2	mixed	96%	176.0

Wrote media/pile_wave_loads.png

A large monopile is inertia dominated, so the load follows the fluid acceleration, which peaks a quarter cycle before the crest. Evaluating the load under the crest, because that is where the water is highest, understates the overturning moment by 62% here. That is what the phase sweep exists to catch. The diameter table shows the transition from drag to inertia dominance as the pile grows, and the scour result shows that waves alone barely scour a pile this large because KC is small; current is what governs.

13 Scour at piers and bridges

Module: `pyCoastal.applications.scour`. Examples: `examples/pier_scour.py`, `examples/bridge_scour.py`.

Monopile 8 m in 30 m: H = 12 m, T = 13 s. 4,297 kN and 77 MNm, 90% inertia

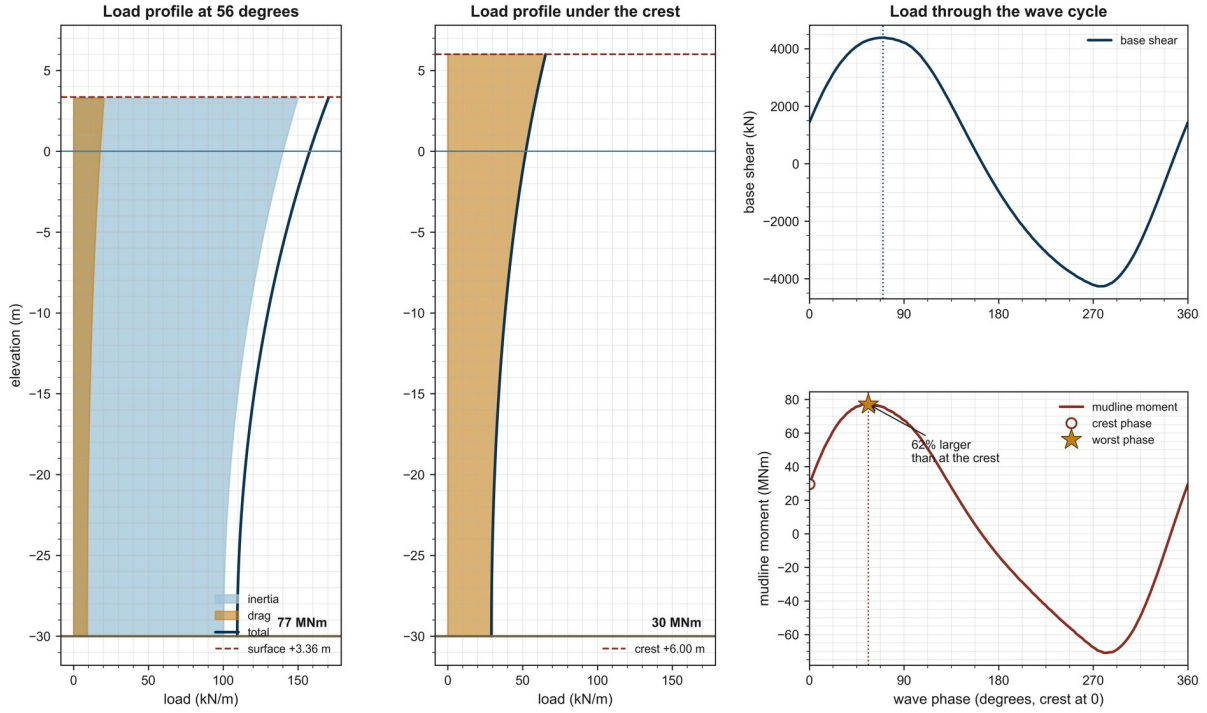


Figure 11: Morison loads on an 8 m monopile: load profiles at the crest and worst phases, and force and moment through the wave cycle.

The hard part of estuarine scour is not any single relation. It is that the drivers do not peak together and do not even point the same way. The tide reverses twice a day, the river does not, the waves come and go with the wind, and the water depth changes under all of it. A calculation done at “the design current” and “the design wave” without asking whether they can occur at the same moment is either optimistic or absurd, and it is usually not obvious which. So this module works a tidal cycle rather than a load case: it evaluates the scour at every phase and reports the envelope, which phase governs, and by how much.

13.1 Pier scour in combined waves and current

Geometry. A pier is a stem (`Pier(diameter, shape="circular", length=None)`) on a base (`PierBase(width, length, height, top_level, skew)`), because that is what piers are, and the base is where the interesting failure lives. A buried pile cap does nothing until the scour hole reaches it; then it is exposed, it is wider than the stem, and the scour deepens because of it, which exposes more of it. The obstacle is reduced to one width by weighting each element over the depth of flow it occupies (Equation 50):

$$D_e = \frac{D_{\text{base}} h_{\text{base}} + D_{\text{stem}} (h - h_{\text{base}})}{h}, \quad (50)$$

where h_{base} is how much of the base stands proud of the *scoured* bed. Each element carries the HEC-18 shape factor K_1 (`SHAPE_FACTOR`) and the alignment factor (Equation 51)

$$K_2 = \left(\cos \theta + \frac{L}{a} \sin \theta \right)^{0.65}, \quad L/a \leq 12. \quad (51)$$

A base set square to the ebb is skewed to the flood (the flood skew is taken as the supplement), which is what makes a tidal estuary awkward.

Combined waves and current. Sumer and Fredsoe (2001) (Equation 52):

$$\frac{S}{D} = 1.3 \{1 - \exp[-A(KC - B)]\}, \quad A = 0.03 + \frac{3}{4}U_{cw}^{2.6}, \quad B = 6 \exp(-4.7U_{cw}), \quad (52)$$

with $KC = U_m T/D$ on the *near-bed* orbital velocity and $U_{cw} = U_c/(U_c + U_m)$. At $U_{cw} = 0$ it is exactly the waves-only relation (Equation 49); at $U_{cw} = 1$ the threshold vanishes and $S/D \rightarrow 1.3$ as KC grows.

The steady-current floor. KC is built on the wave orbital velocity, so a strong current under small waves has a high U_{cw} and a low KC at the same time, and the bare formula returns almost no scour: a pier in a 1.5 m/s current would scour less than in still water with a ripple on it. A steady current is the $KC \rightarrow \infty$ limit of an oscillatory flow, not the $KC \rightarrow 0$ one, and a current that moves the bed digs its own horseshoe vortex. So when the approach current alone is live-bed, the ratio is floored at the steady-current value. This is an addition to the published relation and is flagged as `current_governs` (`scour_ratio_combined(KC, Ucw, current_live_bed=..)`; `None` disables it).

Depth limitation and time scale. `depth_limitation(depth, diameter)` applies $\tanh(h/D)$ (Breusers et al. 1977), worth nothing above $h/D = 3$ and a quarter below $h/D = 1$. The skin-friction Shields parameter under the current uses $u_* = U\kappa/\ln(11h/k_s)$ with $k_s = 2.5d_{50}$, and the time scale (Sumer, Christiansen and Fredsoe 1992) is given by Equation 53,

$$T^* = \frac{1}{2000} \frac{h}{D} \theta^{-2.2}, \quad T = T^* \frac{D^2}{\sqrt{g(s-1)d_{50}^3}}, \quad S(t) = S_{eq} [1 - e^{-t/T}]. \quad (53)$$

For a pier a couple of meters across T is typically most of a day, while the tide reverses every six hours.

Feedback. `equilibrium_scour(pier, base, depth, current, Um, period, ...)` solves the coupling between scour and effective diameter for its *deepest* self-consistent root by bisection, because a scour hole does not refill and the shallow root is not an outcome the pier can stay at.

Protection. `riprap_size(velocity)` follows HEC-23 guideline 12, $d_{50} = 0.692(KV)^2/[2g(s-1)]$ with $K = 1.5$ for round-nosed and 1.7 for rectangular piers, and `scour_protection(pier, base, velocity, scour)` sizes a falling apron reaching two obstacle widths from the face, at least three stones thick, with a launch allowance: an apron relocates the scour to its edge rather than removing it.

The tidal cycle. `EstuaryConditions(mean_depth, tidal_amplitude, tidal_current, river_current, Hs, Tp, tidal_period=M2, current_phase=90, bed="medium_sand", wave_follows_tide=False)` uses positive for ebb: the river adds to the tide on the ebb and opposes it on the flood, which is why estuarine scour is usually an ebb problem. `current_phase` sets the lead of the current over the elevation (90 degrees for a standing tide, 0 for a progressive one). `design_pier_scour(pier, conditions, base, samples=73, protection_factor=1.0)` evaluates every phase and returns a `PierScourDesign` with the equilibrium scour (for protection and the long-term foundation check), the `tidal_limited` scour one half cycle can cut, `base_exposed`, `undermined`, the `envelope`, and the `protection`.

```

from pyCoastal.applications.scour import (
    Pier, PierBase, EstuaryConditions, design_pier_scour,
)

pier = Pier(diameter=2.5, shape="circular")
base = PierBase(width=7.0, length=12.0, height=2.5, top_level=-1.5, skew=20.0)
estuary = EstuaryConditions(mean_depth=9.0, tidal_amplitude=2.2,
                             tidal_current=1.1, river_current=0.4,
                             Hs=1.2, Tp=5.5, bed="medium_sand")
design = design_pier_scour(pier, estuary, base)
design.equilibrium, design.tidal_limited, design.undermined, design.protection

```

13.1.1 Worked example: pier scour

Output of examples/pier_scour.py

Pier 2.5 m on a 7 x 12 m base, top -1.5 m to the bed
Peak ebb 1.50 m/s, peak flood 0.70 m/s

Governing phase	145 deg (ebb)
current	0.78 m/s (peak is 1.50)
depth	7.20 m
effective diameter	5.15 m (stem is 2.50)
equilibrium scour	5.93 m
one tidal half	1.76 m (time scale 17.7 h)
base	UNDERMINED

- Governing phase 145 deg, on the ebb: 0.78 m/s current and 0.47 m/s near-bed orbital velocity
- ↪ in 7.20 m of water.
- $U_{cw} = 0.62$, so the waves and current both matter. $KC = 0.5$ against a threshold of 0.32.
- Equilibrium scour 5.93 m, 1.30 times the effective diameter of 5.15 m.
- Shallow water takes 12% off the unlimited depth: $h/D = 1.4$.
- The hole exposes the base completely (2.5 m) and reaches its underside. The foundation is
- ↪ undermined, not merely exposed, and the bearing check has to be redone on the scoured section.
- Flow misalignment multiplies the base width by 1.32 at 20 deg of skew. A base set square to
- ↪ one direction of a reversing tide is skewed to the other.
- Scour time scale 17.7 hours against a 6.2 hour tidal half cycle, so one half cycle reaches
- ↪ 1.76 m, 30% of equilibrium. The hole does not refill on the reverse flow, so successive tides
- ↪ work it deeper towards the equilibrium. Size protection on the equilibrium either way.
- Sumer and Fredsoe's relations carry a standard deviation near 0.7 diameters in steady
- ↪ current. Treat the number as a mean, not a bound, and design protection with margin on it.

Wrote media/pier_scour.png, media/pier_scour_sheet.png and media/pier_scour_tide.png

Two findings fall out of working the cycle. **The worst phase is not peak current**: once the footing is exposed, a wide base counts for more in shallow water, so the governing phase here is well after peak ebb, at 0.78 m/s rather than the 1.50 m/s peak. **Burial depth is a cliff**: bury the base deeper than the bare stem would scour and nothing happens; a meter shallower and the hole runs away to nearly twice the depth.

13.2 Total bridge scour

Local scour at a pier is the component everyone computes, and on a contracted crossing it is routinely the smallest of the three. HEC-18 splits total scour into contraction, local, and abutment scour; all three are here and all three are worked over the tidal cycle.

The opening. BridgeOpening(approach_width, opening_width, pier_blockage=0, abutment_length=25, abutment_shape="spill_through", abutment_skew=90, slope=5e-4, flow_fraction=1.0, abutment_depth_fraction=0.4) gives the contraction ratio

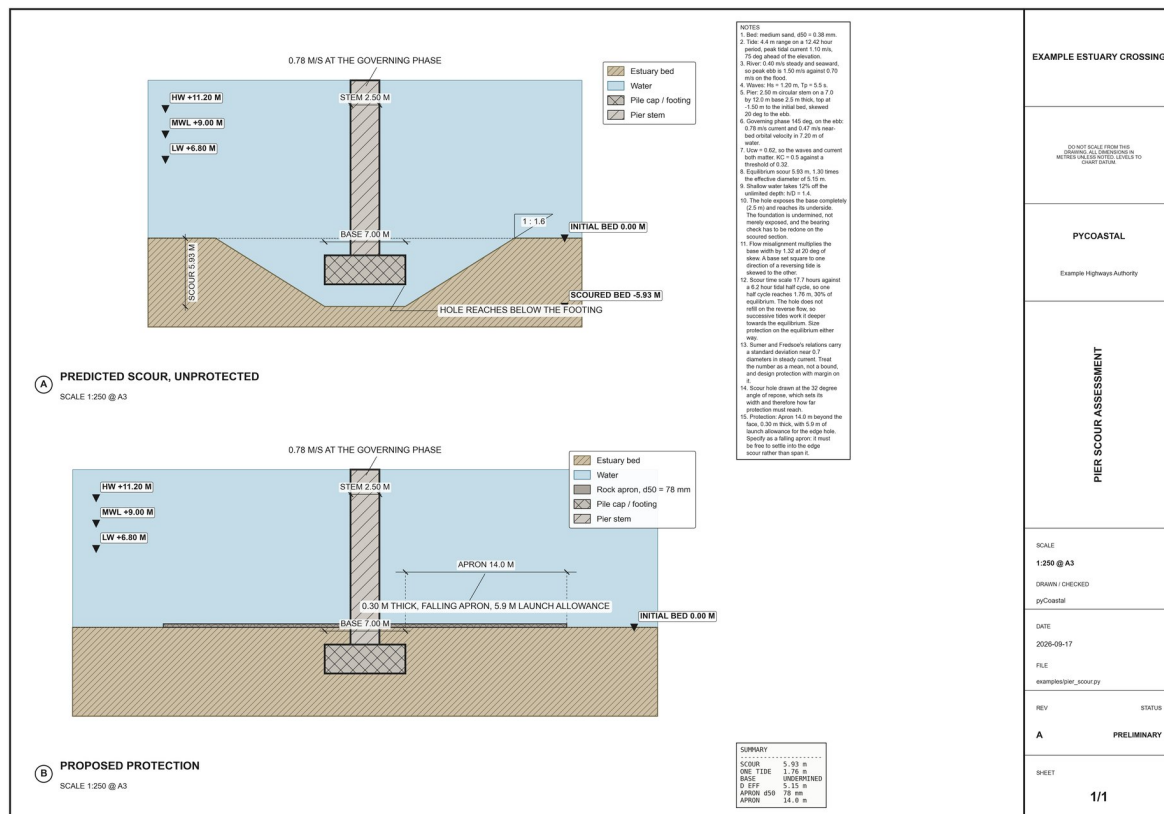


Figure 12: Pier scour assessment sheet. View A is the prediction if nothing is done; view B is the proposed apron on an intact bed. An apron and a fully developed hole cannot share one section, because the apron exists to stop that hole.

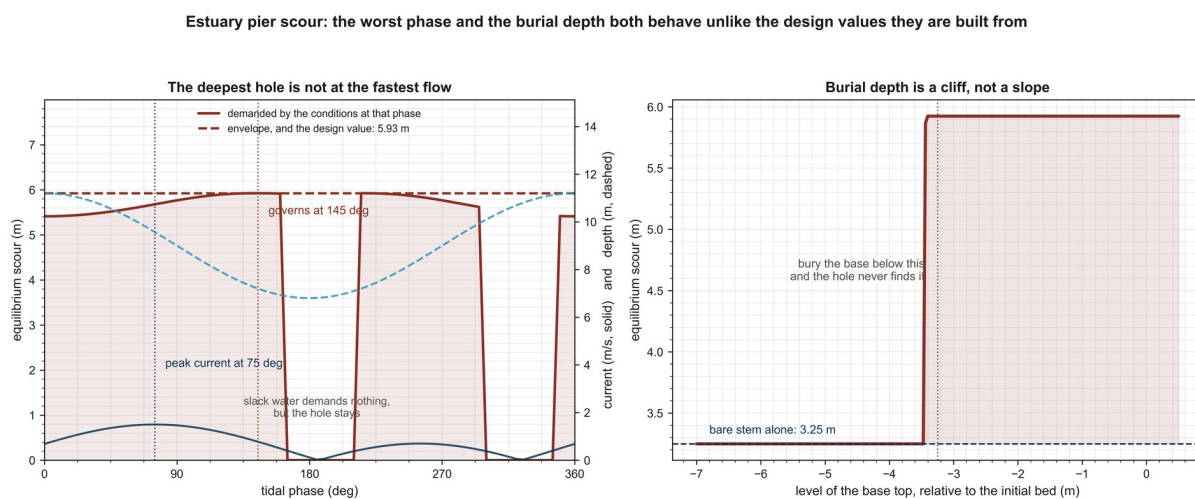


Figure 13: Scour through the tidal cycle and against the burial depth of the base.

W_1/W_2 with the piers subtracted from the gross opening. `flow_fraction` is Q_2/Q_1 , best computed with `river.flow_distribution` (Section 14). `abutment_depth_fraction` sets Froehlich's y_a , the depth where the embankment actually stands; feeding it a mid-channel depth in a deep estuary returns a hole deeper than the water.

Contraction scour. Two modes, switched by the critical velocity $V_c = 6.19 y^{1/6} D_{50}^{1/3}$ (HEC-18 eq. 6.1). Live bed, Laursen (1960) (Equation 54):

$$\frac{y_2}{y_1} = \left(\frac{Q_2}{Q_1} \right)^{6/7} \left(\frac{W_1}{W_2} \right)^{k_1}, \quad (54)$$

with $k_1 = 0.59, 0.64$, or 0.69 as V_*/w is below 0.5 , between 0.5 and 2 , or above 2 (`transport_exponent`). Clear water (HEC-18 eq. 6.4) (Equation 55):

$$y_2 = \left[\frac{0.025 Q_2^2}{D_m^{2/3} W_2^2} \right]^{3/7}. \quad (55)$$

The live-bed relation is a sediment balance: faster water carries more but delivers more, and the two cancel, leaving the width ratio. The two relations were fitted separately and do not meet at the threshold; `contraction_scour(..., regime="live" | "clear")` computes either on demand so the step can be seen.

Abutment scour. Froehlich (1989) for $L'/y < 25$ and HIRE for longer abutments (Equation 56):

$$\frac{y_s}{y_a} = 2.27 K_1 K_2 \left(\frac{L'}{y} \right)^{0.43} Fr^{0.61} + 1, \quad \frac{y_s}{y_a} = 4 Fr^{0.33} \frac{K_1}{0.55} K_2, \quad (56)$$

with K_1 from `ABUTMENT_SHAPE` (1.0 vertical, 0.82 wing wall, 0.55 spill through). The $+1$ in Froehlich is a factor of safety HEC-18 added, not physics, and it is often more than half the answer; it is reported as `safety_margin`. These are the fourth-edition HEC-18 relations, the edition the pier scour also comes from.

Chained, not parallel. `bridge_scour_state` computes the components in the order the water meets them: contraction from the approach flow, then local scour from the flow *in the contracted opening*, which is faster and deeper, and abutment scour from the approach flow it turns. They are added only where they coexist: contraction plus pier at a pier, contraction plus abutment at an abutment. `design_bridge_scour(opening, conditions, pier, base)` envelopes each component over the cycle and returns a `BridgeScourDesign` with `contraction`, `pier_local`, `abutment`, `total_at_pier`, `total_at_abutment`, `component(name)`, and notes. When a total exceeds the water depth the result says so: that is a signal the relations have been pushed past their fitted range, not a foundation level.

```
from pyCoastal.applications.scour import BridgeOpening, design_bridge_scour

opening = BridgeOpening(approach_width=140, opening_width=80, pier_blockage=5,
                        abutment_length=22, abutment_shape="spill_through")
design = design_bridge_scour(opening, estuary, pier, base)
design.total_at_pier, design.total_at_abutment, design.component("pier")
```

13.2.1 Worked example: bridge scour

Output of examples/bridge_scour.py

Waterway 140 m, opening 80 m gross less 5 m of piers
Contraction 1.87 to 1

Governing phase 35 deg (ebb)

approach 1.24 m/s in 10.80 m
through the opening 1.51 m/s in 16.62 m

contraction 6.03 m
pier local 5.93 m
abutment 8.37 m

at a pier 10.72 m
at abutment 14.09 m <- governs

- Opening 80 m gross less 5.0 m of piers in a 140 m waterway: a 1.87 to 1 contraction.
- Governing phase 35 deg on the ebb: 1.24 m/s approaching in 10.80 m, becoming 1.51 m/s in 16.62 m through the opening.
- ↪ - Live-bed contraction scour: the approach at 1.24 m/s is 1.9 times the 0.67 m/s threshold, so the opening is fed from upstream and scours only until it can pass what arrives. Note that this mode does not depend on how fast the water goes: Laursen's relation is a sediment balance, and above the threshold it turns on the width ratio alone.
- ↪ - Components: contraction 6.03 m, pier local 5.93 m, abutment 8.37 m. Each is the worst over the whole cycle; they do not all peak at the same phase.
- ↪ - Total 10.72 m at a pier and 14.09 m at an abutment. The abutment governs. These are not added together: they are depths at two different places, and combining them would invent a hole that exists nowhere.
- ↪ - Abutment scour by Froehlich, of which 4.32 m (52%) is the +1 flow depth HEC-18 adds as a factor of safety rather than as physics. Worth knowing before it is treated as a measurement.
- ↪ - Local scour is computed on the flow in the contracted opening, not on the approach, since the contraction has already accelerated and deepened it. Taking approach values here is the usual way a pier gets understated.
- ↪ - A total of 14.09 m exceeds the 10.80 m of water at the governing phase. That is a signal the relations have been pushed past where they were fitted, not a foundation level. Check the abutment depth and the contraction ratio, and get a physical model or a morphodynamic run before committing to it.
- ↪ - Long-term degradation or aggradation of the reach is not included and is a separate study. HEC-18 adds it to these; on a river with a moving bed it can exceed all three.

Wrote media/bridge_scour.png, media/bridge_scour_sheet.png and media/bridge_scour_components.png

13.3 What is not modeled

A pile cap at bed level can act as a collar and reduce scour; crediting it would be unconservative, so it is not credited. Cohesive beds are refused rather than guessed at. Long-term degradation of the reach and channel migration are separate studies that HEC-18 adds to these totals.

14 Open-channel hydraulics and backwater

Module: pyCoastal.applications.river. *Example:* examples/backwater.py.

The foundation the fluvial side of a crossing stands on. Everything a bridge does to a river it does by changing the depth: it squeezes the flow, the water backs up, and that backwater reaches upstream for a distance nobody guesses correctly by eye. The module also exists for one number in the scour chain: the fraction of the discharge that actually goes through a bridge opening, which is a conveyance calculation, not a guess.

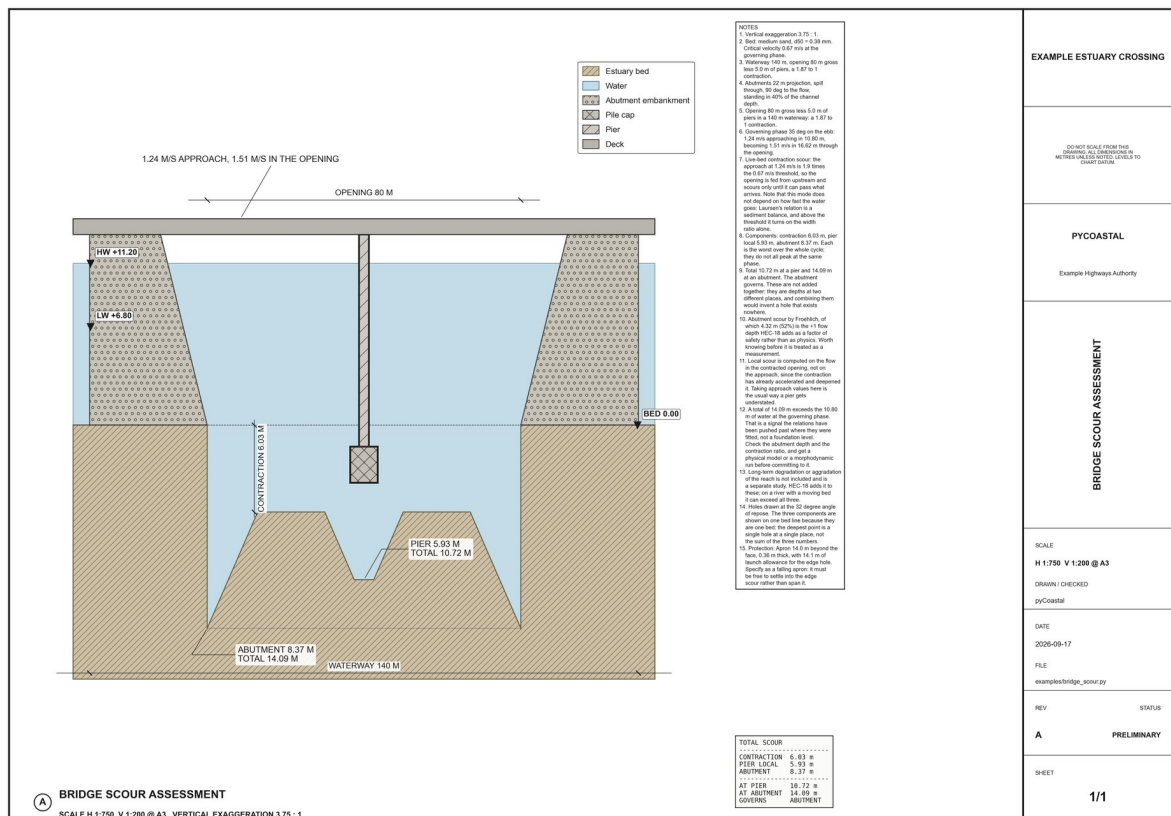


Figure 14: Bridge scour assessment sheet. The three components are drawn on one bed line so that the deepest point reads as one hole at one place, not the sum of three numbers.

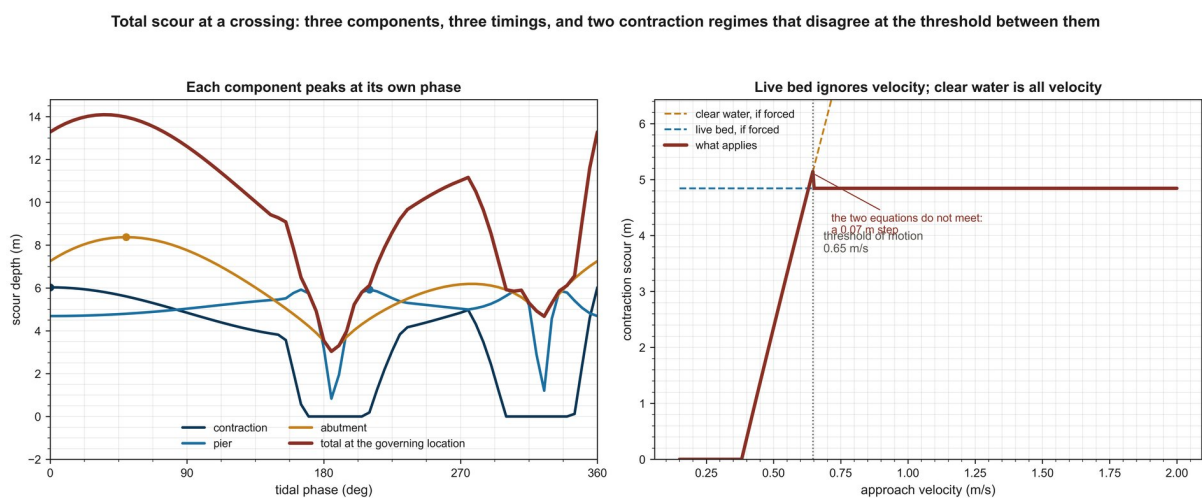


Figure 15: Scour components through the tidal cycle. They peak at different phases, so each has its own envelope.

14.1 The channel

`Channel(width=30.0, side_slope=2.0, roughness="natural_clean")` is a trapezoidal prismatic section (zero width is triangular, zero side slope rectangular). Roughness is a Manning n or a key into MANNING: 0.013 and 0.017 for smooth and rough concrete, 0.022 and 0.030 for clean and weedy earth, 0.028 gravel, 0.035 cobbles, 0.030 and 0.045 for clean and weedy natural channels, and 0.035, 0.07, 0.10 for pasture, brush, and wooded floodplains. The section provides `area`, `perimeter`, `top_width`, `hydraulic_radius`, `velocity`, and the conveyance (Equation 57)

$$K = \frac{AR^{2/3}}{n}, \quad Q = K\sqrt{S}. \quad (57)$$

14.2 Two depths

Normal depth is where friction balances the bed slope, $Q = K(y_n)\sqrt{S_0}$ (`normal_depth`, which raises on a horizontal or adverse slope, where uniform flow does not exist). **Critical depth** is where the specific energy $E = y + V^2/2g$ is least and the Froude number is one, $Q^2T/(gA^3) = 1$ (`critical_depth`), with $Fr = V/\sqrt{gA/T}$ (`froude_number`), the form that is right for any section. `classify_slope(channel, discharge, slope)` compares them: mild, steep, critical, horizontal, or adverse. The classification belongs to the channel and the discharge together, so the same reach can be mild in flood and steep at low flow. `profile_type(depth, classification)` names the profile in Chow's notation: M1 behind a bridge on a mild reach, M2 a drawdown to a free overfall, S1 behind a structure on a steep reach.

14.3 Gradually varied flow

`gvf_profile(channel, discharge, slope, control_depth, steps=200, approach=0.99)` integrates the direct step method (Equation 58),

$$\Delta x = \frac{\Delta E}{S_0 - S_f}, \quad S_f = \left(\frac{Qn}{AR^{2/3}} \right)^2, \quad (58)$$

away from the control. The depths are chosen and the distances computed, which is exact for a prismatic channel and never iterates. The integration stops as the depth approaches normal depth, because it gets there only asymptotically: a backwater curve has no end, and the extent of backwater is always a convention. `approach` sets the one used (0.99 of the way to normal depth by default; 0.95 gives a noticeably shorter and equally defensible reach), and the notes of the returned `BackwaterResult` say which. The result carries `distance`, `depth`, `water_surface`, `bed_level`, `profile`, `reach`, and `depth_at(distance)`.

14.4 Afflux at bridge piers

Yarnell (1934) (Equation 59):

$$\Delta H = K(K + 5Fr^2 - 0.6)(a + 15a^4)Fr^2 y, \quad (59)$$

with a the fraction of the area blocked by the piers, Fr the downstream Froude number, and K the pier shape coefficient (`PIER_SHAPE`: 0.9 semicircular nose to 2.5 for a ten-pile trestle). The fourth power on the blockage is what makes it bite. Yarnell's tests were in a rectangular flume at blockages up to about 0.4 with the flow class unchanged; where a bridge chokes the flow to critical, use the HEC-RAS energy or momentum methods. `yarnell_afflux(channel,`

`discharge, downstream_depth, blockage, shape="semicircular_nose")` returns the afflux and the upstream depth.

14.5 Flow distribution

`flow_distribution(main, main_depth, slope, floodplains=[(Channel, depth), ...])` splits the discharge by conveyance (Equation 60),

$$\frac{Q_i}{Q} = \frac{K_i}{\sum_j K_j}. \quad (60)$$

This is the number `scour.BridgeOpening` calls `flow_fraction`. A wooded floodplain can be half the width of the section and carry a tenth of the flow; guessing the fraction is the largest assumption in a contraction scour calculation, and it hides behind the most precise-looking number in the output.

```
from pyCoastal.applications.river import (
    Channel, classify_slope, flow_distribution, gvf_profile, yarnell_afflux,
)

channel = Channel(width=30.0, side_slope=2.0, roughness="natural_clean")
plain = Channel(width=120.0, side_slope=3.0, roughness="floodplain_trees")

state = classify_slope(channel, 250.0, 0.0008)      # normal, critical, mild
afflux = yarnell_afflux(channel, 250.0, state["normal"], blockage=0.18)
profile = gvf_profile(channel, 250.0, 0.0008, afflux["upstream_depth"])
profile.profile, profile.reach, profile.depth_at(1500.0)

split = flow_distribution(channel, state["normal"], 0.0008, [(plain, 1.2)])
split["main_fraction"]
```

14.6 Worked example

Output of examples/backwater.py

```
250 m3/s on a 0.0008 bed, natural_clean
  normal depth   3.54 m
  critical depth 1.84 m
  reach is mild, so a bridge backs water up upstream

Piers blocking 18% of the section, Froude 0.35
  afflux 0.071 m, so the control sits at 3.61 m
  M1 profile reaching 5.14 km
  - M1 profile on a mild reach. Control at 3.61 m against normal 3.54 m and critical 1.84 m.
  - Backwater reaches 5.14 km to get 99% of the way to normal depth. The curve is asymptotic, so
  ↪ that figure is a convention and moves a long way with it: quoting an extent without quoting
  ↪ the criterion is meaningless.

In flood, 1.2 m over a wooded floodplain:
  the channel is 26% of the width
  and carries 84% of the flow
  contraction scour 1.08 m with the computed split, 1.81 m if it is assumed to be all of it
```

Wrote `media/backwater.png`

In the example the channel is 26% of the section and carries 84% of the flow, because conveyance goes as the roughness and the hydraulic radius, not the area. Feeding the computed fraction into `contraction_scour` gives 1.08 m, where assuming all the flow goes through the opening gives 1.81 m.

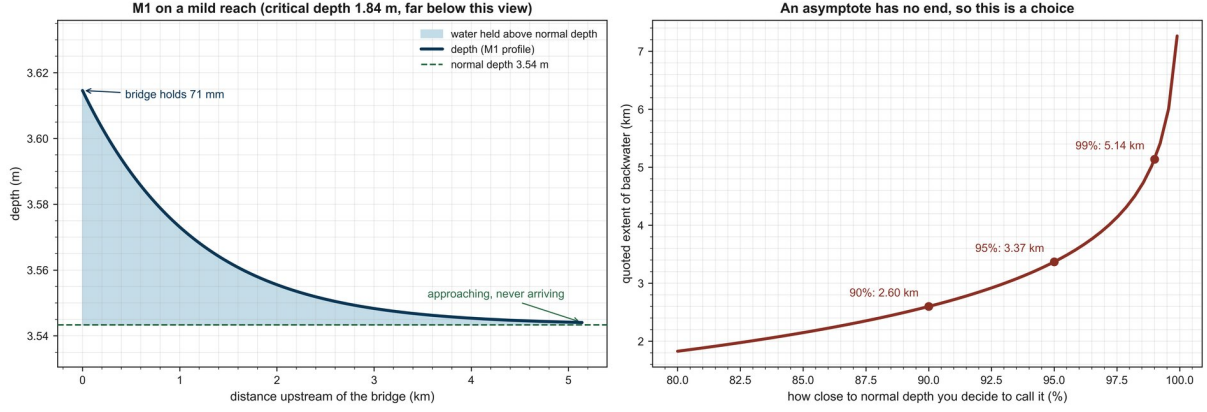


Figure 16: The backwater profile and the extent convention. Depth, not level, is plotted, because the bed rises 4 m over the reach while the depth changes by 7 cm.

15 Navigation channels

Module: `pyCoastal.applications.channel`. *Example:* `examples/navigation_channel.py`.
Browser: Channel.

How deep and how wide does the approach channel have to be? The answer is a stack of allowances, each small, each defensible, and together often several meters. This module builds that stack explicitly so it can be argued over line by line, which is how a dredging budget actually gets agreed. It follows the concept-design method of PIANC (2014), Report 121. The width and lane tables are exposed as editable dictionaries rather than buried, because the governing edition of the guideline, and the pilots on the day, decide the numbers on a real scheme; a real channel is confirmed by simulation.

15.1 The vessel

`Vessel(name, length, beam, draught, block_coefficient=0.75, length_pp=None)` is the design ship. L_{pp} defaults to $0.96L_{oa}$. `displaced_volume` is $C_b L_{pp} B T$ and `displacement` is in tonnes at 1025 kg/m^3 . The same class is used by the berthing module.

15.2 The depth chain

Start at the design water level, subtract the static draught, then in turn the squat as the ship moves, its vertical response to waves, the net clearance over the bed, and the tolerances the dredger and the survey cannot beat. What is left is the dredge level.

Squat. ICORELS (1980), as given by PIANC (Equation 61):

$$S = C_s \frac{\nabla}{L_{pp}^2} \frac{F_{nh}^2}{\sqrt{1 - F_{nh}^2}}, \quad F_{nh} = \frac{V}{\sqrt{gh}}, \quad (61)$$

with $C_s = 2.4$ for an open or wide channel. It blows up as $F_{nh} \rightarrow 1$ and is normally kept to $F_{nh} < 0.7$; the function flags speeds where it is unusable. Barrass (1979) is the back-of-envelope check, $S_{\max} = C_b V_k^2 / 100$ in open water and $/50$ in a confined channel, with the speed in knots (`squat_icorels`, `squat_barrass`).

Wave response. `wave_response_allowance(Hs, factor=0.5, period=None, vessel=None)` takes a fraction of H_s as keel motion: about 0.3 for a long vessel in short head seas up to about 0.7 for a short vessel in long beam seas. With any swell this is the single largest judgement in the depth chain. Given the period and vessel, it also reports the ratio of wavelength to ship length, which governs whether the ship contours or bridges the wave.

Clearance stack. `underkeel_clearance(vessel, squat, wave_allowance, net_clearance=0.6, water_level_allowance=0.0, dredging_tolerance=0.3, survey_tolerance=0.2, siltation_allowance=0.2, density_allowance=0.0)` returns each allowance, the gross total below the keel, and the required depth. Leaving out the dredging and survey tolerances is the most common way a channel ends up shallower than its drawing.

Because the squat depends on the depth and the depth depends on the squat, `design_channel` solves the depth by a short fixed-point iteration.

15.3 The width chain

`channel_width(vessel, manoeuvrability="moderate", section="outer", two_way=False, speed_class="moderate", bank="sloping_channel_edges", conditions=None)` builds the PIANC width: a basic maneuvering lane (MANOEUVRING_LANE: 1.3, 1.5, 1.8 beams for good, moderate, poor maneuverability), plus additional widths from WIDTH_COMPONENTS for vessel speed, crosswind, crosscurrent, longitudinal current, waves, aids to navigation, bottom surface, depth of waterway, and cargo hazard (with separate outer and inner-channel columns), plus a bank clearance each side (BANK_CLEARANCE), plus a passing distance for two-way traffic (PASSING_DISTANCE). **Components you do not specify are taken at their most benign class and reported as assumed**, because a silent default is how a channel ends up too narrow on paper.

15.4 Side slopes, volumes, and turning basins

`dredged_side_slope(bed, factor=2.0)` gives $\cot \beta = f / \tan \phi'$; the factor of 2 covers both the partial factor on a submerged granular slope and the overcut and slumping of dredging, landing near 1:2.6 in gravel and 1:3.8 in silt. `turning_basin_diameter(vessel, assisted=True, current=False)` gives $1.5 L_{oa}$ with tugs, 2.0 unassisted, plus half a length where a current runs through.

`design_channel(vessel, speed, design_water_level, Hs=0, Tp=None, wave_factor=0.5, net_clearance=0.6, ..., bed="medium_sand", existing_bed=None, **width_kwargs)` returns a `ChannelDesign` with the dredge_level, required_depth, the clearance stack, squat and wave details, width, top_width, side_slope, dredge_volume(length), warnings, and summary().

```
from pyCoastal.applications.channel import Vessel, design_channel

vessel = Vessel(name="Post-Panamax container ship", length=336, beam=48.2,
                draught=14.5, block_coefficient=0.68)
channel = design_channel(vessel, speed=8.0, design_water_level=1.20,
                        Hs=1.8, Tp=9.0, existing_bed=-11.5, two_way=True,
                        conditions={"crosswind": "moderate", "waves": "moderate"})
channel.dredge_level, channel.width, channel.dredge_volume(1000)
```

15.5 Worked example

Output of `examples/navigation_channel.py`

Design vessel Post-Panamax container ship
 336 x 48.2 x 14.5 m, Cb = 0.68, 157,130 t
 Speed 8.0 knots (Fn_h = 0.31)

Depth chain (m below the design water level)

static draught	14.50
squat	0.37
wave response	0.90
density	0.00
net clearance	0.80
water level uncertainty	0.30
dredging tolerance	0.30
survey tolerance	0.20
siltation	0.20
required depth	17.57

Design water level +1.20 m CD
 Dredge level -16.37 m CD

Channel width 415 m (8.6 beams, 2-way)

basic manoeuvring lane	72.3
crosswind (moderate)	14.5
crosscurrent (low)	9.6
waves (moderate)	24.1
aids_to_navigation (good)	9.6
bottom_surface (smooth_and_soft)	4.8
depth_of_waterway (shallow)	9.6
bank clearance, each side	24.1
passing distance	77.1

Existing bed -11.50 m CD
 Dredge depth 4.87 m
 Top width 463 m at 1:5 side slopes
 Volume 2,135 thousand m³ per km

Turning basin 672 m (2.0 x Lo_a, assisted, current running)

Squat, two ways

6.0 kn	ICORELS 0.20 m	Barrass 0.24 m	Fn _h 0.24
8.0 kn	ICORELS 0.37 m	Barrass 0.44 m	Fn _h 0.31
10.0 kn	ICORELS 0.59 m	Barrass 0.68 m	Fn _h 0.39
12.0 kn	ICORELS 0.89 m	Barrass 0.98 m	Fn _h 0.47

What moves the dredge level

wave factor 0.3 rather than 0.5	-0.35 m	-162 k m ³ /km
wave factor 0.7 rather than 0.5	+0.35 m	+164 k m ³ /km
speed 12 kn rather than 8 kn	+0.49 m	+229 k m ³ /km
net clearance 1.5 m rather than 0.8 m	+0.69 m	+320 k m ³ /km
no siltation allowance	-0.20 m	-90 k m ³ /km

Wrote media/navigation_channel.png, media/navigation_channel_sheet.png and

↪ media/navigation_channel.dxf

Wrote media/channel_depth_chain.png

The sensitivity table at the end is the lesson: one judgement call, the fraction of the wave height taken as vertical vessel motion, moves the dredge level by 0.35 m either way, more than the dredging and survey tolerances together.

16 Berthing energy and fenders

Module: pyCoastal.applications.berthing. *Example:* examples/berth_fenders.py.

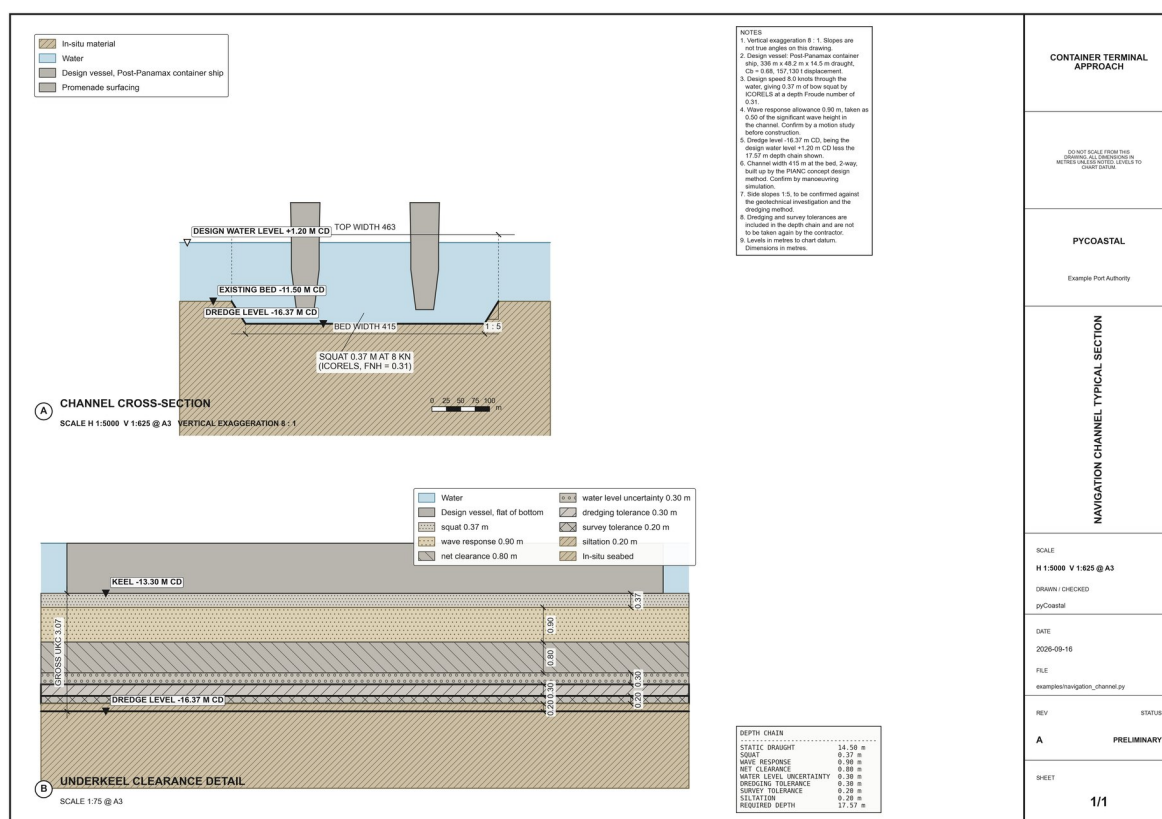


Figure 17: Navigation channel drawing sheet: the whole channel with a stated vertical exaggeration, and the underkeel clearance at a true scale where the allowances read as real thicknesses.

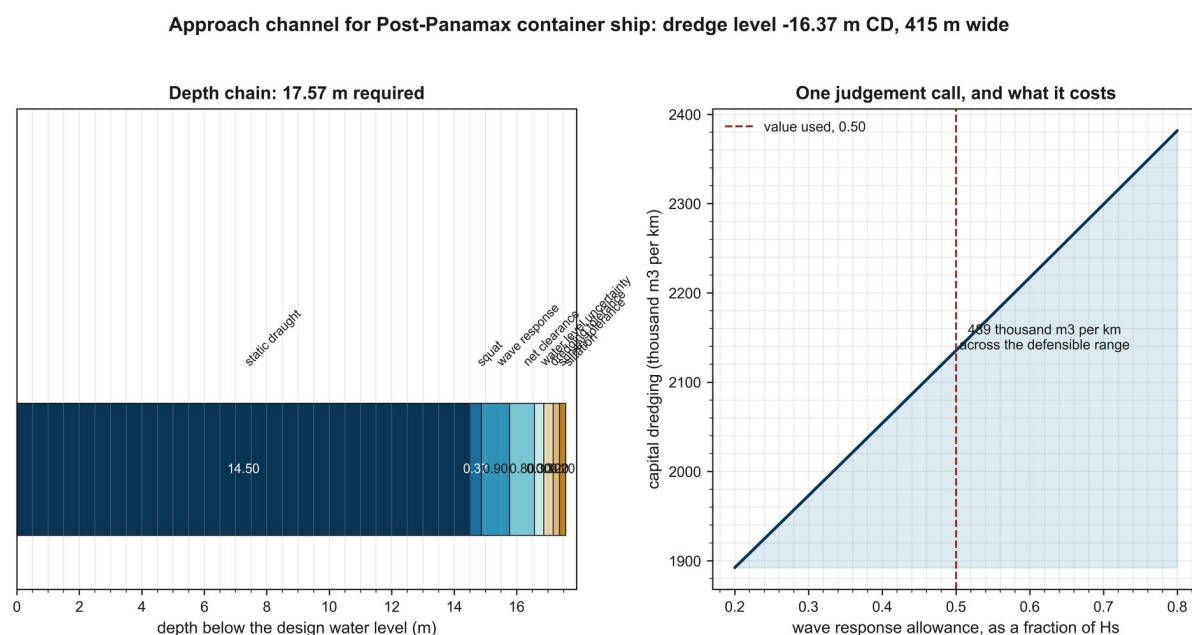


Figure 18: The depth chain and its sensitivity to each judgement.

A ship comes alongside carrying kinetic energy, and something has to absorb it. A berth fails in three different ways and only one of them is about energy. The fender can be too small, and the ship reaches the quay. The panel can be too small, and the reaction dents the side shell even though the energy was absorbed perfectly. The fenders can be too far apart, and the hull touches the structure between them. All three are checked.

16.1 The energy chain

PIANC (2002, Working Group 33) writes the normal berthing energy as the kinetic energy times four factors (Equation 62):

$$E_N = \frac{1}{2}MV^2 C_m C_e C_s C_c. \quad (62)$$

- **Added mass** (Vasco Costa 1964), $C_m = 1 + 2T/B$, the water the ship drags sideways. It is the only factor that makes the energy larger. Below about 1.1 draughts of water depth the form starts to understate it, and `added_mass_factor(vessel, depth)` warns.
- **Eccentricity**, $C_e = (K^2 + R^2 \cos^2 \gamma) / (K^2 + R^2)$ with the radius of gyration $K = (0.19C_b + 0.11)L_{pp}$, R the distance from the contact point to the center of mass along the berth (default $L_{pp}/4$, the quarter point), and γ the angle between the velocity and that line (default 90 degrees). The rotation carries away energy the fender never sees.
- **Softness** C_s : 1.0 for a soft fender, 0.9 where it is stiff enough that the hull deflects with it.
- **Configuration** C_c (BERTH_CONFIGURATION): 0.8 solid quay, 0.9 semi-solid, 1.0 open piled.

The **abnormal factor**, applied afterwards (ABNORMAL_FACTOR: 1.25 for a large tanker up to 2.0 for ro-ro, ferries and tugs, 1.5 for container ships), covers the berthing that goes wrong: a parted tug line, an engine that does not go astern. It is reported separately because it is frequently the largest single term.

Velocity is squared and is the least well known quantity. BERTHING_VELOCITY holds the Brolsma et al. (1977) style table (Table 10); `berthing_velocity(vessel, condition)` picks the size class from the displacement.

Table 10: Indicative design berthing velocities (m/s), BERTHING_VELOCITY.

Condition	small	medium	large
easy_sheltered	0.20	0.15	0.10
difficult_sheltered	0.30	0.20	0.15
easy_exposed	0.40	0.30	0.20
good_exposed	0.50	0.40	0.25
difficult_exposed	0.80	0.60	0.40

16.2 Fender, panel, and spacing

Fender family. Within one family and rubber grade, performance scales with size: energy with the volume of rubber, reaction with the section (Equation 63),

$$E = E_0 \left(\frac{H}{H_0} \right)^3, \quad R = R_0 \left(\frac{H}{H_0} \right)^2. \quad (63)$$

A fender chosen to absorb twice the energy pushes back only about 1.6 times as hard. `FenderFamily` holds a reference size (the default `CONE_FENDER` is a mid-grade cone fender, 1.0 m, 500 kNm, 1000 kN, in standard heights from 0.3 to 2.5 m) and `select_fender(energy, family)` returns the smallest standard size that absorbs the energy and how much of its capacity is used. This is a scaling model, not a catalogue: size the fender you are asking for here, then select against the manufacturer's tested curves.

Hull pressure. `hull_pressure(reaction, panel_width, panel_height, vessel_class)` checks the panel against `HULL_PRESSURE_LIMIT` (150 kN/m² for large tankers, 200 for small tankers and bulk carriers, 400 for container ships and general cargo, 700 for tugs).

Spacing. `fender_spacing(vessel, spacing, projection, bow_radius=None, smallest_vessel=None, clearance=0.15)` applies two checks. Geometric: with the fenders compressed, the standoff must exceed the sagitta of the hull across the gap (Equation 64),

$$s \leq 2\sqrt{R_b^2 - (R_b - p + c)^2}. \quad (64)$$

Practical: the smallest ship using the berth must reach two fenders at once, about $0.15L_{pp}$. Without a bow radius the contact is on the straight parallel midbody, so the geometric check cannot bind and the length rule governs; the rule uses the *smallest* vessel, not the design one.

`design_berth(vessel, velocity=None, condition="easy_sheltered", vessel_class="container", configuration="open_piled", ..., smallest_vessel=None, panel_aspect=(1.2, 2.0), family=CONE_FENDER)` runs the whole chain and returns a `BerthDesign` with normal, abnormal, energy, fender, panel, pressure, spacing, adequate, and notes.

```
from pyCoastal.applications.berthing import design_berth
from pyCoastal.applications.channel import Vessel

ship = Vessel(name="Post-Panamax container", length=366, beam=48.2,
              draught=15.2, block_coefficient=0.68)
feeder = Vessel(name="Feeder", length=140, beam=22.0, draught=8.5,
               block_coefficient=0.68)
design = design_berth(ship, velocity=0.15, vessel_class="container",
                    configuration="open_piled", depth=17.0,
                    smallest_vessel=feeder)
design.energy, design.fender["height"], design.pressure["pressure"], design.adequate
```

16.3 Worked example

Output of examples/berth_fenders.py

Post-Panamax container: 179 thousand tonnes
Approaching at 0.15 m/s

kinetic energy	2018 kNm
x Cm 1.631	added water
x Ce 0.478	rotation about the contact
x Cs 1.000	fender softness
x Cc 1.000	berth configuration
normal berthing	1573 kNm
x 1.50	abnormal allowance
design energy	2360 kNm
fender	2.00 m, 4000 kNm rated (59% used)
reaction	4000 kN
hull pressure	325 kN/m ² against 400 allowed
spacing	16.1 m against 20.2 m allowed

ADEQUATE

True

- Post-Panamax container: 366 m by 48.2 m, 15.2 m draught, 179 thousand tonnes displacement.
- Berthing at 0.15 m/s, as specified. Velocity is squared in the energy, so it carries more of the answer than anything else here.
- Factors: Cm 1.63 adds the water the ship drags, Ce 0.48 takes away what the ship rotates out, Cs 1.00 and Cc 1.00 take away a little more. Net, the fender sees 78% of the 2018 kNm the ship arrived with.
- Eccentricity is taken at the quarter point, the usual design assumption. A ship that lands flat on its midships rotates about nothing and hands the fender all of it: Ce becomes 1 and the energy rises by 2.09 times.
- Abnormal factor 1.50 for a container, taking the design energy from 1573 to 2360 kNm. This is the allowance for the berthing that goes wrong, and it is the largest single term in this calculation.
- Cone fender, mid grade, 2.00 m: 4000 kNm rated against 2360 kNm required, so it works at 59% of capacity and pushes back 4000 kN.
- Hull pressure 325 kN/m² over a 2.8 by 4.4 m panel, within the 400 kN/m² allowed for this class.
- Fenders at 16.1 m centres, inside the 20.2 m limit set by vessel length for Feeder. Contact is on the parallel midbody, which is straight, so only the vessel length rule binds; check the end fenders separately with a bow radius.
- Fender performance here is a scaling model within one family, not a catalogue. Size the fender you are asking for from this, then select against the manufacturer's tested curves at the design temperature, angle and velocity, which is what they guarantee.

Wrote media/berth_energy.png

Three of the four PIANC factors reduce the energy and the abnormal allowance puts most of it back: the design energy of 2360 kNm is close to the 2018 kNm the ship arrived with, which makes it easy to conclude wrongly that the factors did not matter.

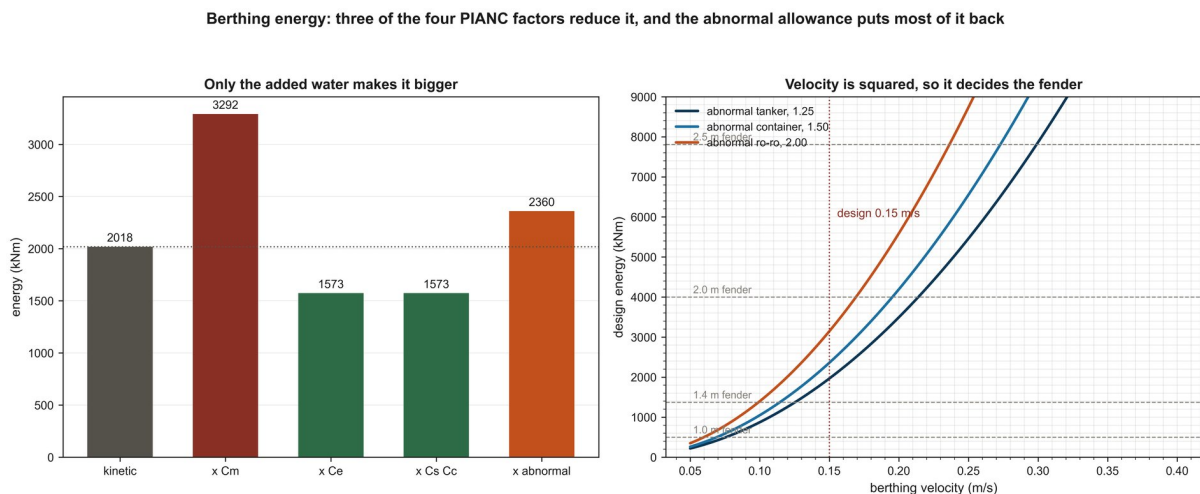


Figure 19: The berthing energy chain and the design levers.

17 Beach nourishment

Module: pyCoastal.applications.nourishment. *Examples:* examples/nourishment_design.py, examples/nourishment_profile.py. *Browser:* Nourishment.

A beach fill is designed in two directions at once. Alongshore, it is a perturbation to a straight shoreline that spreads and leaves the project area; the question is how long it lasts. Cross-shore, the beach takes the profile its own grain size can hold; the question is how much dry beach a given volume buys, and how much that depends on where the sand comes from.

17.1 The planform: how long a fill lasts

Under small wave angles the one-line model (Equation 31) reduces to a diffusion equation. With the CERC transport in the solid-volume SI form this model needs (Equation 65),

$$Q = K_{\text{cerc}} H_b^{5/2} \sin 2\alpha, \quad K_{\text{cerc}} = \frac{K \sqrt{g/\gamma_b}}{16(s-1)}, \quad (65)$$

(`cerc_coefficient(K=0.39, s=2.65, gamma_b=0.78)`; the solver divides by $1-p$ itself, so a bulk-volume coefficient would double-count porosity), linearization gives Equation 66,

$$\frac{\partial y}{\partial t} = \varepsilon \frac{\partial^2 y}{\partial x^2}, \quad \varepsilon = \frac{2K_{\text{cerc}} H_b^{5/2}}{(1-p)(D+B)}, \quad (66)$$

(`longshore_diffusivity`). For an initially rectangular fill of width W and length $L = 2a$, Pelnard-Considere (1956) gives Equation 67,

$$y(x, t) = \frac{W}{2} \left[\operatorname{erf} \left(\frac{a-x'}{2\sqrt{\varepsilon t}} \right) + \operatorname{erf} \left(\frac{a+x'}{2\sqrt{\varepsilon t}} \right) \right], \quad (67)$$

with x' from the fill center (`pel_nard_considere`). The center width $W \operatorname{erf}(a/2\sqrt{\varepsilon t})$ halves when the argument reaches $\operatorname{erf}^{-1}(1/2) = 0.4769$, which defines the spreading half-life, the natural clock of a fill (`sections.spreading_half_life`).

`NourishmentDesign(length=1000, berm_width=30, taper=100, center=None, D=8, B=2, porosity=0.4)` is the fill: full-width length, shoreline advance at placement, linear tapers at each end, closure depth D , and berm height B . `WaveClimate(Hb=1.0, T=8.0, alpha0=0.0, Kcerc=KCERC_DEFAULT)` drives it; a zero angle means the fill diffuses symmetrically without migrating. `simulate_nourishment(design, climate, duration, domain_length=None, dx=10.0, n_outputs=25, bc="fixed_ends", cfl=0.9)` evolves the tapered planform with `tools.shoreline` on a domain five times the fill footprint and returns a `NourishmentResult` with `planforms`, `volume_retained`, `retained_fraction`, `berm_width`, and `design_life(threshold=0.5)`. `renourishment_schedule(design, climate, horizon, threshold=0.5)` returns the interval, the number of renourishments, the placement times, and the total volume over a planning horizon. The analytical solution verifies the solver in the test suite.

Output of `examples/nourishment_design.py`

```
Placed volume      : 960,000 m3
Active profile height: 10.0 m
Diffusivity        : 0.0175 m2/s  (551,100 m2/yr)
```

year	retained %	berm width (m)
0.0	100.0	30.0
4.0	54.0	16.6
8.0	41.5	12.3
12.0	34.9	10.2
16.0	30.7	8.9
20.0	27.8	8.0

```
Design life (50% retained): 5.0 yr
Renourishment interval    : 4.9 yr
Renourishments in 30 yr   : 6
Total sand over 30 yr      : 6,720,000 m3
```

Wrote `media/nourishment_design.png`

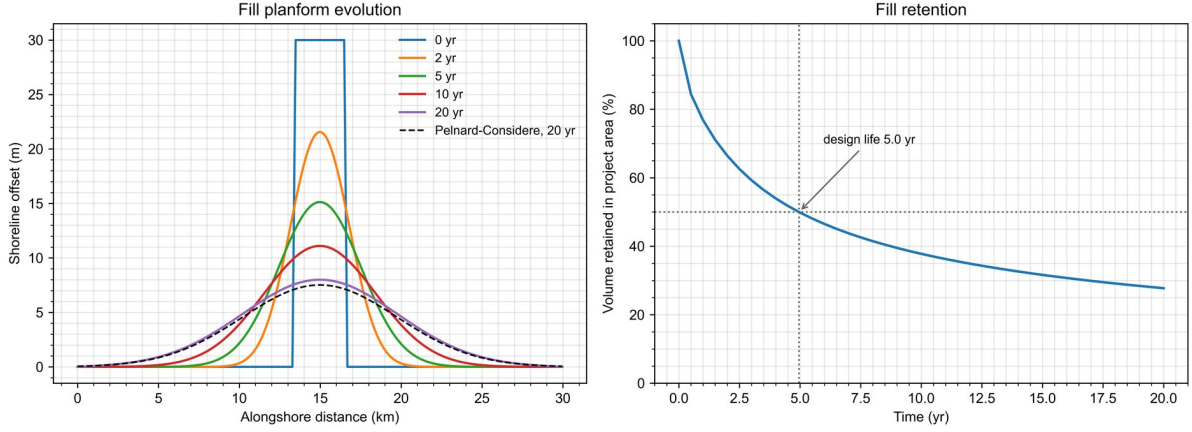


Figure 20: Nourishment design study: planform evolution against the Pelnard-Considere solution, retained volume, and the renourishment cycle.

17.2 The profile: what a borrow source is worth

Dean's equilibrium profile is $h = Ay^{2/3}$ (`equilibrium_profile`), with the scale parameter tied to the fall velocity (Kriebel, Kraus and Larson 1991) (Equation 68),

$$A = 0.067 w^{0.44} \quad (w \text{ in cm/s}), \quad (68)$$

so a sediment cannot have one settling velocity for scour and another for its profile (`dean_scale`; fine sand at 0.19 mm gives 0.094 and medium sand at 0.38 mm gives 0.141, against Dean's tabulated 0.10 and 0.14).

A fill with scale A_f placed on a native beach with scale A_n pushes the profile seaward at each depth by $\Delta(h) = a + (h/A_f)^{3/2} - (h/A_n)^{3/2}$, where a is the shoreline advance. Integrating over *depth* (not distance, because the active profile is bounded by the closure contour) from the waterline to closure gives Equation 69,

$$V = Ba + ah_L + 0.4 h_L^{5/2} (A_f^{-3/2} - A_n^{-3/2}), \quad (69)$$

where h_L is the closure depth, or the depth where the profiles meet if the fill is coarse enough to intersect first (`fill_volume_for_advance`). For matched sand it is exactly $V = (B + h_*)a$.

Three cases follow: coarser fill **intersects** the native profile and buys more beach per cubic meter; finer fill is **non-intersecting**; and below a **critical volume** (`critical_volume`) a fine fill buys no dry beach at all, because the whole placement goes into flattening the underwater profile. `shoreline_advance(A_native, A_fill, volume, berm_height, closure_depth)` solves for the advance by bisection (one solver that cannot pick the wrong branch) and names the case. `profile_overfill_factor(native, borrow, ...)` is the ratio of borrow to native volume for the same advance, which is not James's (1975) textural overfill ratio. `grain_compatibility(native, borrow)` compares the phi means and sortings ($\delta = (\phi_b - \phi_n)/\sigma_n$) and gives a verdict; `phi_size` and `size_from_phi` convert grain sizes.

Output of `examples/nourishment_profile.py`

Native beach: Medium sand, d50 = 0.38 mm, A = 0.141
 Berm 2.0 m, closure 6.0 m, fill 250 m3 per metre

borrow	A	delta	profile	dry beach	critical
Very fine sand	0.051	+3.78	submerged	0.0 m	2362 m3/m
Fine sand	0.094	+1.82	submerged	0.0 m	566 m3/m
Medium sand	0.141	+0.00	matched	31.2 m	0 m3/m
Coarse sand	0.183	-1.78	intersecting	54.5 m	0 m3/m
Fine gravel	0.307	-7.24	intersecting	66.2 m	0 m3/m

What that costs, per metre of beach, for 40 m of dry sand

Very fine sand	2682 m3/m	(8.38 times the native requirement)
Fine sand	886 m3/m	(2.77 times the native requirement)
Medium sand	320 m3/m	(1.00 times the native requirement)
Coarse sand	164 m3/m	(0.51 times the native requirement)
Fine gravel	131 m3/m	(0.41 times the native requirement)

Compatibility

Very fine sand	much finer than native, poorly suited
Fine sand	much finer than native, poorly suited
Medium sand	close to native, suitable
Coarse sand	coarser than native, well suited
Fine gravel	coarser than native, well suited

Planform, 1500 m of fill in Hb = 1.2 m:

spreading half-life	0.57 yr
placed volume	719 thousand m3 in place

Wrote media/nourishment_profile.png, media/nourishment_plan.png and media/nourishment_sheet.png

Wrote media/nourishment_borrow.png

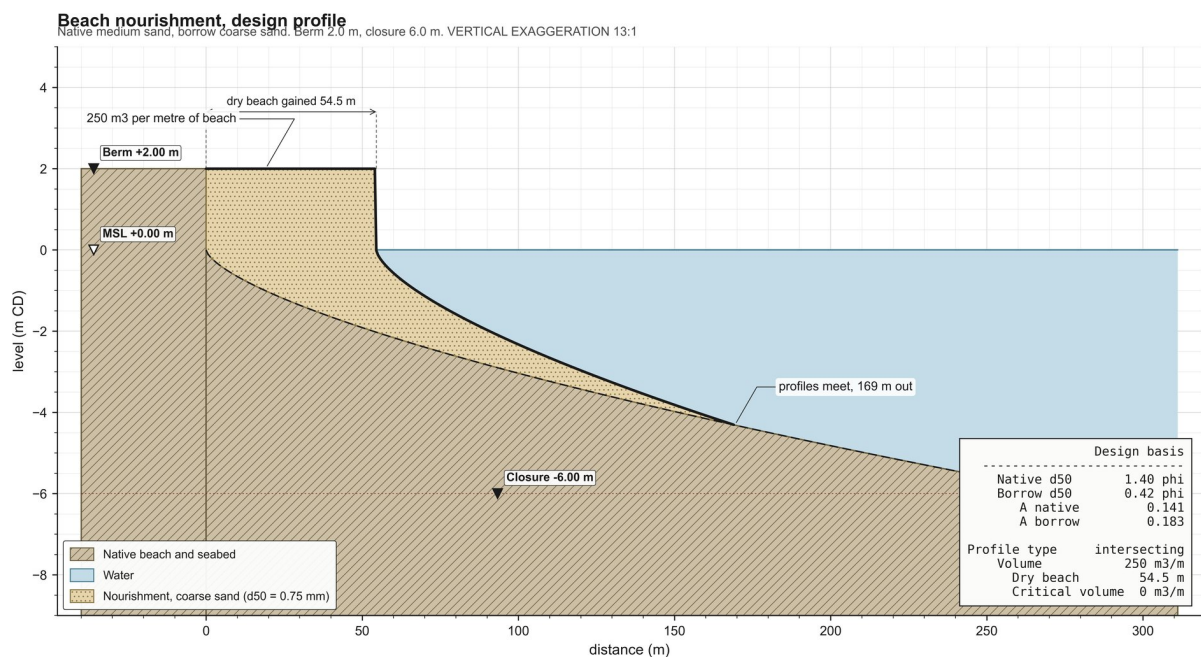


Figure 21: Profiles for borrow sources from very fine sand to fine gravel on a medium sand beach.

17.3 Limits

The analytical planform is for a rectangular fill with constant diffusivity; the numerical solver handles tapers but still uses one wave climate. The profile method assumes both profiles reach equilibrium and ignores winnowing of fines, which the compatibility check addresses separately.

Nourishment on medium sand: the borrow grain size decides what the sand is worth

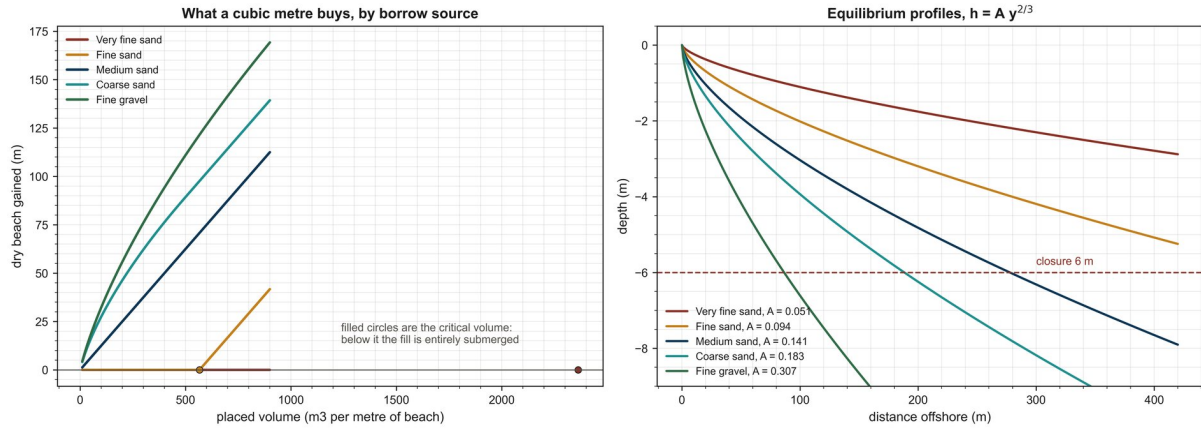


Figure 22: The cost of borrow sources: volume per meter of beach for the same dry beach width.

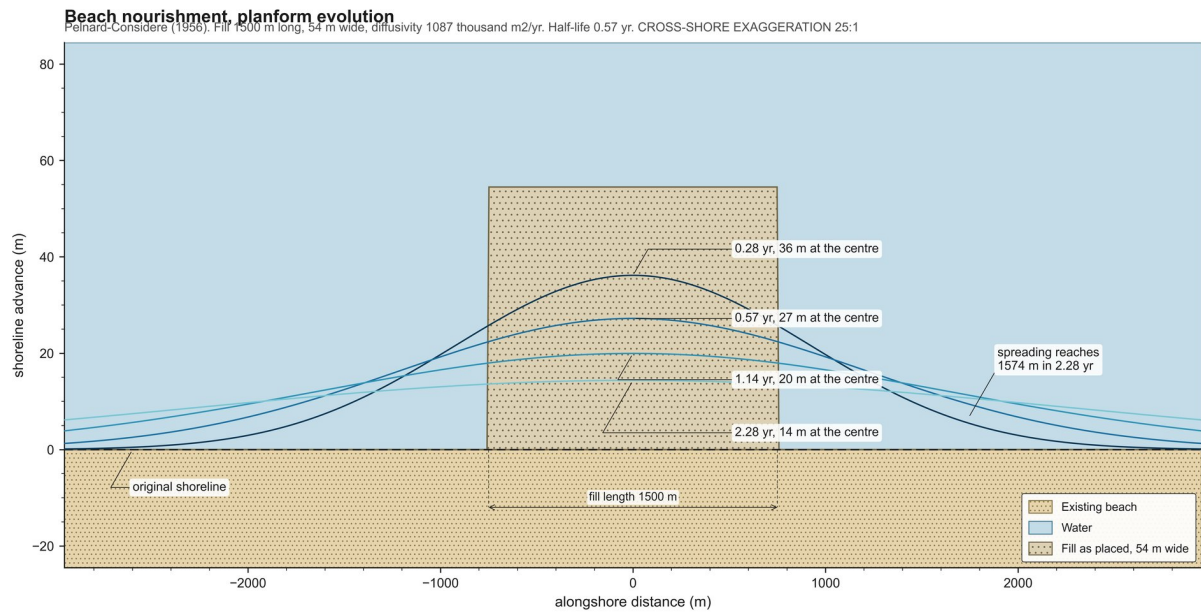


Figure 23: Planform spreading alongshore from the analytical solution, at times taken from the fill's own half-life. The cross-shore axis is stretched to fill the sheet and the factor is stated on the drawing.

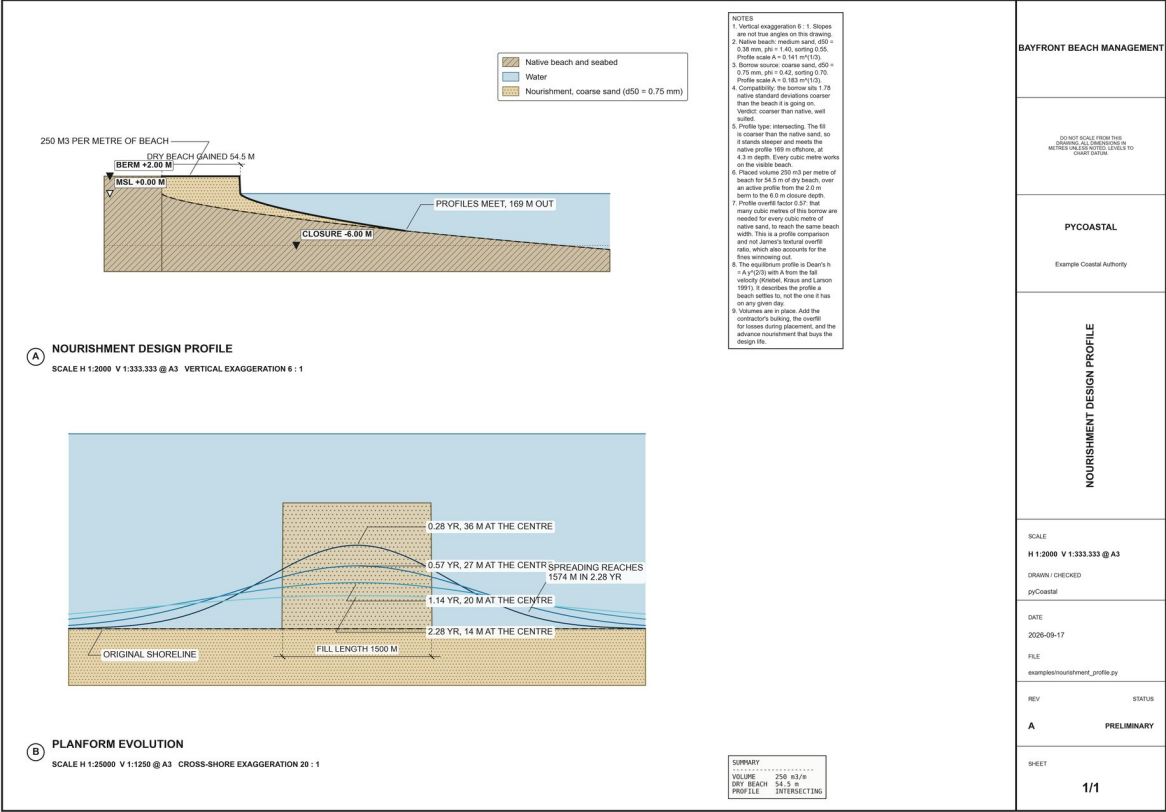


Figure 24: Nourishment drawing sheet with the design profile and the planform on one sheet, each at its own standard scale.

18 Storm surge and coastal flooding

Module: `pyCoastal.applications.surge`. *Example:* `examples/storm_surge_flooding.py`.

The module builds a total water level from its components and then finds how far a storm floods: along a cross-shore profile, or over a terrain grid with hydraulic connectivity, so that low ground behind a continuous barrier is not flooded by a level that cannot physically reach it.

18.1 The water level budget

The still water level is the sum of the astronomical tide (supplied, not predicted), any sea-level rise, and three setups:

- **Barometric setup**, the inverted-barometer response, $\eta_p = (p_a - p_c)/(\rho g)$, close to 1 cm per hPa (`barometric_setup`). It is the equilibrium response and an upper bound for a fast-moving storm.
- **Wind setup**, from integrating $d\eta/dx = nkW^2 \cos \theta / (gd)$ over the fetch, with $k = 3.3 \times 10^{-6}$ and a return-flow factor n of 1.15 to 1.30. Over a constant depth (Equation 70),

$$\eta_w = \frac{nkW^2 F \cos \theta}{gd}, \quad (70)$$

(`wind_setup`). Because setup goes as $1/d$ it is dominated by the shallowest part of the shelf, so `wind_setup_profile(depths, dx, ...)` integrates across a real shelf, carrying the accumulated setup into the local depth $d + \eta$, which damps further setup.

- **Wave setup** in the surf zone, from the breaking height (`surf_zone_setup(breaking_height, gamma=0.8)`).

Runup is added on top for the total water level, from Stockdon et al. (2006) (Equation 71):

$$R_{2\%} = 1.1 \left(0.35\beta \sqrt{H_0 L_0} + \frac{\sqrt{H_0 L_0 (0.563\beta^2 + 0.004)}}{2} \right), \quad (71)$$

returned as setup and swash separately (`stockdon_runup`). Runup is a swash oscillation, not a sustained level, so flood extent is taken from the still water level and the wave hazard at the shoreline from the total.

`StormConditions(wind_speed=40, central_pressure=95000, tide=0, Hm0=6, Tp=12, fetch=100e3, shelf_depth=20, wind_angle=0, sea_level_rise=0)` is the storm; `total_water_level(storm, beach_slope=0.05, breaker_index=0.78, shelf_depths=None, dx=None, return_flow=1.15)` returns every component, the still water level, and the total water level.

18.2 Flood mapping

A plain elevation threshold floods every low cell, including basins sealed off from the sea by higher ground. `bathtub_flood(terrain, level, seed=None, connectivity=4)` walks outward from the coast instead, so only ground the water can reach is flooded; `seed` marks where the sea enters (default: any below-level cell on the domain edge) and `connectivity` is 4 or 8. `isolated_low_ground` returns the cells below the level that the sea cannot reach, `flooded_area(flooded, dx)` and `flood_volume(terrain, flooded, level, dx)` quantify the map, and `flood_depths(z,`

level) gives depths regardless of connectivity. On a profile, `inundation_limit(x, z, level)` walks landward and stops at the first ground above the level.

```
import numpy as np
from pyCoastal.applications.surge import (
    StormConditions, total_water_level, bathtub_flood, isolated_low_ground,
)

storm = StormConditions(wind_speed=45, central_pressure=94000, tide=0.6,
                        Hm0=7.0, Tp=13.0, fetch=120e3)
shelf = np.linspace(60.0, 8.0, 200)
levels = total_water_level(storm, shelf_depths=shelf, dx=120e3 / 199)

sea = np.zeros_like(terrain, dtype=bool); sea[0, :] = True
flooded = bathtub_flood(terrain, levels["still_water_level"], seed=sea)
stranded = isolated_low_ground(terrain, levels["still_water_level"], seed=sea)
```

18.3 Worked example

Output of `examples/storm_surge_flooding.py`

```
Water level budget (m above datum)
      tide      0.60
  sea level rise  0.00
barometric setup  0.73
    wind setup   3.28
    wave setup   1.75
still water level  6.36
      runup R2%   2.32
total water level  8.68

Still water level 6.36 m
threshold-only flooded area : 19.17 km2
connected flooded area      : 18.82 km2
overstated by the threshold : 0.35 km2 (2% of it)
water volume on land        : 113.81 Mm3

with the inlet open   : 18.82 km2 flooded
with the inlet sealed : 5.45 km2 flooded
a single breach in the barrier is what floods the bay

inundation limit, intact barrier: 1418 m
inundation limit, at the inlet  : 4990 m
```

Wrote `media/storm_surge_flooding.png`

Connectivity matters: on the synthetic barrier island, a single breach in the barrier is the difference between 5.45 km² and 18.82 km² flooded.

Storm surge flooding: 45 m/s, 940 hPa, Hm0 = 7.0 m, tide 0.6 m

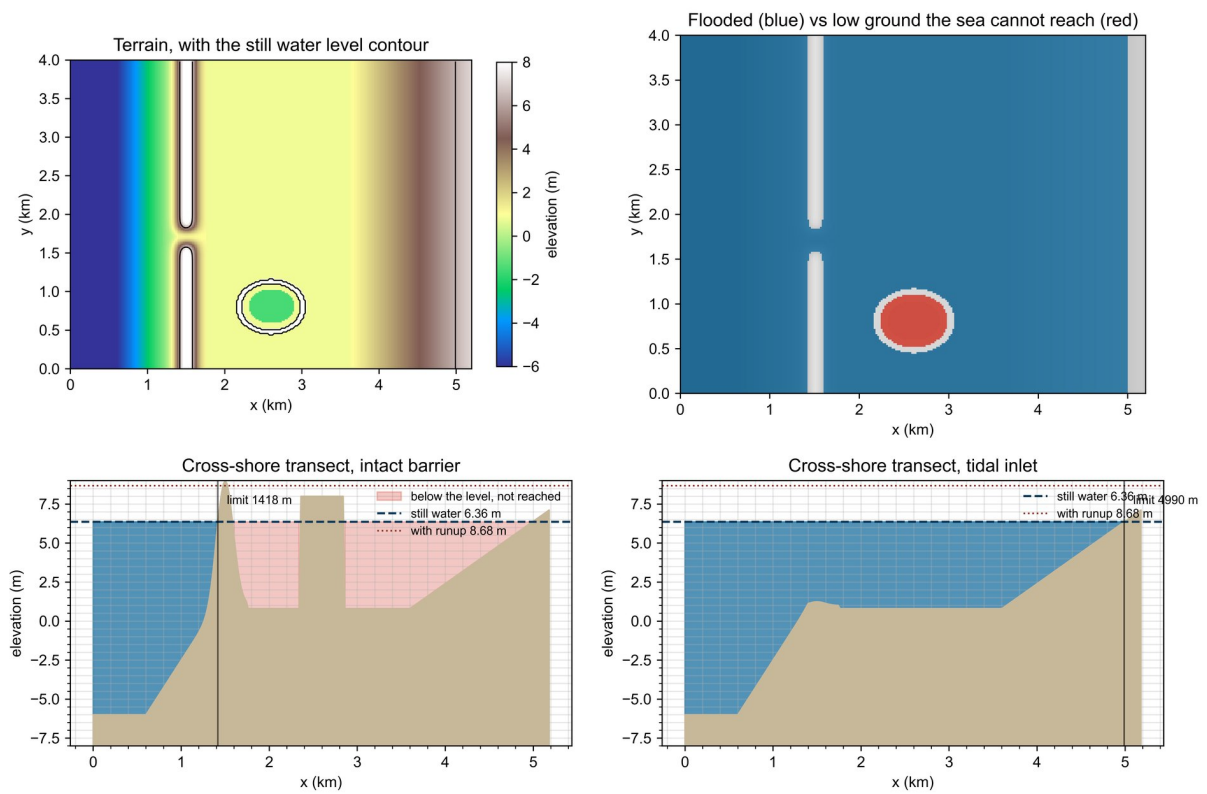


Figure 25: Storm surge flood mapping on a barrier-island terrain: threshold-only against connected flooding, with the inlet open and sealed.

Part V

Drawings and deliverables

19 Drawings, sections, and sheets

Modules: `pyCoastal.drafting`, `pyCoastal.applications.sections`, `pyCoastal.plotting`.
Needs the `plots` extra.

A coastal cross-section is not a plot. It is a scaled drawing: the geometry is in real meters, line weights carry meaning, materials are hatched rather than colored by value, and every dimension that governs the design is called out on the paper. `pyCoastal.drafting` gives the small set of primitives that takes matplotlib from plotting to drafting, and `applications.sections` turns each design object into a drawing and a take-off.

19.1 Drafting primitives

`Section(title="", subtitle="", figsize=(13, 7), ax=None, caps=False, dim_style="arrow", frame=True, exaggeration=1.0)` is a drawing in real coordinates (x cross-shore, z elevation), locked to equal aspect so a 1:2 slope looks like a 1:2 slope. Text is sized in points and placed with offsets in points, so annotation stays readable when the section is rescaled; only the geometry scales. Its methods are the vocabulary of a drawing office:

- `material(points, kind, label=None)` fills a closed polygon with a material from the library (Table 11);
- `line(points, weight="medium", style="-")` draws construction, ground, or hidden lines with the line weights in `WEIGHTS` (thin 0.5, medium 1.0, heavy 1.8, extra 2.4);
- `water(x0, x1, level, bed=None, label=None)` fills water between a level and the bed, with the level symbol;
- `dim_h(x0, x1, z, text=None, extend_from=None)` and `dim_v(z0, z1, x, text=None, side="right")` draw dimension lines with extension lines and terminators;
- `level(x, z, text=None, side="left", symbol="level")` draws the levelling triangle and elevation callout;
- `slope(apex, cot_alpha, rise, direction="left")` draws the slope triangle, labelled $1 : \cot \alpha$;
- `note(xy, text, offset=(40, 30))` places a leader line and note;
- `detail_bubble(label, title, scale="")` places the circled detail marker;
- `notes_block(lines, title="NOTES", numbered=True)` places numbered specification notes;
- `table(rows, title="Design parameters")` places the parameter block (a drawing that does not carry its design inputs is not a deliverable);
- `scale_bar(length, divisions=4)` and `key(loc="upper left")` add a checkered scale bar and a material key;
- `fit_scale(xlim, zlim, scales=None, paper="")` sets the view to a true, round drawing scale that fits the extents, and `auto_exaggeration(xlim, zlim, cap=200)` stretches the second axis just enough to fill the viewport when a true scale is unreadable;
- `finish(xlim=None, zlim=None, grid=True)`, `save(path, dpi=600)`, and `to_dxf(path, scale=1.0)`.

Table 11: The material library in `pyCoastal.drafting`.

Key	Drawn as
<code>armour, secondary, underlayer, toe</code>	individual stones at the layer's stone size
<code>core</code>	quarry run, dotted hatch
<code>concrete, reinforced</code>	mass and reinforced concrete, diagonal and cross hatch, heavy edge
<code>blinding</code>	blinding layer
<code>rock_fill, granular, sand</code>	fills, circle and dot hatches
<code>subgrade</code>	in-situ seabed, dense diagonal hatch
<code>pavement</code>	promenade surfacing
<code>water</code>	water, thin edge

`Material(name, face, edge, hatch=None, weight="medium", stones=0.0)` defines a new one, and `use_crisp_style()` sets global matplotlib settings for legible technical figures.

`Sheet(titleblock, size="A3", dpi=400, margin=8.0, strip=52.0)` is a drawing sheet with a border and a title block strip down the right edge, in any of PAPER (A0 to A4). `viewport(rect)` adds a Section view within the usable area, `set_scale_from(view)` copies a view's fitted scale into the title block, `save(path)` writes the sheet with no trimming (the border is the edge), and `to_dxf(path)` exports every view to one DXF. `TitleBlock` holds `project`, `title`, `organisation`, `client`, `scale`, `date`, `file`, `sheet`, `revision`, `drawn_by`, `checked_by`, `status`, and a standard disclaimer.

`DXF.write_dxf(path, entities, scale=1.0)` writes polylines on named layers to a minimal DXF R12 file that any CAD package opens.

19.2 Sections for each design

Every structure in `applications.sections` has three entry points, and all three take a design object rather than loose numbers:

- `draw_*` puts the geometry and annotation into an open `Section` and returns the extents it needs;
- `*_section` is a standalone figure with plot axes, for a report or slide;
- `*_sheet` is a full drawing sheet with border, title block, and notes, at a true stated scale; extra keyword arguments go to the `TitleBlock`.

Table 12: Drawing functions in `pyCoastal.applications.sections`.

Structure	draw	section	sheet	notes
Seawall	<code>draw_seawall, draw_seawall_toe_detail</code>	<code>seawall_section</code>	<code>seawall_sheet</code>	<code>seawall_notes</code>
Rubble mound	<code>draw_rubble_mound</code>	<code>rubble_mound_section</code>	<code>rubble_mound_sheet</code> (trunk and head)	<code>rubble_mound_notes</code>

Structure	draw	section	sheet	notes
Channel	draw_ channel, draw_ channel_ detail	channel_section	channel_ sheet	channel_ notes
Nourishment profile	draw_ nourishment	nourishment_ section	nourishment_ sheet	nourishment_ notes
Nourishment plan	draw_ nourishment_ plan	nourishment_ plan_section	via nourishment_ sheet(plan_ design=...)	
Pier scour	draw_pier_ scour	pier_scour_ section	pier_scour_ sheet	pier_scour_ notes
Bridge scour	draw_ bridge_ scour	bridge_scour_ section	bridge_ scour_sheet	bridge_ scour_notes

Supporting geometry: `MoundProfile(x_sea, x_land, crest, cot_sea, cot_land)` describes a trapezoidal mound, with `offset(thickness)` giving the exact perpendicular offset of a layer (the crest edges move by $t(\sqrt{1 + \cot^2 \alpha} - \cot \alpha)$), `band`, and `area`; `mound_layer_volumes(profiles, bed)` takes off the layer quantities. The underlayer follows the Rock Manual rule of a tenth of the armor mass, so its D_{n50} falls by $10^{1/3} \approx 2.15$. `vessel_outline` draws a midship section, `scour_hole_profile` a hole whose faces stand at the angle of repose (`REPOSE_ANGLE = 32` degrees), `bridge_bed_profile` the scoured bed across the whole waterway as one line, and `plan_margin` and `spreading_half_life` set the extents and times of a planform.

19.2.1 Scale and exaggeration

Sections are drawn at a true scale by default, fitted to a round standard scale that the title block then states. Where a true scale is unreadable, as for a channel 600 m wide and 20 m deep or a beach profile hundreds of meters long and a few meters deep, the exaggeration is applied deliberately and printed on the drawing next to the scale. A drawing set then carries a second view at true scale where the detail matters: the channel keel and underkeel allowances, and the seawall toe.

19.3 The drawing sheets

The worked examples of Part IV issue these sheets; they are collected here as the deliverables of the design chain.

The examples also write DXF files of the geometry (`media/seawall_section.dxf`, `media/breakwater_section.dxf`, `media/navigation_channel.dxf`), with each material on its own layer.

19.4 Plot helpers

`pyCoastal.plotting` holds the color maps and helpers shared by the examples: `water_colormap()`, a diverging map for the instantaneous surface built to look like water; `agitation_colormap()`, a sequential map for wave height or disturbance coefficient; `surface_norm(snapshots, percentile=99.5)`, a symmetric color scale centered on still water; `land_overlay()`

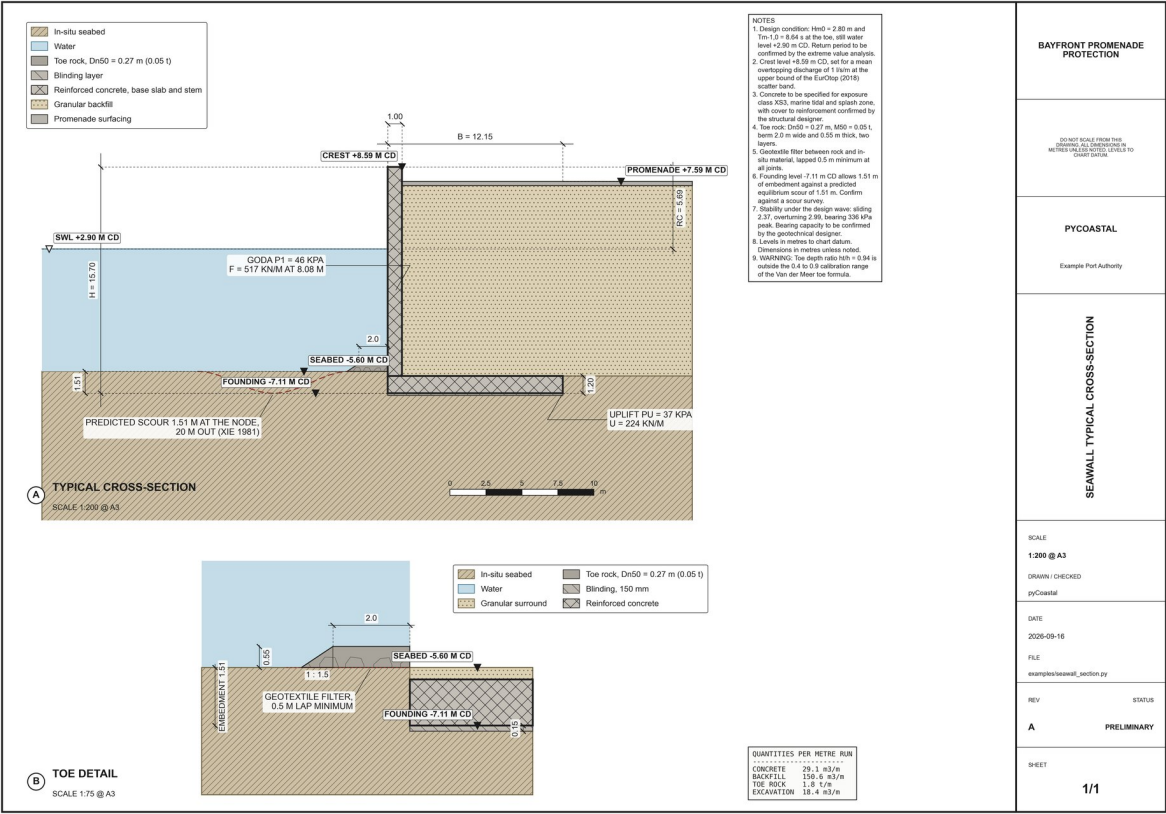


Figure 26: Seawall typical cross-section with toe detail (seawall_sheet).

land), a float array that is 1 on structures and NaN on water; and LAND_COLOR. For quantitative fields use agitation_colormap or a standard scientific map, and keep water_colormap for reading the wave pattern at a glance.

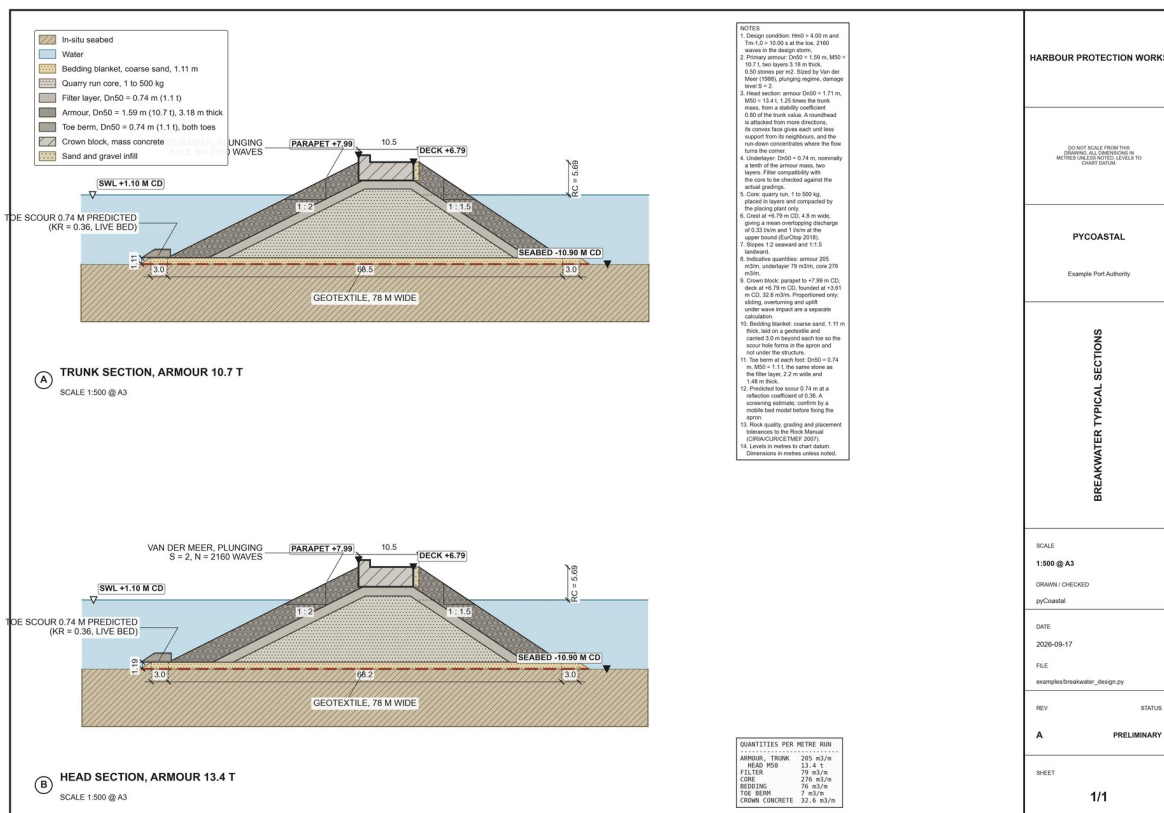


Figure 27: Breakwater trunk and head sections (rubble_mound_sheet).

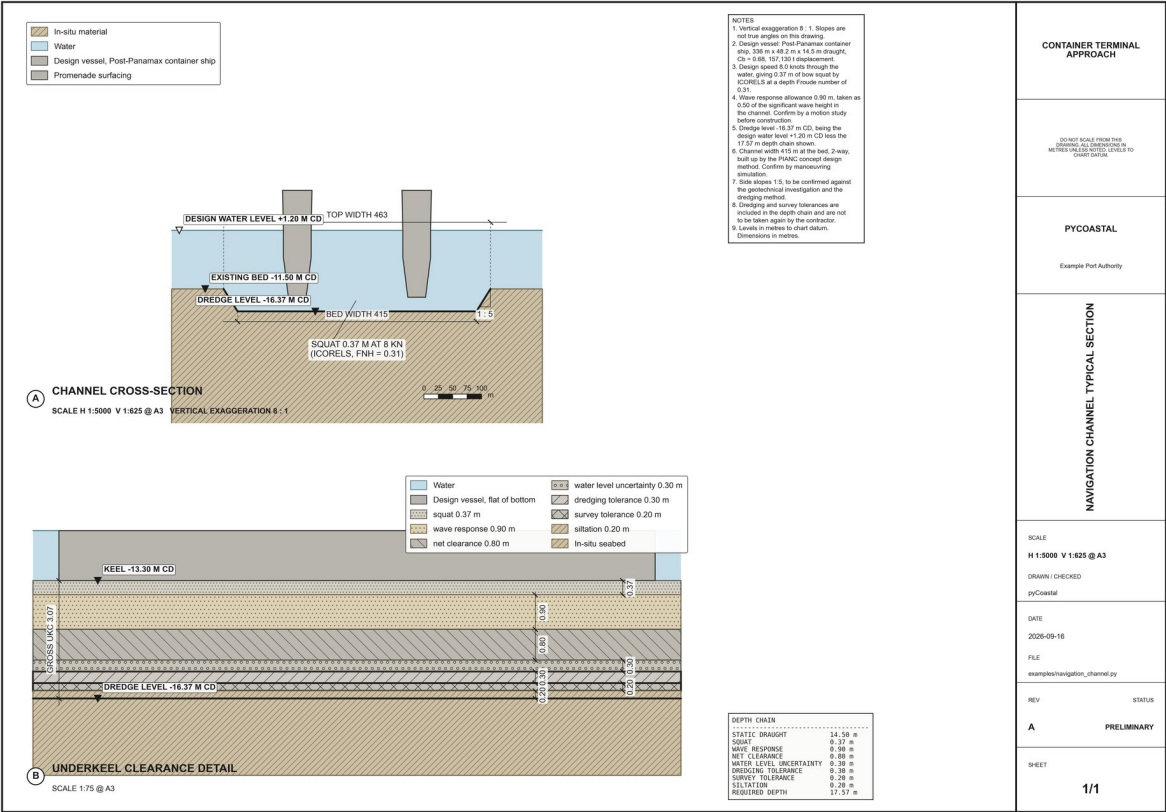


Figure 28: Navigation channel with true-scale keel detail (channel_sheet).

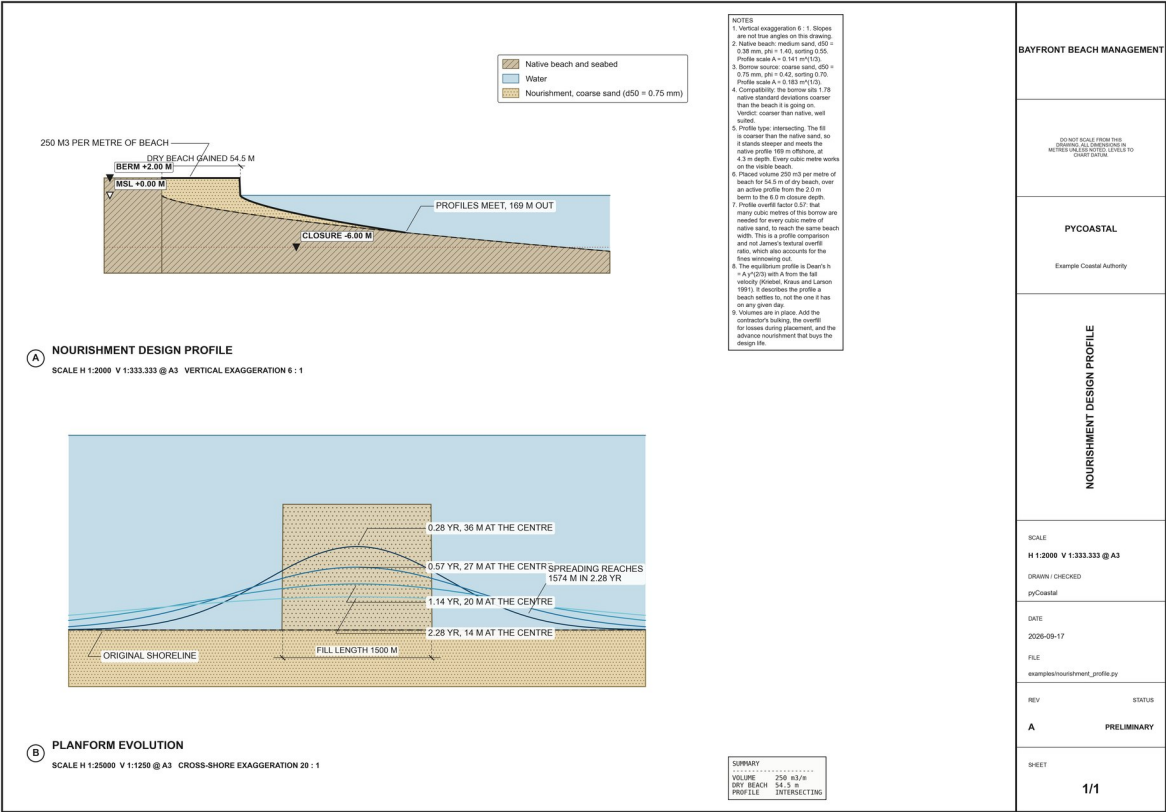
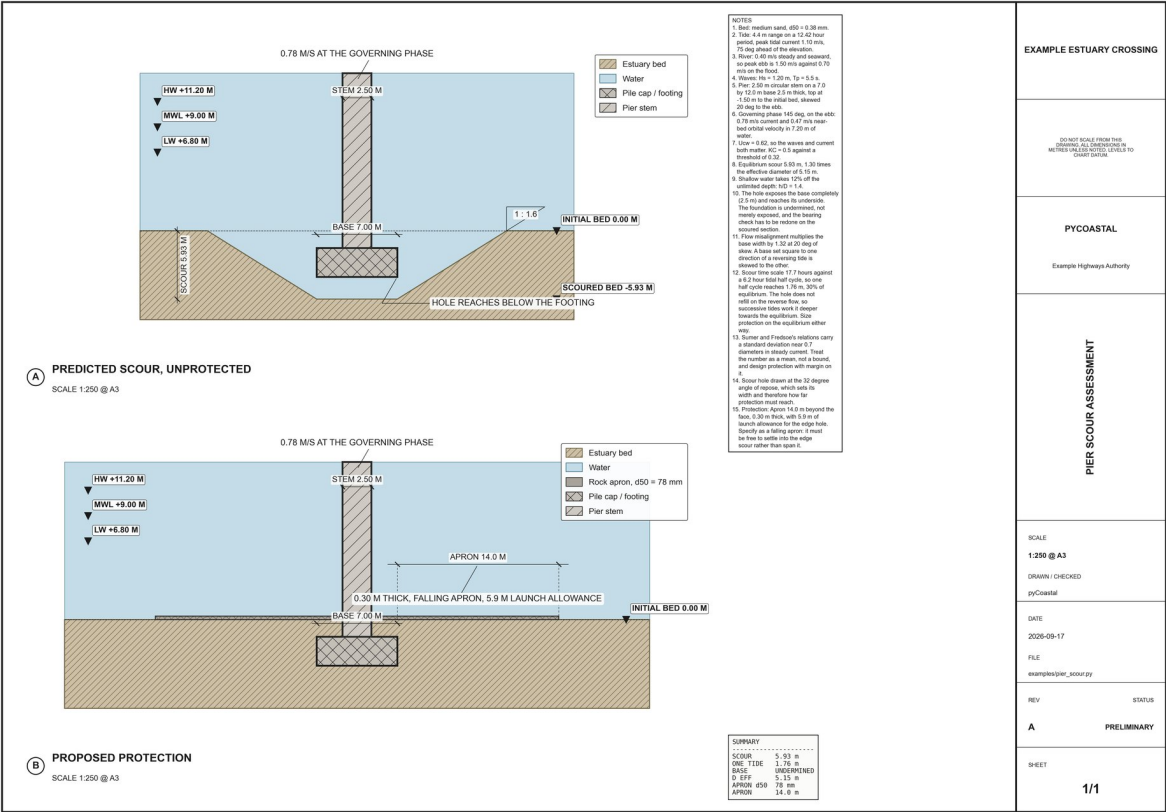


Figure 29: Nourishment design profile and planform (nourishment_sheet).



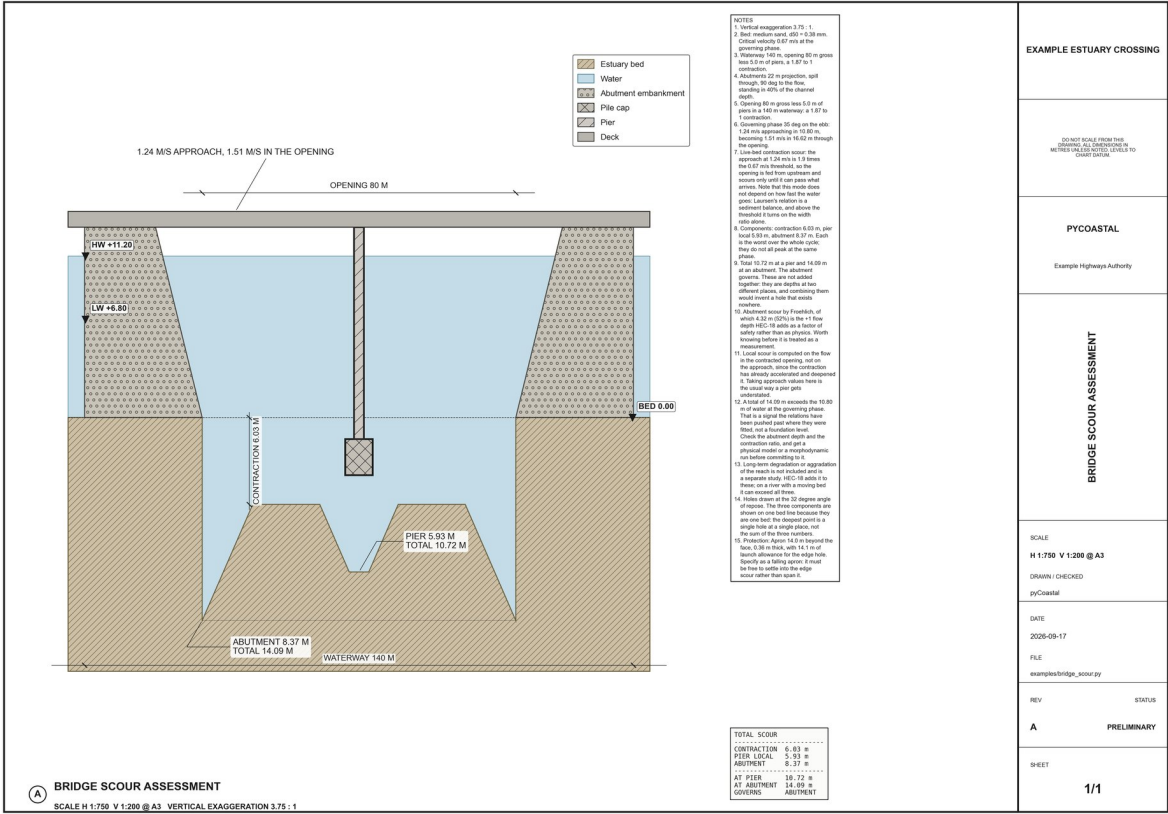


Figure 31: Bridge scour, all three components on one bed line (bridge_scor_sheet).

Part VI

PyCoaTools

20 The browser app

Directory: `webapp/`.

PyCoaTools is a static web page that runs the pyCoastal design modules in the browser, draws the result as a sheet, writes the design report, and puts next to it what the peer-reviewed literature says about the ground the relations stand on. Its tagline is the whole idea: size a structure with the pyCoastal relations, then read what the literature says about the ground they stand on.

20.1 How it works

A browser has no Python, so `webapp/engine.js` carries a JavaScript transliteration of the design modules, function for function: where the Python bisects, the JavaScript bisects; where the Python iterates on the base width, so does the JavaScript. Units follow the Python. A second implementation of a design code is worth having only if somebody checks it, so:

- `make_vectors.py` dumps reference cases straight out of pyCoastal into `vectors.json`, deliberately including awkward ones (the surging armor branch, an impulsive wall, a depth-limited Goda wave, drag- and inertia-dominated piles, a resultant outside the middle third);
- `verify_engine.py` runs `engine.js` in QuickJS against those cases and fails on any disagreement beyond floating-point noise;
- the page re-runs the same comparison on load and shows the result as a chip in the masthead, so the user sees it too.

`app.js` defines the modules, their inputs, the run functions, the report, and the checks; `draw.js` draws each module's sheet as SVG; and `index.html` is the application frame: a fixed masthead, an input panel, the drawing, the design report, and the Theory and sources panel, each scrolling on its own. Above 1560 px the report sits beside the drawing in its own pane. The page uses the fonts the desktop already has (Segoe UI and Consolas) and fetches no web font, so the first frame is correct.

20.2 Modules and inputs

The app has six modules. Each input is a slider with a range, a select, or a toggle, and the defaults follow the worked examples of Part IV.

Seawall

Gravity wall: crest from EurOtop, base from Goda pressures. (module key `seawall`)

Input	Label	Unit	Control	Range or options	Default
Hm0	Wave height	m	slider	0.5 to 8, step 0.1	2.8
	Hm0				
Tp	Peak period Tp	s	slider	4 to 18, step 0.1	9.5

Input	Label	Unit	Control	Range or options	Default
swl	Still water level	m CD	slider	-1 to 6, step 0.1	2.9
bed	Seabed level	m CD	slider	-20 to 2, step 0.1	-5.6
use	Tolerable overtopping	-	select	pedestrians_unaware, pedestrians_aware, harbour_quay_equipment, trained_staff, vehicles_low_speed	trained_staff
beta	Wave angle	deg	slider	0 to 60, step 5	0
scour	Scour coefficient	-	slider	0 to 0.6, step 0.05	0.4
backfill	Backfill material	-	select	silt, very_fine_sand, fine_sand, medium_sand, coarse_sand, fine_gravel, coarse_gravel, rock_fill, soft_clay, stiff_clay	medium_sand
table	Water table below surface	m	slider	0 to 16, step 0.5	0
surcharge	Surcharge on the promenade	kPa	slider	0 to 60, step 5	10

Breakwater

Rock armour by Van der Meer, crest by EurOtop. (module key **breakwater**)

Input	Label	Unit	Control	Range or options	Default
Hm0	Wave height Hm0	m	slider	0.5 to 9, step 0.1	4.0
Tp	Peak period Tp	s	slider	4 to 18, step 0.1	11.0
depth	Depth at toe	m	slider	2 to 40, step 0.5	12.0
cot	Seaward slope 1 :	-	slider	1.5 to 4, step 0.25	2.0
storm	Storm duration	h	slider	1 to 48, step 1	6
damage	Damage level S	-	slider	2 to 17, step 1	2

Input	Label	Unit	Control	Range or options	Default
use	Tolerable overtopping	-	select	pedestrians_unaware, pedestrians_aware, harbour_quay_equipment, trained_staff, vehicles_low_speed	trained_staff
armour	Armour type	-	select	rock_two_layer_permeable, rock_two_layer_impermeable, cubes_two_layer_random, tetrapod, accropode, xbloc	rock_two_layer_permeable
bed	Seabed material	-	select	silt, very_fine_sand, fine_sand, medium_sand, coarse_sand, fine_gravel, coarse_gravel, rock_fill, soft_clay, stiff_clay	medium_sand
section	Section	-	select	trunk, head	trunk

Channel

PIANC depth chain and width build-up. (module key `channel`)

Input	Label	Unit	Control	Range or options	Default
length	Vessel length	m	slider	80 to 400, step 5	336
beam	Beam	m	slider	12 to 62, step 0.5	48.2
draught	Draught	m	slider	3 to 20, step 0.1	14.5
cb	Block coefficient	-	slider	0.5 to 0.9, step 0.01	0.68
speed	Speed	kn	slider	3 to 14, step 0.5	8
wl	Design water level	m CD	slider	-1 to 4, step 0.1	1.2
Hs	Wave height in channel	m	slider	0 to 4, step 0.1	1.8
factor	Wave response factor	-	slider	0.2 to 0.8, step 0.05	0.5
net	Net clearance	m	slider	0.3 to 2, step 0.1	0.8
bed	Existing bed	m CD	slider	-25 to -5, step 0.5	-11.5
twoway	Two-way traffic	-	toggle	on / off	true

Input	Label	Unit	Control	Range or options	Default
material	Bed material	-	select	silt, very_fine_sand, fine_sand, medium_sand, coarse_sand, fine_gravel, coarse_gravel, rock_fill, soft_clay, stiff_clay	medium_sand

Monopile

Morison loads through the wave cycle. (module key `monopile`)

Input	Label	Unit	Control	Range or options	Default
diameter	Pile diameter	m	slider	0.5 to 12, step 0.1	8.0
H	Wave height H	m	slider	1 to 20, step 0.5	12
T	Period T	s	slider	4 to 20, step 0.5	13
depth	Water depth	m	slider	5 to 60, step 1	30
rough	Marine growth	-	toggle	on / off	true
bed	Seabed material	-	select	silt, very_fine_sand, fine_sand, medium_sand, coarse_sand, fine_gravel, coarse_gravel, rock_fill, soft_clay, stiff_clay	medium_sand

Nourishment

What a borrow source is worth, on Dean's equilibrium profile. (module key `nourishment`)

Input	Label	Unit	Control	Range or options	Default
native	Native beach sand	-	select	silt, very_fine_sand, fine_sand, medium_sand, coarse_sand, fine_gravel, coarse_gravel	medium_sand
borrow	Borrow source	-	select	silt, very_fine_sand, fine_sand, medium_sand, coarse_sand, fine_gravel, coarse_gravel	coarse_sand
volume	Placed volume	m ³ /m	slider	20 to 1200, step 10	250
berm	Berm height	m	slider	0.5 to 5, step 0.1	2.0
closure	Depth of closure	m	slider	2 to 14, step 0.5	6.0
view	View	-	select	profile, planform	profile

Input	Label	Unit	Control	Range or options	Default
length	Fill length (plan-form)	m	slider	200 to 6000, step 50	1500
Hb	Breaking wave height (plan-form)	m	slider	0.3 to 3.0, step 0.1	1.2

Return values

Generalised Pareto tail above a storm threshold. (module key **extremes**)

Input	Label	Unit	Control	Range or options	Default
threshold	Threshold	m	slider	0.5 to 8, step 0.1	2.5
scale	Scale	m	slider	0.1 to 3, step 0.05	0.55
shape	Shape	-	slider	-0.4 to 0.4, step 0.01	-0.08
rate	Peaks per year	-	slider	0.5 to 30, step 0.5	6
record	Record length	yr	slider	5 to 100, step 1	40
period	Design return period	yr	slider	5 to 500, step 5	100

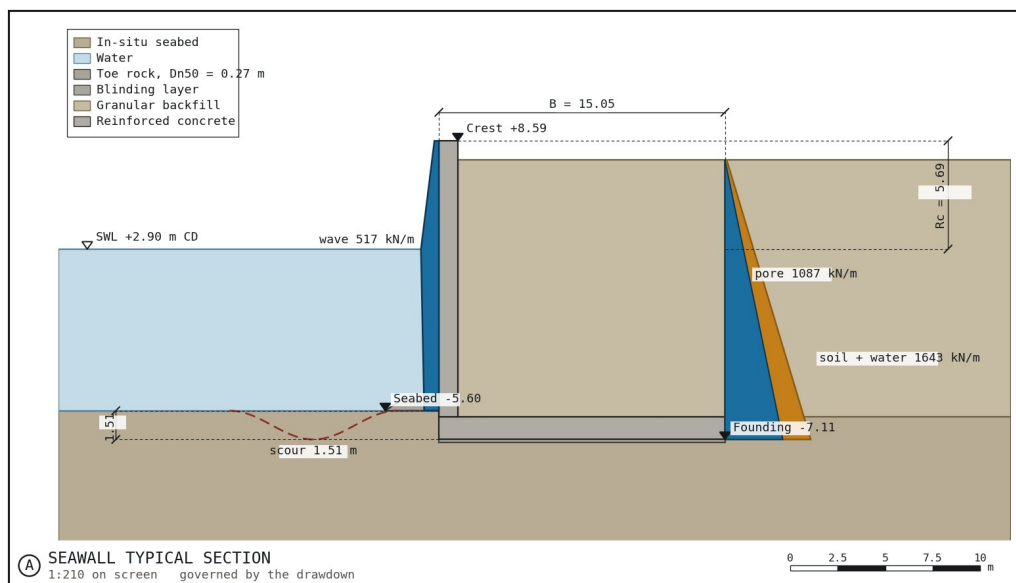


Figure 32: Seawall module sheet as rendered by **draw.js**.

20.3 Theory and sources

The Theory and sources panel reads **knowledge.json**, the extract of PyCoaPedia that **pedia/build_pedia.py** writes. The source database is about 150 MB of SQLite, which no browser will

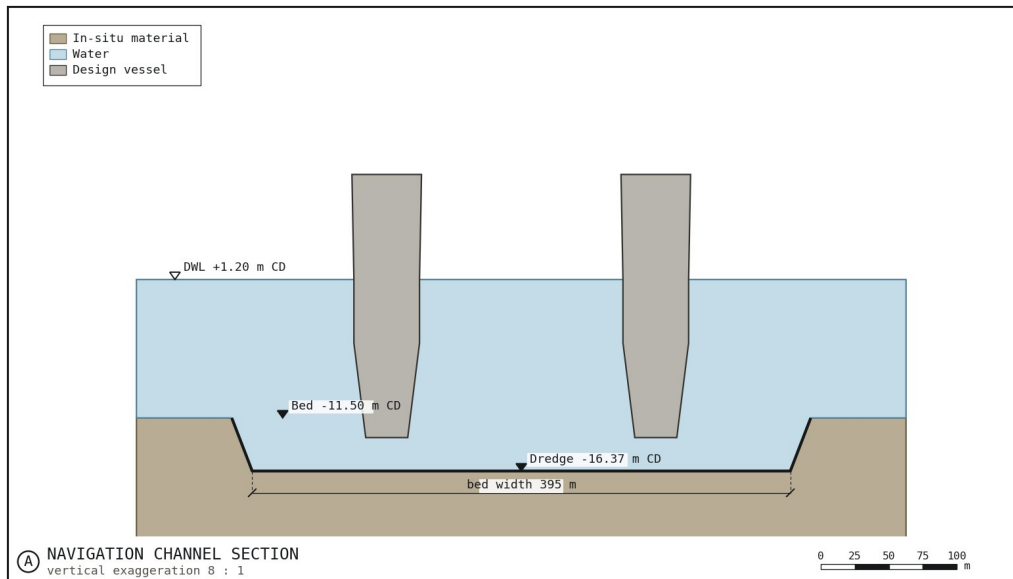


Figure 33: Channel module sheet.

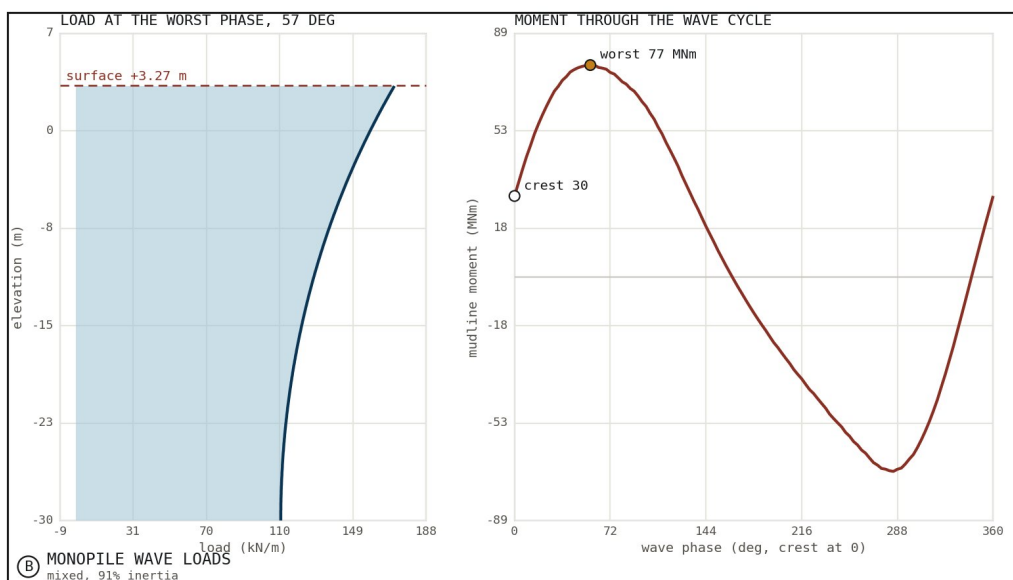


Figure 34: Monopile module sheet.

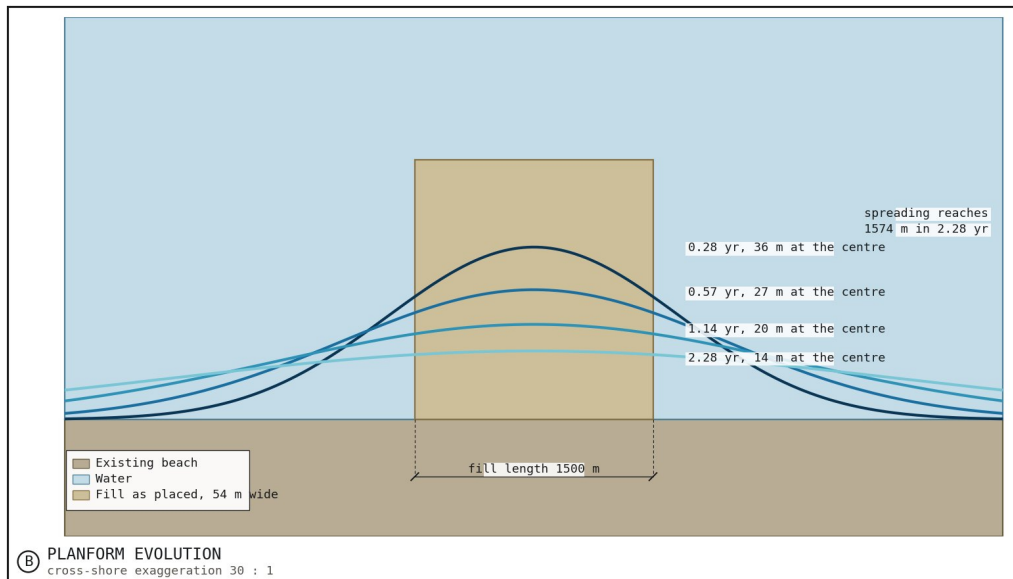


Figure 35: Nourishment module, planform view.

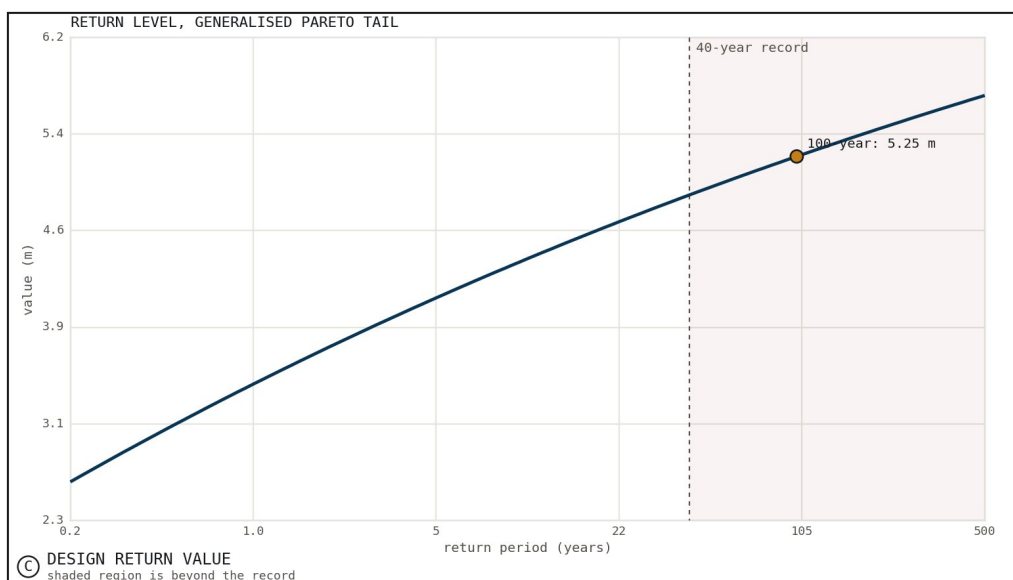


Figure 36: Return values module.

open; the app needs the topic tree, the curated synthesis for each topic, the atomic claims with their regime bounds, the equations, and enough paper metadata to cite and link, which comes to under 2 MB. Each design module is mapped to the PyCoaPedia topics that bear on it (the mapping is deliberately generous, because a design relation sits where several topics meet), and the panel shows each topic's synthesis, claims, and equations with a DOI behind every piece. Coverage is uneven by design: PyCoaPedia records what has been screened, not what exists. The PyCoaPedia button in the masthead opens the full explorer, described with the rest of the knowledge base in Section 21.

20.4 Verification and review tools

- `verify_app.py` loads the page's own scripts into QuickJS behind a stub DOM, so the module definitions, run functions, report and check builders, and in-page verification all execute at least once before publishing. It catches the things that break a static page: a typo in a function name, an input key that no run function reads, a check that throws on the defaults.
- `render_preview.py` runs `draw.js` in QuickJS behind a DOM stub that serializes to real SVG, then rasterizes it, because a script that runs can still draw nothing (a sheet that paints its background last passes every check and shows a blank page). Output goes to `media/webapp/`.
- `rasterise.py` draws the app's SVG with matplotlib, since the SVG is deliberately narrow (paths of M, L, H, V, Z plus rects, circles, and text). It is a review tool: is the thing on the page, in the right place, in the right order.

```
pip install quickjs svglib
python webapp/make_vectors.py --out webapp/vectors.json
python webapp/verify_engine.py
python webapp/verify_app.py
python webapp/render_preview.py --out media/webapp
```

To use the app locally, serve the folder (`python -m http.server` inside `webapp/`) and open `index.html`; the page loads `knowledge.json` and `vectors.json` with `fetch`, which needs a server rather than a file URL.

Part VII

PyCoaPedia and agentic retrieval

21 PyCoaPedia: the knowledge base

Every design relation in pyCoastal names a source and states where it stops being valid. PyCoaPedia holds the same shape of information for the wider literature, as data: the peer-reviewed coastal and ocean engineering literature distilled into a tree of topics, each with a curated synthesis, atomic claims that carry the regime they were established in, equations with their variables, and structured extractions of each paper, every piece traceable to a DOI. A relation quoted outside its range is the most expensive kind of mistake, and PyCoaPedia exists so that the range is always one query away.

21.1 What is in it

The build shipped with this version of pyCoastal was generated on 2026-09-22. It holds 156 topics in 26 top-level branches, 1,779 claims linked by 1,359 claim-to-claim relationships, 163 equations, and 1,655 papers, 1,655 of them with a structured extraction, drawn from 4,926 papers screened.

The knowledge is organized around four kinds of record.

Topics form a tree, from 26 top-level branches (waves, structures, scour, sediment transport, storm surge, ports, numerical modeling, and so on) down to specific subjects such as `structures.overtopping` or `waves.transformation.breaking.runup`. The dotted identifier is the path.

Synthesis is written per topic under fixed headings: what is well established, the governing physics, the dimensionless parameters, the major equations, typical methods, numerical models, experimental datasets, validated ranges, recent advances, disagreements, limitations, open questions, and the seminal papers. It is the place to start on a topic.

Claims are atomic findings. Each carries the regime it holds in, a confidence class (a direct finding, a literature review statement, an inferred relationship, or a proposed hypothesis), the kind of evidence (field, experimental, numerical, analytical, mixed, or review), the location in the paper, and the paper itself. Claims are linked to each other where one supports, contradicts, extends, validates, or compares against another, which is how disagreement in the literature becomes something that can be queried rather than remembered.

Papers carry their bibliographic record and abstract, the topics they are classified under, and a structured extraction: the research question, the principal results, the quantitative findings, the parameter ranges, the engineering implications, the limitations and the applicability conditions. Parameters, numerical models, datasets and field sites used by each paper are recorded as well.

21.2 How it is built

PyCoaPedia is distilled from a larger research database that an ingestion pipeline keeps growing: literature discovery through Crossref, OpenAlex and Semantic Scholar, relevance screening, classification into the topic tree, and extraction of claims, equations and paper summaries. That database also holds discovery logs, raw source records and full-text bookkeeping, and is not

shipped. `pedia/build_pedia.py` takes a snapshot of it and writes three things from one pass, so they cannot disagree (Figure 37):

- `pyCoastal/pedia/pycoapedia.sqlite`, the distilled database with a full-text index, shipped inside the Python package;
- `pedia/topics/*.md`, one Markdown page per topic, with an index (`pedia/README.md`), a module map (`pedia/modules.md`) and the schema (`pedia/SCHEMA.md`);
- `webapp/knowledge.json`, the extract the PyCoaTools app and the PyCoaPedia explorer load.

```
python pedia/build_pedia.py --db ../2026_CoastalWiki/data/coastalwiki.db
```

Rerunning it after the pipeline has added papers refreshes all three, and the tests in `tests/test_pedia.py` check the result.

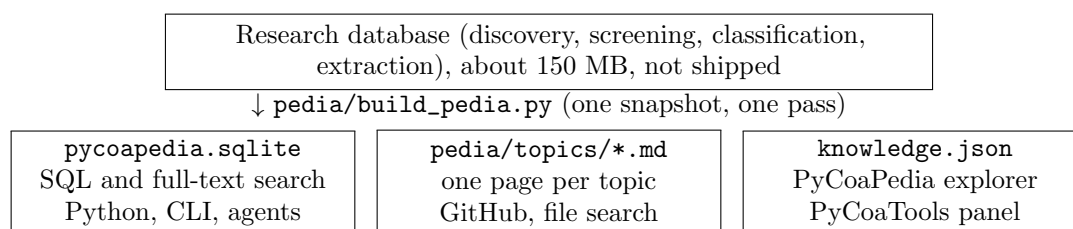


Figure 37: How PyCoaPedia is built and where each output is read.

21.3 Navigating the knowledge

The same knowledge can be reached five ways, and they suit different readers. A typical workflow runs from the design question to the evidence: start from the `pyCoastal` module you are using, read the synthesis of its topics (above all the validated ranges and the disagreements), then drill into the claims whose regime matches the case, and from a claim to its paper and the paper's quantitative findings.

21.3.1 In the browser: the PyCoaPedia explorer

`webapp/pedia.html` is a standalone page that loads the same extract as `PyCoaTools`. The topic tree is on the left with the number of claims on each branch; a search box searches topics, synthesis, claims and equations at once; and chips filter everything to the topics behind one `pyCoastal` design module. A topic page shows its breadcrumb and subtopics, the synthesis under its headings, the equations (typeset when `KaTeX` is reachable), and the claims, filterable by kind of evidence, each linking to its paper by DOI. Topic and claim views have their own URLs, so a finding can be shared as a link. From `PyCoaTools`, the `PyCoaPedia` button in the masthead and the Theory and sources panel beside each design lead to the same pages.

21.3.2 From Python

`pyCoastal.pedia` is a small read-only interface to the database, with no dependencies beyond the standard library:

```
from pyCoastal import pedia
```

```

pedia.stats()                                # build date and counts
pedia.tree("scour")                          # a branch of the topic tree
hits = pedia.search("overtopping shallow foreshore", limit=10)
t = pedia.topic("structures.overtopping")    # synthesis, claims, equations
t["synthesis"]["validated_ranges"]
pedia.claims("structures.overtopping", evidence="experimental")

```

```

pedia.related_claims(410)                                # supports, contradicts, ...
pedia.for_module("seawall")                              # topics behind a design module
pedia.paper(doi="10.1016/j.coastaleng.2016.07.002")

```

`search` runs on an FTS5 index with Porter stemming, so every word must appear but “overtopping” also finds “overtopped”; `kind=` restricts it to topics, synthesis, claims, equations or papers, `topic=` to one branch, and `raw=True` accepts the full FTS5 syntax (OR, NEAR, prefixes). For anything else, `pedia.connect()` returns a read-only connection to the database, and the environment variable `PYCOAPEDIA_DB` points the module at another build.

21.3.3 From the shell

```

python -m pyCoastal.pedia stats
python -m pyCoastal.pedia search "wave overtopping" --kind claim --limit 10
python -m pyCoastal.pedia tree structures
python -m pyCoastal.pedia topic structures.overtopping --claims
python -m pyCoastal.pedia module seawall
python -m pyCoastal.pedia paper --doi 10.1016/j.coastaleng.2016.07.002
python -m pyCoastal.pedia sql "SELECT id, label FROM topics WHERE level = 0"

```

Every command accepts `--json` for machine-readable output.

21.3.4 In SQL

The database is an ordinary SQLite file, so any client or language can query it. `pedia/SCHEMA.md` documents every table and its controlled vocabularies, with worked queries; two of them show the idea:

```

-- the validated ranges behind a pyCoastal design module
SELECT m.topic_id, s.statement
FROM module_topics m JOIN synthesis s ON s.topic_id = m.topic_id
WHERE m.module = 'seawall' AND s.section = 'validated_ranges';

-- where the literature disagrees
SELECT a.text, b.text, r.rationale
FROM claim_relationships r
JOIN claims a ON a.id = r.source_claim_id
JOIN claims b ON b.id = r.target_claim_id
WHERE r.relationship = 'contradicts';

```

21.3.5 As Markdown

`pedia/README.md` draws the whole topic tree with links, `pedia/modules.md` lists the topics behind each design module, and each page in `pedia/topics/` holds one topic: breadcrumb, subtopics, synthesis, equations in LaTeX, claims with their regime and citation, and the most relevant papers. The pages read on GitHub and suit tools that search files.

21.4 The design modules and their topics

Each design module of `pyCoastal`, and of `PyCoaTools`, is mapped to the topics whose literature bears on it (Table 19). The mapping is deliberately generous, because a design relation sits where several topics meet: an engineer setting a crest level wants the overtopping topic and the runup topic, not one of them.

Table 19: Design modules and their PyCoaPedia topics.

Module	Topics
Navigation channel (channel)	Navigation and vessel response (<code>ports.navigation</code>); Harbor sedimentation and dredging (<code>ports.sedimentation</code>); Mooring hydrodynamics (<code>ports.mooring</code>); Codes and design guidance (<code>design.guidance</code>)
Design conditions (extremes)	Extreme wave climates (<code>waves.generation.extremes</code>); Structural and system reliability (<code>uncertainty.reliability</code>); Aleatory and epistemic uncertainty (<code>uncertainty.sources</code>); Probabilistic hazard analysis (<code>risk.probablilistic</code>); Extreme-event observations (<code>field.extreme</code>)
Beach nourishment (nourishment)	Beach nourishment (<code>beaches.nourishment</code>); Longshore currents (<code>nearshore.longshore</code>); Planform evolution (<code>morphodynamics.planform</code>); Shoreline change (<code>beaches.change</code>)
Pile wave loads (piles)	Wave loads (<code>wave_structure.loads</code>); Impact and impulsive loads (<code>wave_structure.loads.impact</code>); Local scour at structures (<code>scour.structures</code>); Pipeline and cable scour (<code>scour.pipelines</code>)
Harbour agitation (port)	Harbor agitation and resonance (<code>ports.agitation</code>); Diffraction (<code>waves.transformation.diffraction</code>); Reflection (<code>waves.transformation.reflection</code>)
Seawall design (seawall)	Seawalls and revetments (<code>structures.seawalls</code>); Overtopping (<code>structures.overtopping</code>); Structural failure and damage (<code>structures.failure</code>); Local scour at structures (<code>scour.structures</code>); Scour protection (<code>scour.protection</code>); Design conditions and criteria (<code>design.criteria</code>)
Breakwater design (structures)	Breakwaters (<code>structures.breakwaters</code>); Overtopping (<code>structures.overtopping</code>); Transmission and reflection (<code>wave_structure.transmission</code>); Porous structures (<code>wave_structure.porous</code>); Performance-based design (<code>design.performance</code>)
Storm surge and flooding (surge)	Surge forcing (<code>storm_surge.forcing</code>); Wind stress and drag (<code>storm_surge.forcing.wind</code>); Atmospheric pressure forcing (<code>storm_surge.forcing.pressure</code>); Wave setup contribution to storm surge (<code>storm_surge.coupling.wave_setup</code>); Compound flooding (<code>storm_surge.compound</code>); Wave runoff (<code>waves.transformation.breaking.runup</code>); Inundation modeling (<code>inundation.models</code>); Flood-map uncertainty (<code>inundation.uncertainty</code>)

21.5 Coverage and its limits

Coverage is uneven by design: PyCoaPedia records what has been screened, not what exists. A topic with few claims is a topic that has not yet been read closely, not one where nothing is known, and the paper counts say how much of each branch has been screened. Claims are extracted from papers and checked, but a claim is only as good as the regime attached to it, and it is the regime that should be read first.

22 The repository for AI agents

pyCoastal is written for two kinds of reader. Engineers read this manual. AI agents, working on a design or answering a question on a user’s behalf, search the repository, and they work best

from plain text, structured data, and a clear map of where things are. The repository is laid out so that every piece of knowledge in it has a machine-readable form next to the human one (Table 20).

Table 20: Human and machine-readable forms of the same knowledge.

Knowledge	For people	For agents
Where everything is	this manual, <code>README.md</code>	<code>AGENTS.md</code> , <code>llms.txt</code>
Theory and usage	<code>pyCoastal manual.pdf</code>	<code>docs/manual/chapters/*.md</code>
API	the source	<code>docs/reference/*.md</code> , one file per module
Worked examples	Part IV	<code>examples/*.py</code> , <code>docs/examples.md</code>
Literature	PyCoaPedia explorer	<code>pycoapedia.sqlite</code> , <code>pedia/topics/*.md</code> , <code>python -m pyCoastal.pedia --json</code>

22.1 Entry points

AGENTS.md at the repository root is the map an agent should read first: what each directory holds, the conventions (units, levels, axis order), how to run the tests and the examples, how to query PyCoaPedia, and the rules the code follows (every relation names its source, nothing silently extrapolates, warnings are part of the result). **llms.txt** gives the same map in the compact format some tools look for.

The **manual chapters** in `docs/manual/chapters/` are Markdown with LaTeX mathematics, one file per chapter, which is exactly the source of this PDF; an agent can search them without parsing a PDF.

The **API reference** in `docs/reference/` is generated from the docstrings by the manual build, one Markdown file per module with every public class, function and constant, its signature, and its full docstring, so it cannot drift from the code.

The **example index** `docs/examples.md` lists every script in `examples/` with what it demonstrates and how to run it.

22.2 Querying the literature

For literature questions an agent should use the database rather than read Markdown pages one by one. A good pattern is search, then open, then cite:

```
python -m pyCoastal.pedia --json search "toe scour vertical wall" --limit 20
python -m pyCoastal.pedia --json topic scour.structures
python -m pyCoastal.pedia --json paper 2229
```

Every claim returned carries its regime and its source paper, with the DOI, so an answer built from PyCoaPedia can state where each statement holds and where it comes from. `pedia/SCHEMA.md` lists the tables, the controlled vocabularies, and worked SQL queries for anything the command line does not cover.

22.3 Keeping it current

`python docs/manual/build_manual.py` regenerates the PDF, the API reference and the example index; `python pedia/build_pedia.py --db <source>` refreshes the knowledge base in all

its forms. Both are deterministic, so their output belongs in version control and a diff shows exactly what changed.

Part VIII

Quality assurance

23 The test suite

A design code that nobody checks is a liability, so pyCoastal carries a test suite that grew with every module. It runs with

```
python -m pytest tests/
```

and in continuous integration on Ubuntu and Windows under Python 3.10, 3.11, and 3.13 on every push to `main` and every pull request.

23.1 What the tests check

The tests are not only regression checks. Most of them pin a result to an independent reference, and the kinds of check recur across modules:

- **Published values and limits.** The Van der Meer plunging and surging branches against the published forms; the Sumer and Fredsoe relation collapsing to the waves-only form at $U_{cw} = 0$ and to $S/D = 1.3$ in pure current; the toe scour recovering Xie at $K_r = 1$; Dean's profile scale against his table.
- **Analytical solutions.** The one-line solver against Pelnard-Considere; the port solver against the semi-infinite breakwater shadow; the fill volume and critical volume for matched sand; normal depth against Manning's equation.
- **Physical consistency.** Monotonic responses (more overtopping at lower crest, deeper scour at higher velocity), conservation (volume in the nourishment model), and conventions (axis order, units, sign of levels).
- **Honest failure.** Functions that must refuse (cohesive beds in scour, normal depth on an adverse slope, non-positive freeboard in the overtopping relations) raise, and functions that must warn outside their range do so.
- **Drawings.** Sheets reject unknown paper sizes and report their fitted scale, and DXF export round-trips one line per edge with every layer queued.

23.2 Tests by file

The suite holds 852 test functions in 25 files (parametrized tests expand to more cases when run).

Table 21: Test files and what they cover.

File	Tests	Scope
<code>test_applications_berthing.py</code>	49	Berthing energy and fender selection.
<code>test_applications_bridge_scour.py</code>	47	Contraction and abutment scour: the other two thirds of HEC-18.
<code>test_applications_channel.py</code>	46	Navigation channel design: vessel, squat, clearance, width and depth.
<code>test_applications_extremes.py</code>	51	Extreme value analysis: L-moments, fits, return values and diagnostics.

File	Tests	Scope
<code>test_applications_nourishment.py</code>	25	Beach nourishment simulator.
<code>test_applications_nourishment_profile.py</code>	30	Cross-shore nourishment: equilibrium profiles and borrow compatibility.
<code>test_applications_piles.py</code>	45	Morison wave loads on piles: kinematics, coefficients, loads and scour.
<code>test_applications_plan_view.py</code>	34	The nourishment planform: the analytical map and the scales it is drawn at.
<code>test_applications_port.py</code>	28	Port layout simulator.
<code>test_applications_port_layouts.py</code>	24	Built-in port layouts and oblique wave incidence.
<code>test_applications_river.py</code>	47	Open channel hydraulics: normal and critical depth, backwater, afflux.
<code>test_applications_scour.py</code>	76	Scour at a pier in combined waves, tide and river current.
<code>test_applications_seawall.py</code>	52	Vertical seawall design: Goda pressures, stability, scour and toe.
<code>test_applications_sections.py</code>	34	Section drawings: geometry, layer offsets, quantities and sheets.
<code>test_applications_sediment.py</code>	36	Sediment and soil: material properties, mobility, and earth pressure.
<code>test_applications_structures.py</code>	70	Armour sizing and wave overtopping.
<code>test_applications_surge.py</code>	48	Storm surge components and flood mapping.
<code>test_drafting.py</code>	23	Drawing primitives: geometry, scale, and the DXF export.
<code>test_numerics_grid.py</code>	8	Grid geometry and boundary-index bookkeeping.
<code>test_numerics_operators.py</code>	17	Finite-difference operators, checked against analytic derivatives.
<code>test_pedia.py</code>	10	PyCoaPedia: the shipped knowledge base and its reader.
<code>test_physics_conventions.py</code>	8	The whole package uses one array layout: x is axis 0, y is axis 1.
<code>test_tools_sediment.py</code>	13	Sediment transport formulae: closed-form checks and physical scaling.
<code>test_tools_structural.py</code>	16	Armour sizing, run-up and wave force formulae.
<code>test_tools_wave.py</code>	15	Wave kinematics and spectra, checked against analytic limits.

23.3 The browser engine

The JavaScript engine of PyCoaTools is checked against the Python by `webapp/verify_engine.py` on the reference cases of `webapp/make_vectors.py`, and the page repeats the comparison on load (Section 20). `webapp/verify_app.py` executes every module of the page once. These verifiers are not yet wired into continuous integration and are run by hand before publishing the app.

Appendices

A List of symbols

Symbol	Meaning	Unit
x, y, z	horizontal coordinates and elevation (positive up)	m
$t, \Delta t$	time and time step	s
$\Delta x, \Delta y$	grid spacings	m
$u, v, \mathbf{u}$	velocity components and vector	m/s
h, d	water depth	m
η	free-surface elevation, setup	m
g	gravitational acceleration, 9.81	m/s ²
ρ, ρ_s	water and sediment (or rock) density	kg/m ³
Δ	relative buoyant density $\rho_s/\rho - 1$	-
ν, ν_t	kinematic and eddy viscosity	m ² /s
k, ε, ω	turbulent kinetic energy, dissipation, specific dissipation	m ² /s ² , m ² /s ³ , 1/s
C_s	Smagorinsky constant	-
$\sigma(x), L_{sp}$	sponge damping rate and thickness	1/s, m
H, H_s, H_{m0}, H_b	wave height, significant height, spectral height, breaking height	m
$T, T_p, T_{m-1,0}$	period, peak period, spectral period	s
L, L_0, k, ω	wavelength, deep-water wavelength, wavenumber, angular frequency	m, m, 1/m, 1/s
c, c_g	phase and group celerity	m/s
ξ	surf similarity (Iribarren) number	-
U	Ursell number	-
$S(f), m_n$	variance density spectrum, spectral moment	m ² /Hz
α, β	structure slope angle; wave obliquity or beach slope	rad or deg
D_{n50}, M_{50}	nominal stone diameter and median mass	m, kg
P, S, N	notional permeability, damage level, number of waves	-
K_D	Hudson stability coefficient	-
R_c	crest freeboard	m
q	mean overtopping discharge	l/s/m
$\gamma_f, \gamma_\beta, \gamma_b, \gamma_v$	EurOtop roughness, obliquity, berm, and wall factors	-
p_1, p_3, p_4, p_u	Goda pressures and uplift	kPa
K_r, K_d	reflection and disturbance coefficients	-
C_d, C_m	Morison drag and inertia coefficients	-

Symbol	Meaning	Unit
KC	Keulegan-Carpenter number	-
U_c, U_m, U_{cw}	current, near-bed orbital velocity, current share	m/s, m/s, -
S, S_{eq}	scour depth, equilibrium scour depth	m
D_e	effective pier diameter	m
θ, θ_{cr}	Shields parameter and its critical value	-
d_{50}, D_*, w_s	median grain size, dimensionless grain size, fall velocity	m, -, m/s
ϕ', K_a, K_0	friction angle, active and at-rest pressure coefficients	deg, -
n, K, S_f	Manning coefficient, conveyance, friction slope	s/m ^{1/3} , m ³ /s, -
Fr	Froude number	-
F_{nh}	depth Froude number of a ship	-
C_b, L_{pp}, B, T	block coefficient, length between perpendiculars, beam, draught	-, m, m, m
C_m, C_e, C_s, C_c	PIANC berthing factors	-
A	Dean profile scale	m ^{1/3}
ε	longshore diffusivity (one-line model)	m ² /s
Q, K_{cerc}	longshore transport and CERC coefficient	m ³ /s
u, σ, ξ, λ	threshold, scale, shape, and rate of an extreme value fit	-

B Engineering references

The design relations implemented in pyCoastal, as cited in the module docstrings.

- Barrass, C. B. (1979). A unified approach to squat calculations for ships. *PIANC Bulletin*, 32.
- Breusers, H. N. C., Nicollet, G., and Shen, H. W. (1977). Local scour around cylindrical piers. *Journal of Hydraulic Research*, 15(3), 211-252.
- Brolsma, J. U., Hirs, J. A., and Langeveld, J. M. (1977). On fender design and berthing velocities. *PIANC 24th Congress*.
- BS 6349-4 (2014). *Maritime works. Code of practice for design of fendering and mooring systems*. British Standards Institution.
- Chorin, A. J. (1968). Numerical solution of the Navier-Stokes equations. *Mathematics of Computation*, 22(104), 745-762.
- Chow, V. T. (1959). *Open-Channel Hydraulics*. McGraw-Hill.
- CIRIA, CUR, and CETMEF (2007). *The Rock Manual: The Use of Rock in Hydraulic Engineering*, 2nd ed. CIRIA C683.
- Coles, S. (2001). *An Introduction to Statistical Modeling of Extreme Values*. Springer.
- Dean, R. G., and Dalrymple, R. A. (1991). *Water Wave Mechanics for Engineers and Scientists*. World Scientific.
- DNV (2010). *DNV-RP-C205: Environmental Conditions and Environmental Loads*. Det Norske Veritas.

- EurOtop (2018). *Manual on Wave Overtopping of Sea Defences and Related Structures*, 2nd ed. Van der Meer, J. W., Allsop, N. W. H., Bruce, T., De Rouck, J., Kortenhaus, A., Pullen, T., Schüttrumpf, H., Troch, P., and Zanuttigh, B.
- FEMA (2005). *Guidelines and Specifications for Flood Hazard Mapping Partners*. Federal Emergency Management Agency.
- Ferziger, J. H., and Perić, M. (2002). *Computational Methods for Fluid Dynamics*. Springer.
- Froehlich, D. C. (1989). Local scour at bridge abutments. *Proceedings, ASCE National Hydraulic Conference*, 13-18.
- Goda, Y. (1974). New wave pressure formulae for composite breakwaters. *Proceedings of the 14th International Conference on Coastal Engineering*, 1702-1720.
- Goda, Y. (2010). *Random Seas and Design of Maritime Structures*, 3rd ed. World Scientific.
- Gottlieb, S., Shu, C.-W., and Tadmor, E. (2001). Strong stability-preserving high-order time discretization methods. *SIAM Review*, 43(1), 89-112.
- Hasselmann, K., et al. (1973). Measurements of wind-wave growth and swell decay during the Joint North Sea Wave Project (JONSWAP). *Deutsche Hydrographische Zeitschrift*, A8(12).
- Henderson, F. M. (1966). *Open Channel Flow*. Macmillan.
- Hosking, J. R. M., and Wallis, J. R. (1997). *Regional Frequency Analysis: An Approach Based on L-Moments*. Cambridge University Press.
- Hudson, R. Y. (1959). Laboratory investigation of rubble-mound breakwaters. *Journal of the Waterways and Harbors Division*, 85(3), 93-121.
- Hydrologic Engineering Center (2016). *HEC-RAS River Analysis System, Hydraulic Reference Manual, Version 5.0*. US Army Corps of Engineers.
- James, W. R. (1975). *Techniques in evaluating suitability of borrow material for beach nourishment*. Technical Memorandum 60, US Army Coastal Engineering Research Center.
- Kriebel, D. L., Kraus, N. C., and Larson, M. (1991). Engineering methods for predicting beach profile response. *Proceedings of Coastal Sediments '91*, 557-571.
- Lagasse, P. F., et al. (2009). *Bridge Scour and Stream Instability Countermeasures*, HEC-23, 3rd ed. Federal Highway Administration.
- Laursen, E. M. (1960). Scour at bridge crossings. *Journal of the Hydraulics Division*, 86(2), 39-54.
- Launder, B. E., and Spalding, D. B. (1974). The numerical computation of turbulent flows. *Computer Methods in Applied Mechanics and Engineering*, 3(2), 269-289.
- LeVeque, R. J. (2002). *Finite Volume Methods for Hyperbolic Problems*. Cambridge University Press.
- Morison, J. R., O'Brien, M. P., Johnson, J. W., and Schaaf, S. A. (1950). The force exerted by surface waves on piles. *Petroleum Transactions, AIME*, 189, 149-154.
- Pelnard-Considère, R. (1956). Essai de théorie de l'évolution des formes de rivage en plages de sable et de galets. *4èmes Journées de l'Hydraulique*, Question III, 289-298.
- PIANC (2002). *Guidelines for the Design of Fender Systems*. Report of Working Group 33, Maritime Navigation Commission.
- PIANC (2014). *Harbour Approach Channels: Design Guidelines*. Report 121.
- Pierson, W. J., and Moskowitz, L. (1964). A proposed spectral form for fully developed wind seas. *Journal of Geophysical Research*, 69(24), 5181-5190.
- Pope, S. B. (2000). *Turbulent Flows*. Cambridge University Press.
- Richardson, E. V., and Davis, S. R. (2001). *Evaluating Scour at Bridges*, HEC-18, 4th ed. Federal Highway Administration.

- Seelig, W. N., and Ahrens, J. P. (1981). *Estimation of wave reflection and energy dissipation coefficients for beaches, revetments, and breakwaters*. Technical Paper 81-1, US Army Coastal Engineering Research Center.
- Smagorinsky, J. (1963). General circulation experiments with the primitive equations. *Monthly Weather Review*, 91, 99-164.
- Soulsby, R. L. (1997). *Dynamics of Marine Sands*. Thomas Telford.
- Soulsby, R. L., and Whitehouse, R. J. S. (1997). Threshold of sediment motion in coastal environments. *Proceedings of Pacific Coasts and Ports '97*, 149-154.
- Stockdon, H. F., Holman, R. A., Howd, P. A., and Sallenger, A. H. (2006). Empirical parameterization of setup, swash, and runup. *Coastal Engineering*, 53(7), 573-588.
- Sumer, B. M., Christiansen, N., and Fredsøe, J. (1992). Time scale of scour around a vertical pile. *Proceedings of the 2nd International Offshore and Polar Engineering Conference*, 308-315.
- Sumer, B. M., and Fredsøe, J. (2001). Scour around pile in combined waves and current. *Journal of Hydraulic Engineering*, 127(5), 403-411.
- Sumer, B. M., and Fredsøe, J. (2002). *The Mechanics of Scour in the Marine Environment*. World Scientific.
- Sumer, B. M., Fredsøe, J., and Christiansen, N. (1992). Scour around vertical pile in waves. *Journal of Waterway, Port, Coastal, and Ocean Engineering*, 118(1), 15-31.
- Takahashi, S., Tanimoto, K., and Shimosako, K. (1994). A proposal of impulsive pressure coefficient for design of composite breakwaters. *Proceedings of the International Conference on Hydro-Technical Engineering for Port and Harbor Construction*, 489-504.
- US Army Corps of Engineers (1984). *Shore Protection Manual*. Coastal Engineering Research Center.
- US Army Corps of Engineers (2002). *Coastal Engineering Manual*, EM 1110-2-1100.
- Van der Meer, J. W. (1988). *Rock slopes and gravel beaches under wave attack*. PhD thesis, Delft University of Technology; Delft Hydraulics Publication 396.
- Van Rijn, L. C. (1984). Sediment transport, part I: bed load transport. *Journal of Hydraulic Engineering*, 110(10), 1431-1456.
- Vasco Costa, F. (1964). The berthing ship: the effect of impact on the design of fenders and other structures. *The Dock and Harbour Authority*, 45.
- Wheeler, J. D. (1970). Method for calculating forces produced by irregular waves. *Journal of Petroleum Technology*, 22(3), 359-367.
- Wilcox, D. C. (1998). *Turbulence Modeling for CFD*. DCW Industries.
- Xie, S. L. (1981). *Scouring patterns in front of vertical breakwaters and their influence on the stability of the foundations of the breakwaters*. Delft University of Technology.
- Yarnell, D. L. (1934). *Bridge piers as channel obstructions*. Technical Bulletin 442, US Department of Agriculture.